

광학·영상 시스템 기초

장비 구조에서 Camera·Lens·FOV·Resolution·Illumination까지

-
- | | |
|----|---|
| 01 | Chapter 1. 검사 장비와 비전 SW 전체 구조 |
| 02 | Chapter 2. Image와 Pixel 기본 |
| 03 | Chapter 3. Camera Pixel Size와 실제 물체의 관계 |
| 04 | Chapter 4. Camera + Lens + Optical System |
| 05 | Chapter 5. FOV · Resolution · Magnification 집중 계산 |
| 06 | Chapter 6. Optical Resolution과 실제 검사 Resolution |
| 07 | Chapter 7. Illumination: 조명으로 검사 특징 만들기 |
-

원본 20개 Markdown의 내용·코드·표·수식을 보존하여 주제별로 편집

Chapter 1. 검사 장비와 비전 SW 전체 구조

Phase 0 · 예상 학습 시간: 8~10시간 · 난이도: 입문 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- 검사 장비의 `Camera → Result → Review/CIM` 전체 데이터 흐름
- Vision SW와 Equipment SW의 책임 경계
- Image Acquisition, Trigger, Exposure, Frame, Buffer
- Inspection Recipe, ROI, Measurement, Judgement, Result
- 이미지 좌표계(Image Coordinate)와 장비 좌표계(Machine Coordinate)
- 실패 처리, 추적성(Traceability), 재현성(Reproducibility)

실무 목표

학습 후에는 “이미지 한 장이 어떻게 생성되고, 어떤 단계를 거쳐 OK/NG 및 Review/CIM 데이터가 되는가?”를 모듈과 데이터 관점에서 설명할 수 있어야 한다. 또한 카메라 입력, 알고리즘 결과, 장비 상태를 한 클래스에 섞지 않고 책임을 나눠 간단한 C++ Pipeline을 설계할 수 있어야 한다.

2. 선수 지식

2.1 단위

- pixel: 이미지의 이산 좌표 및 샘플 단위
- μm , mm: 실제 물체 및 장비 이동 거리의 단위
- ms: Exposure와 처리 시간의 단위

`100 pixel` 은 그 자체로 실제 길이가 아니다. Calibration 또는 Object Space Pixel Resolution이 있어야 `몇  $\mu\text{m}$` 인지 알 수 있다.

2.2 C++ 관점

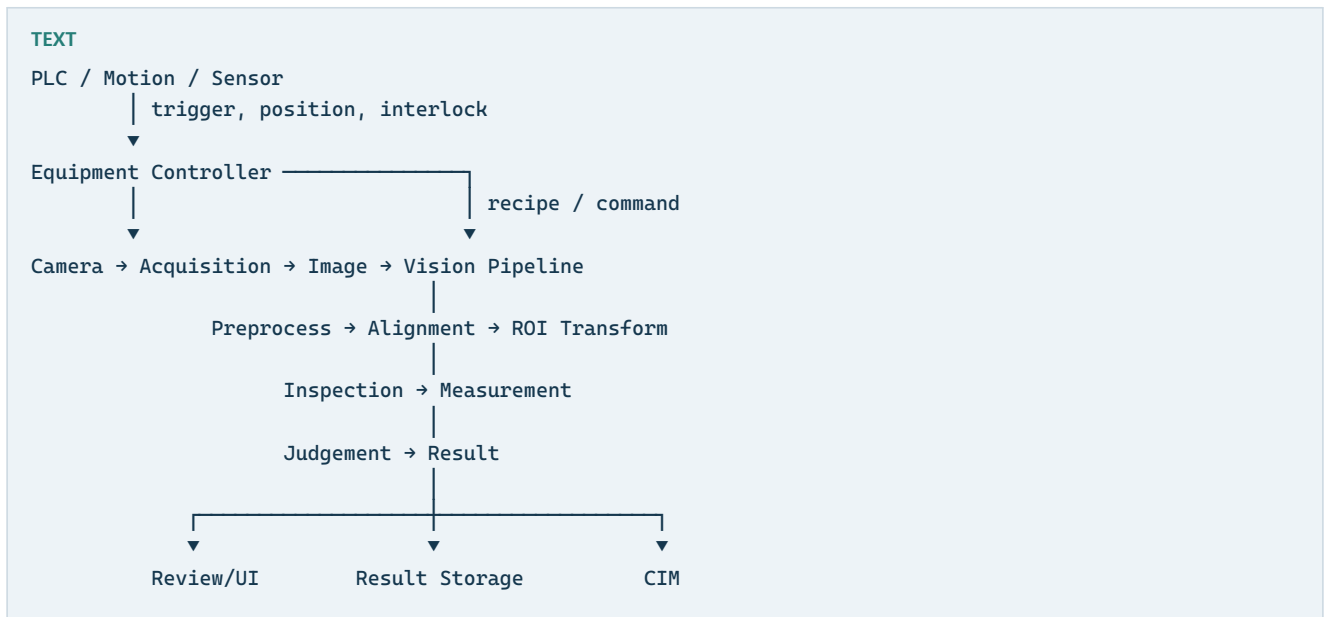
- `struct` : Recipe/Result처럼 데이터를 표현
- `class` : Camera/InspectionEngine처럼 상태와 동작을 캡슐화
- RAII: 카메라 핸들, 버퍼 등 자원을 객체 수명으로 관리
- `const` : 검사 입력과 Recipe가 의도치 않게 바뀌지 않도록 계약 표현
- 예외와 검사 결과: 시스템 실패와 제품 NG를 구분

[!WARNING]

제품이 불량인 `NG` 는 정상적인 검사 결과다. 카메라 연결 끊김, 잘못된 Recipe, 빈 Image 같은 상태는 검사 시스템 오류다. 둘을 같은 `bool false` 로 표현하면 복구와 추적이 어려워진다.

3. 핵심 개념

3.1 검사 장비의 일반적인 SW 구조



검사 장비 SW는 단지 이미지를 처리하는 프로그램이 아니다. 제품 투입, 축 이동, Trigger, 안전 Interlock, Recipe 전환, 검사, 배출, 결과 전송까지 하나의 상태 흐름을 관리한다.

3.2 Vision SW와 Equipment SW

구분	Vision SW	Equipment SW
주 책임	이미지로부터 위치·치수 결함을 추출	장비 시퀀스, Motion, I/O, 안전, 생산 흐름
대표 입력	Image, Recipe, Calibration	센서, PLC, 작업 지시, Vision Result
대표 출력	Align Pose, Measurement, OK/NG, Overlay	축 명령, 분류/배출, Host 보고, Alarm
시간 관심사	알고리즘 처리 시간, 이미지 품질	Cycle Time, Timeout, Interlock
실패 예	Focus 불량, Match 실패, 과노출	Servo Alarm, Trigger Timeout, 제품 미도착

경계는 제품마다 달라질 수 있지만 인터페이스는 명시해야 한다. 예를 들어 Vision이 반환하는 **X/Y/Theta**의 단위, 좌표 원점, 축 방향, 회전 부호, 신뢰도(Score)를 계약으로 고정한다.

3.3 Camera 제어와 Image Acquisition

1. Equipment Controller가 검사 위치 도착을 확인한다.
2. 조명 안정화 후 Trigger를 발생시킨다.
3. Camera가 Exposure 시간 동안 광자를 수집한다.
4. Sensor 데이터가 Camera Interface 또는 Frame Grabber를 통해 전송된다.
5. 수신 Buffer가 완전한 Frame인지 검증한다.
6. Frame ID, Timestamp, Recipe ID, Camera ID를 Image와 결합한다.
7. 소유권이 보장된 Image를 Vision Pipeline에 전달한다.

핵심 용어:

- **Trigger**: 촬영 시작을 지시하는 사건. Software/Hardware Trigger가 있다.
- **Exposure**: 센서가 빛을 모으는 시간. 길면 밝아지지만 Motion Blur 위험이 증가한다.
- **Frame**: 한 번의 촬영으로 생성된 완전한 이미지 단위.
- **Frame Grabber / Interface**: Camera Link, CoaXPress, GigE Vision, USB3 Vision 등의 영상을 호스트 메모리로 전달한다.
- **Buffer**: 전송 중이거나 처리 대기 중인 Frame의 메모리. 수명과 재사용 시점이 중요하다.
- **Image Acquisition**: Trigger부터 유효한 Frame과 Metadata를 애플리케이션에 넘기는 전체 과정.

3.4 Image Processing, Alignment, Inspection

- **Preprocessing(전처리)**: 노이즈 제거, 밝기 정규화, Contrast 개선 등 다음 단계가 안정적으로 동작하도록 Image를 준비한다.
- **Alignment(정렬)**: 제품의 기준 위치와 현재 위치의 차이인 X/Y/Theta 등을 구한다.
- **ROI Transform**: 기준 Recipe에 저장된 ROI를 Align 결과에 맞게 이동·회전한다.
- **Inspection(검사)**: Presence, 결함, 형상 조건 등을 판정할 특징을 찾는다.
- **Measurement(측정)**: 위치, 거리, 폭, 면적 같은 연속값을 계산한다.
- **Judgement(판정)**: 측정값과 Recipe Spec을 비교하여 OK/NG를 결정한다.

Measurement와 Judgement를 분리하면 Spec만 변경할 때 영상처리를 다시 설계하지 않아도 되고, Review에서 “왜 NG인가?”를 수치로 설명할 수 있다.

3.5 Recipe, Result, Review, CIM

Inspection Recipe

제품 종류별 검사 조건의 버전 관리된 집합이다.

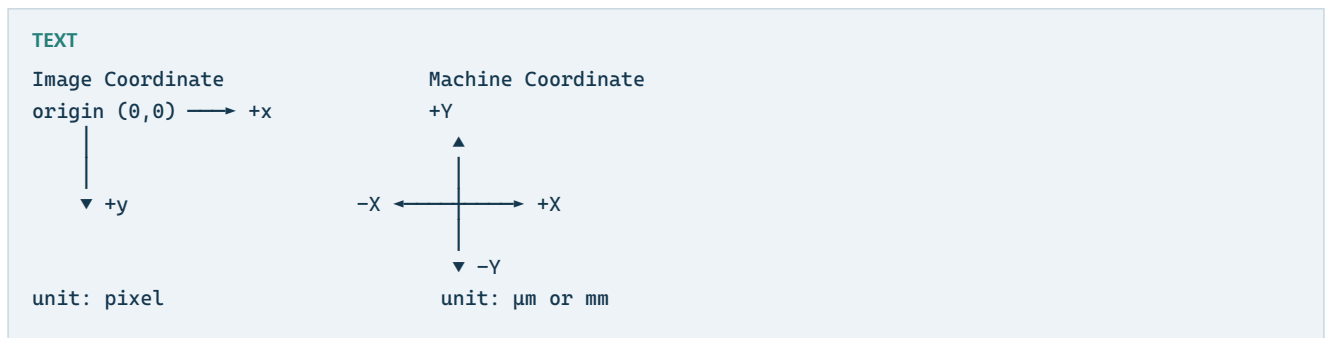
- Camera: Exposure, Gain, Trigger Mode
- Optical context: Camera/Lens/조명 구성 식별자
- Algorithm: Threshold, Filter Size, Match Score
- Geometry: 기준점, ROI, Calibration ID
- Judgement: Min/Max Spec

Result Data

최소한 다음을 남긴다.

- Lot/Panel/Product ID, Sequence ID
- Timestamp, Equipment/Camera ID
- Recipe 이름과 버전
- Calibration 버전
- Align X/Y/Theta와 Score
- 검사 항목별 Measurement, Spec, Judgement
- 전체 OK/NG 및 Error Code
- 원본/NG/Overlay Image 경로

3.6 좌표계



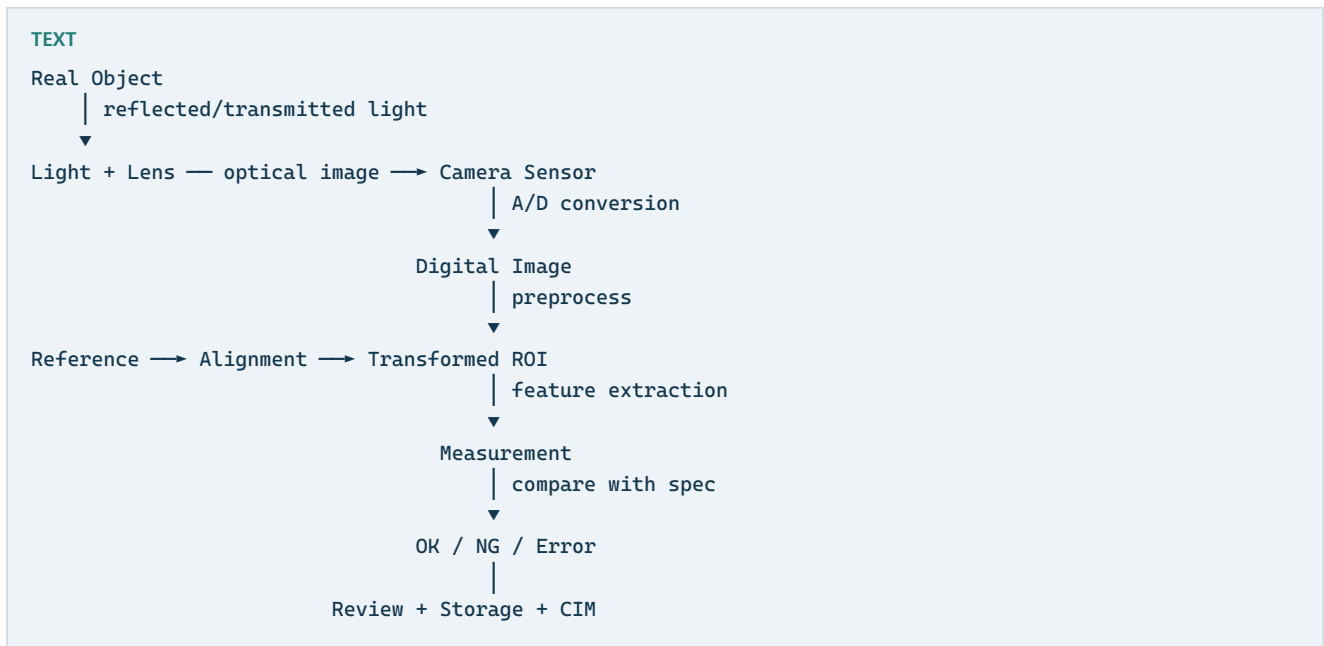
이미지는 보통 좌상단 원점, 오른쪽 +x, 아래쪽 +y다. 장비 좌표는 장비 설계에 따라 원점과 방향이 다르다. 그러므로 **pixel × scale** 만으로는 일반적인 좌표 변환을 완성할 수 없다. 원점 Offset, 축 반전, Rotation, Lens Distortion을 고려할 수 있다.

3.7 전체 흐름의 연결

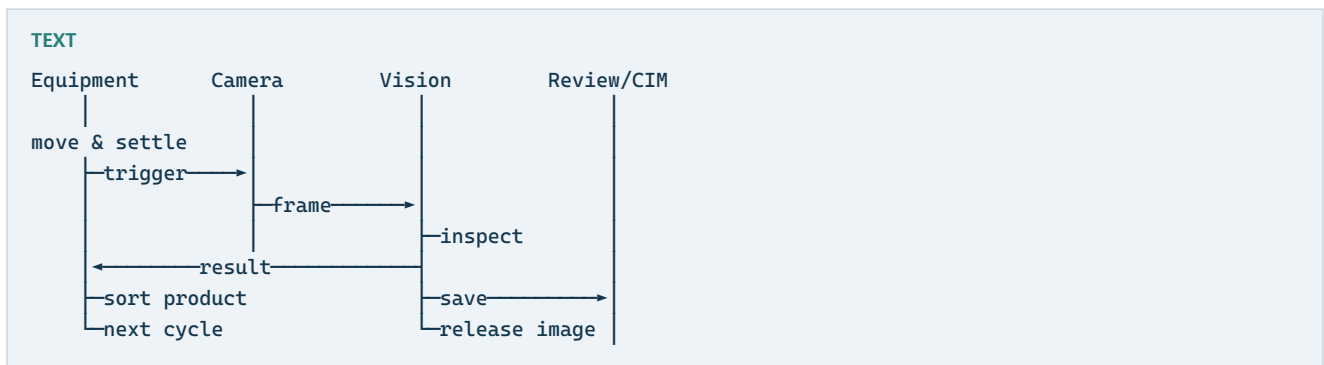


4. 그림으로 이해하기

4.1 물리 세계에서 Result까지



4.2 시퀀스 관점



5. 실제 검사 장비에서 어디에 사용하는가?

상황 A: 제품이 매번 조금씩 틀어져 들어온다

고정 ROI로 검사하면 실제 검사 부위가 ROI 밖으로 벗어난다. Pattern Matching으로 X/Y/Theta를 구하고 ROI Transform 후 검사해야 한다. 이때 Align 실패는 곧바로 제품 결함을 뜻하지 않는다. 이미지 품질, 제품 방향 오류, 기준 패턴 가림 등을 별도 Error/NG 정책으로 다뤄야 한다.

상황 B: Review Image와 실제 결과가 맞지 않는다

검사 Buffer를 Camera가 재사용한 뒤 Review Thread가 같은 메모리를 읽었을 수 있다. Frame을 복사하거나 참조 카운팅된 불변 Image로 전달하고, **Frame ID ↔ Product ID ↔ Result ID** 를 한 묶음으로 관리해야 한다.

상황 C: 카메라는 촬영했지만 이전 제품 이미지가 검사된다

Trigger Sequence와 Frame ID 검증이 없거나 비동기 Queue에 오래된 Frame이 남아 있을 수 있다. Trigger ID, Timestamp, Encoder/Stage Position을 Metadata로 묶고 예상 Sequence와 일치하는지 검사한다.

6. 숫자로 이해하기

예제 1: Cycle Time 예산

한 제품의 목표 Cycle Time이 **200 ms** 이고 다음 시간이 필요하다고 하자.

단계	시간
Motion settle	50 ms
Exposure	8 ms
Image transfer	12 ms
Preprocessing	15 ms
Alignment	35 ms
Inspection	45 ms
Result packaging	5 ms

총 시간:

FORMULA · CALCULATION

$$50 + 8 + 12 + 15 + 35 + 45 + 5 = 170 \text{ ms}$$

$$\text{Margin} = 200 - 170 = 30 \text{ ms}$$

현재 예산상 가능하지만 저장과 CIM 전송을 동기식으로 40 ms 수행하면 **210 ms** 가 되어 목표를 넘는다. 저장/전송은 Result의 무결성을 유지한 채 비동기화하는 설계를 검토할 수 있다. 단, 다음 제품과 ID가 섞이지 않도록 해야 한다.

예제 2: Frame 데이터 크기와 대역폭

2048 × 2048 , 8-bit Gray Image 한 장의 원시 크기:

FORMULA · CALCULATION

$$2048 \times 2048 \times 1 \text{ byte} = 4,194,304 \text{ byte} \approx 4 \text{ MiB}$$

초당 20 Frame이면 순수 Pixel 데이터만:

FORMULA · CALCULATION

$$4 \text{ MiB} \times 20 = 80 \text{ MiB/s}$$

카메라 4대라면 약 **320 MiB/s** 다. 실제 시스템은 전송 오버헤드, Buffer, 복사, 이미지 저장을 추가로 고려해야 한다. 모든 Frame을 여러 번 깊은 복사하면 메모리 대역폭과 지연이 커진다.

예제 3: 좌표의 첫 계산

Calibration 결과가 **1 pixel = 2 μm** 이고, 이미지에서 기준점 대비 오른쪽으로 **100 pixel** , 아래로 **50 pixel** 이 동했다고 하자. 장비 좌표가 오른쪽 +X, 위쪽 +Y라면:

FORMULA · CALCULATION

$$\text{Machine } \Delta X = +100 \times 2 \mu\text{m} = +200 \mu\text{m}$$

$$\text{Machine } \Delta Y = -50 \times 2 \mu\text{m} = -100 \mu\text{m}$$

아래 방향인 Image +y가 Machine -Y이므로 부호가 바뀐다. 실제 장비에서는 Rotation과 Offset까지 포함한 Calibration Matrix를 사용한다.

7. C++ 구현

아래 예제는 실제 SDK 대신 OpenCV Image를 받아 검사하는 최소 구조다. 핵심은 **Acquisition** , **Recipe** , **Engine** , **Result** 의 책임과 제품 NG/시스템 오류를 분리하는 것이다.

CPP · 1/3

```
#include <opencv2/opencv.hpp>

#include <algorithm>
#include <chrono>
#include <cstdint>
#include <optional>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

enum class Judgement {
    Ok,
    Ng
};

enum class ErrorCode {
    None,
    EmptyImage,
    InvalidRecipe
};

struct Frame final {
    cv::Mat image;
    std::uint64_t frameId{};
    std::string cameraId;
    std::chrono::steady_clock::time_point capturedAt;
};

struct InspectionRecipe final {
    std::string name;
    std::uint32_t version{};
    cv::Rect roi;
    int threshold{};
    double minArea{};
    double maxArea{};
};

struct Measurement final {
    std::string name;
    double value{};
    std::string unit;
    Judgement judgement{Judgement::Ng};
};
```

CPP · 2/3

```

struct InspectionResult final {
    std::uint64_t frameId{};
    std::string recipeName;
    std::uint32_t recipeVersion{};
    Judgement overall{Judgement::Ng};
    ErrorCode error{ErrorCode::None};
    std::vector<Measurement> measurements;
    cv::Mat overlay;

    [[nodiscard]] bool HasSystemError() const noexcept {
        return error != ErrorCode::None;
    }
};

class InspectionEngine final {
public:
    [[nodiscard]] InspectionResult Inspect(
        const Frame& frame,
        const InspectionRecipe& recipe) const
    {
        InspectionResult result;
        result.frameId = frame.frameId;
        result.recipeName = recipe.name;
        result.recipeVersion = recipe.version;

        if (frame.image.empty()) {
            result.error = ErrorCode::EmptyImage;
            return result; // 제품 NG가 아니라 시스템 입력 오류
        }

        if (!IsValidRoi(recipe.roi, frame.image.size()) ||
            recipe.minArea > recipe.maxArea) {
            result.error = ErrorCode::InvalidRecipe;
            return result;
        }

        cv::Mat gray;
        if (frame.image.channels() == 1) {
            gray = frame.image; // 입력은 const이므로 수정하지 않는다.
        } else {
            cv::cvtColor(frame.image, gray, cv::COLOR_BGR2GRAY);
        }
    }
};

```

CPP · 3/3

```

const cv::Mat roiView = gray(recipe.roi);
cv::Mat binary;
cv::threshold(roiView, binary, recipe.threshold, 255,
              cv::THRESH_BINARY);

std::vector<std::vector<cv::Point>> contours;
cv::findContours(binary, contours, cv::RETR_EXTERNAL,
                 cv::CHAIN_APPROX_SIMPLE);

double largestArea = 0.0;
for (const auto& contour : contours) {
    largestArea = std::max(largestArea, cv::contourArea(contour));
}

const bool areaOk = largestArea >= recipe.minArea &&
                    largestArea <= recipe.maxArea;
result.measurements.push_back({
    "largest_blob_area", largestArea, "pixel^2",
    areaOk ? Judgement::Ok : Judgement::Ng
});
result.overall = areaOk ? Judgement::Ok : Judgement::Ng;

cv::cvtColor(gray, result.overlay, cv::COLOR_GRAY2BGR);
cv::rectangle(result.overlay, recipe.roi,
              areaOk ? cv::Scalar(0, 255, 0)
                  : cv::Scalar(0, 0, 255),
              2);
return result;
}

private:
[[nodiscard]] static bool IsValidRoi(
    const cv::Rect& roi,
    const cv::Size& imageSize) noexcept
{
    const cv::Rect imageBounds{0, 0, imageSize.width, imageSize.height};
    return roi.width > 0 && roi.height > 0 &&
           (roi & imageBounds) == roi;
}
};

```

코드에서 봐야 할 점

1. `Frame` 은 Image와 Frame ID/Camera ID/Timestamp를 함께 전달한다.
2. `InspectionRecipe` 는 알고리즘 조건과 버전을 가진다.
3. `InspectionResult` 는 측정값, 판정, 오류, Overlay를 함께 보관한다.
4. `Judgement::Ng` 와 `ErrorCode` 를 분리했다.
5. 입력은 `const` 참조로 받아 수정하지 않는다.
6. ROI 유효성을 먼저 확인하여 OpenCV 내부 Assertion보다 의미 있는 오류로 처리한다.
7. 면적 단위는 아직 `pixel2` 다. 실제 단위 변환에는 Calibration이 필요하다.

간단한 테스트 예제

C++

```
#include <cassert>

void TestBrightBlobIsDetected()
{
    cv::Mat image = cv::Mat::zeros(200, 200, CV_8UC1);
    cv::rectangle(image, {80, 80, 20, 20}, cv::Scalar(255), cv::FILLED);

    const Frame frame{image, 1, "CAM-01", std::chrono::steady_clock::now()};
    const InspectionRecipe recipe{
        "sample", 1, {50, 50, 100, 100}, 128, 350.0, 450.0
    };

    const InspectionEngine engine;
    const InspectionResult result = engine.Inspect(frame, recipe);

    assert(!result.HasSystemError());
    assert(result.overall == Judgement::Ok);
    assert(result.measurements.size() == 1);
}
```

[!NOTE]

이 예제는 Architecture 입문용이다. 아직 Align, Dynamic ROI, Calibration, 저장 Queue, SDK Buffer 수명, Thread Safety는 구현하지 않았다. 이후 Chapter에서 단계적으로 확장한다.

8. 실무에서 발생하는 문제

문제 1: Trigger와 제품 ID가 어긋남

- 증상: 다른 제품의 Result가 현재 제품에 매핑됨
- 원인: 단순 FIFO 가정, Frame Drop, 재촬영, Timeout 후 늦게 도착한 Frame
- 대응: Trigger/Sequence/Frame/Product ID의 명시적 상관관계, Timeout 이후 Frame 폐기 정책, 로그에 ID 기록

문제 2: Camera Buffer 수명 오류

- 증상: 이미지가 간헐적으로 깨지거나 Review Image가 바뀜
- 원인: SDK가 Buffer를 재사용하는데 비동기 Consumer가 포인터를 계속 사용
- 대응: SDK의 소유권 규칙 확인, 필요 시 Clone, RAII Wrapper, 불변 Frame 전달

문제 3: Recipe가 검사 도중 변경됨

- 증상: 한 제품의 검사 항목마다 서로 다른 Parameter 적용
- 원인: 공유 Recipe 객체를 UI가 직접 수정
- 대응: 검사 시작 시 불변 Recipe Snapshot 사용, Version을 Result에 저장, 안전한 전환 시점 정의

문제 4: 조명·Exposure 변경 후 기존 Threshold가 실패

- 증상: 정상 제품 Blob 면적이 급변하거나 Align Score가 하락
- 원인: 영상 조건과 알고리즘 Parameter를 독립적으로 관리
- 대응: Camera/조명 조건을 Recipe에 포함하고 변경 이력 기록, Histogram/Reference Image 비교, 재검증

문제 5: 좌표계 부호·단위 오류

- 증상: 보정 명령이 반대 방향으로 움직이거나 1,000배 차이
- 원인: pixel/mm/μm 혼용, Image +y와 Machine +Y 혼동, Rotation 부호 미정의
- 대응: 좌표계 계약 문서, 단위를 타입/변수명에 표현, Known Point를 이용한 자동 테스트

문제 6: 저장 지연이 Cycle Time을 초과

- 증상: 검사 알고리즘은 빠르는데 장비 생산성이 낮음
- 원인: 원본/Overlay 압축 및 네트워크 저장을 검사 Thread에서 동기 실행
- 대응: 제한된 비동기 Queue, Backpressure/Drop 정책, NG 우선 저장, 저장 실패 Alarm과 재시도 정책

9. 흔한 오해

Frame 무결성, Metadata, Timeout, Buffer 수명, Sequence 검증까지 포함해야 한다.

1. **“Camera가 이미지를 주면 Acquisition은 끝이다.”**

Align은 매 제품의 자세 변화를 찾고, Calibration은 좌표계 사이의 지속적인 기하 관계를 정한다.

2. **“Align과 Calibration은 둘 다 좌표를 바꾸므로 같다.”**

NG는 제품에 대한 정상 판정이고 Error는 검사 신뢰성을 보장할 수 없는 시스템 상태다.

3. **“NG와 Error는 모두 검사 실패다.”**

처리량은 줄지만 제품 이동량과 Align 오차를 포함하지 않으면 특징이 잘릴 수 있다.

4. **“ROI를 줄이면 언제나 좋은 최적화다.”**

Recipe/Calibration Version, Overlay, 측정값, Spec, Align 결과가 있어야 재현 가능하다.

5. **“Review에는 NG Image만 있으면 된다.”**

원인 분석, CIM 보고, 재검, 통계에는 항목별 측정값과 상태가 필요하다.

6. **“검사 결과는 `bool` 이면 충분하다.”**

10. 면접에서 나올 수 있는 질문

Q1. 검사 장비의 Vision Pipeline을 설명해 주세요.

초보자가 이해할 수 있는 답변 카메라로 이미지를 얻고, 보기 좋게 전처리한 뒤 제품 위치를 맞춥니다. 그 위치에 검사 영역을 옮기고 특징과 크기를 측정한 다음 기준값과 비교해 OK/NG를 만듭니다. 결과와 이미지는 Review 및 상위 시스템으로 보냅니다.

실무자 답변 Trigger와 Exposure를 포함한 Acquisition에서 Frame/Metadata 무결성을 보장하고, Preprocessing, Alignment, ROI Transform, Feature Extraction, Measurement, Judgement 순으로 처리합니다. Result에는 Recipe/Calibration Version과 항목별 측정값, 오류 상태, Image 경로를 넣고 UI·Storage·CIM이 동일한 Result를 소비하게 합니다.

면접용 30초 답변 "장비가 Trigger를 주면 Camera Frame과 제품 ID를 결합하고, 전처리 후 Align으로 X/Y/Theta를 구합니다. 기준 ROI를 현재 제품 위치로 변환해 특징과 치수를 측정하고 Recipe Spec으로 OK/NG를 판정합니다. 확정 Result에는 측정값, Recipe/Calibration Version, Overlay 경로를 포함해 장비 시퀀스, Review, CIM이 동일하게 사용하도록 설계합니다."

Q2. Vision SW와 Equipment SW의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Vision SW는 이미지에서 위치와 결함을 찾고, Equipment SW는 장비가 제품을 옮기고 촬영하고 분류하는 전체 순서를 제어합니다.

실무자 답변 Vision은 Image/Recipe를 입력받아 Pose, Measurement, Judgement를 생성합니다. Equipment는 Motion, I/O, Interlock, Cycle, 제품 추적을 관리합니다. 두 영역 사이에는 좌표계, 단위, Timeout, Error 정책이 포함된 명시적 인터페이스가 필요합니다.

면접용 30초 답변 "Vision SW의 핵심 책임은 이미지로부터 정렬값과 검사 측정값을 만드는 것이고, Equipment SW는 Motion·I/O·안전·제품 흐름을 제어합니다. 실제 통합에서는 X/Y/Theta의 좌표계와 단위, Frame/Product ID, Timeout 및 Error 정책을 인터페이스 계약으로 고정하는 것이 중요합니다."

Q3. Align과 Calibration의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Align은 매번 달라지는 제품 위치를 찾는 것이고, Calibration은 이미지의 위치가 실제 장비의 어디인지 연결하는 기준을 만드는 것입니다.

실무자 답변 Align은 기준 제품 대비 현재 제품의 Pose, 주로 X/Y/Theta를 Frame마다 추정합니다. Calibration은 Image/Camera/Stage/Machine 좌표계 간 Scale, Rotation, Offset, 필요 시 Distortion 관계를 장비 설정 시 추정합니다. Align 결과를 Calibration된 좌표로 변환해 Motion 보정이나 실제 치수에 사용합니다.

면접용 30초 답변 "Align은 제품마다 발생하는 위치·회전 편차를 구하는 동적 보정이고, Calibration은 pixel 좌표와 실제 장비 좌표 사이의 Scale·회전·Offset을 정하는 좌표계 보정입니다. 예를 들어 Align에서 10 pixel 이동을 찾고 Calibration으로 이를 실제 μm 이동량과 장비 축 방향으로 변환합니다."

Q4. 제품 NG와 검사 시스템 Error를 왜 분리해야 하나요?

초보자가 이해할 수 있는 답변 NG는 카메라와 검사는 정상인데 제품이 기준을 벗어난 경우입니다. Error는 이미지 자체가 없거나 조건이 잘못되어 제품을 믿고 판정할 수 없는 경우입니다.

실무자 답변 둘을 분리해야 장비가 분류, 재검, 정지, Alarm 중 올바른 동작을 선택할 수 있습니다. 통계에서도 품질 불량률과 설비 가동률을 왜곡하지 않으며, CIM의 결과 코드와 장애 코드도 정확히 관리할 수 있습니다.

면접용 30초 답변 "NG는 유효한 검사 결과이고 Error는 유효한 판정을 만들 수 없는 시스템 상태입니다. 그래서 Result에 Judgement와 ErrorCode를 분리하고, NG는 제품 분류로, Camera Timeout이나 Invalid Recipe는 재시도 또는 장비 Alarm으로 처리합니다."

Q5. Recipe와 Result에 Version 정보가 필요한 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 나중에 같은 이미지를 다시 검사할 때 당시 어떤 조건을 썼는지 알아야 같은 결과를 확인할 수 있기 때문입니다.

실무자 답변 Threshold, ROI, Spec뿐 아니라 Camera/Light/Calibration 조건 변경이 결과에 영향을 준다. Result에 Recipe와 Calibration Version을 저장하면 NG 원인 분석, 변경 전후 비교, Audit, 재현 검사가 가능하다. 검사 중에는 Version이 고정된 Snapshot을 사용해야 한다.

면접용 30초 답변 “검사 결과의 재현성과 추적성을 위해서입니다. Result에 Recipe-Calibration Version을 남기고 검사 시작 시 불변 Snapshot을 사용하면, Parameter가 변경된 뒤에도 당시 ROI-Threshold-Spec-좌표 변환으로 결과를 재현하고 변경 영향을 분석할 수 있습니다.”

Q6. ROI는 왜 사용하나요?

초보자가 이해할 수 있는 답변 전체 이미지 중 검사할 부분만 잘라 처리하면 더 빠르고 엉뚱한 부분을 결함으로 찾을 가능성이 줄어듭니다.

실무자 답변 ROI는 계산량과 False Positive를 줄이고 검사 의도를 공간적으로 표현한다. 다만 고정 ROI는 제품 편차에 취약하므로 Align Pose로 Dynamic ROI를 만들고, 최대 이동·회전과 Transform 보간, Image 경계를 검증해야 한다.

면접용 30초 답변 “ROI는 검사 위치를 제한해 속도와 Robustness를 높입니다. 하지만 제품이 움직이면 고정 ROI가 실제 특징을 놓치므로 먼저 Align으로 X/Y/Theta를 구하고 기준 ROI를 변환합니다. 변환 후에는 이미지 경계와 충분한 Margin을 확인합니다.”

11. 실습 문제

실습 1: 현재 경험을 Pipeline에 매핑하기

본인이 개발한 CIM, Review, 장비 제어/UI 기능을 아래 단계에 배치한다.

TEXT

Acquisition → Preprocessing → Alignment → ROI → Inspection
→ Measurement → Judgement → Result → Review/UI/CIM

각 모듈에 대해 입력, 출력, 단위, Timeout, 실패 코드, 담당 Thread/Process를 표로 작성한다.

완료 조건

- 모듈 사이에 전달되는 ID를 정의했다.
- NG와 Error의 처리 경로가 구분된다.
- Image/Result의 소유권과 수명이 명시된다.

실습 2: 처리 시간과 Buffer 계산

다음 조건을 계산하고 설계를 제안한다.

- 4096×3000, 8-bit Gray
- 15 fps, Camera 2대
- Vision 처리 시간 평균 90 ms, 최대 160 ms
- Camera는 촬영을 계속하며 Frame Drop은 허용하지 않음

질문:

1. Frame 한 장과 초당 원시 데이터 크기는 얼마인가?
2. Vision이 직렬 처리라면 안정적으로 따라갈 수 있는가?
3. 순간 지연을 흡수할 Queue 크기를 어떤 근거로 정할 것인가?
4. 메모리가 무한 증가하지 않게 어떤 Backpressure 정책을 둘 것인가?

실습 3: C++ 설계 확장

Chapter 코드에 다음을 추가한다.

- `productId` , `sequenceId` , `calibrationVersion`
- 항목별 `lowerSpec` , `upperSpec` , `measuredValue`
- `AlignFailed` , `AcquisitionTimeout` Error
- Result를 JSON/CSV로 저장하는 별도 Writer Interface
- 빈 Image와 잘못된 ROI를 검증하는 Unit Test

UI나 CIM이 `InspectionEngine` 내부 상태를 직접 읽지 않고 확정된 `InspectionResult` 만 소비하게 한다.

실습 4: 장애 시나리오 설계

아래 상황별로 **Retry / Skip / NG / Error / Equipment Stop** 중 정책을 선택하고 이유를 적는다.

- Camera Trigger Timeout
- Pattern Match Score 미달
- ROI가 Image 경계를 벗어남
- Result DB 저장 실패
- CIM 전송 일시 실패
- Calibration Version 불일치

Phase 0 미니 프로젝트: Inspection Flow Simulator

실제 Camera 없이 폴더의 이미지를 Frame처럼 순서대로 읽어 다음을 구현한다.

1. `Frame ID + Product ID + Timestamp` 생성
2. Recipe Snapshot 선택
3. ROI Threshold 및 Blob Area 측정
4. OK/NG/Error 분리
5. Result CSV 저장
6. NG Overlay Image 저장
7. 처리 시간 측정과 로그 출력

권장 Class 구조

TEXT

```

InspectionFlowSimulator
├── ImageSource
├── RecipeRepository
├── InspectionEngine
├── ResultWriter
├── ImageWriter
└── CycleLogger

```

검증 시나리오

- 정상 Image → OK
- 기준 면적 밖 Image → NG
- 손상되거나 빈 Image → Error
- 잘못된 ROI Recipe → Error
- 같은 입력·Recipe → 같은 Measurement

12. Chapter 핵심 요약

- ● 검사 장비 SW는 촬영부터 Review/CIM까지 이어지는 전체 생산 Pipeline이다.
- ● Vision SW는 특징·측정·판정을, Equipment SW는 시퀀스·Motion-I/O·안전을 주로 담당한다.
- ● Frame에는 Image뿐 아니라 Product/Sequence/Camera/Recipe 식별 Metadata가 필요하다.
- ● Preprocessing → Alignment → ROI Transform → Inspection → Measurement → Judgement 순서를 구분한다.
- ● Align은 제품의 현재 Pose를, Calibration은 좌표계 사이 관계를 구한다.
- ● 제품 NG와 시스템 Error는 의미와 장비 대응이 다르다.
- ● Recipe Snapshot과 Version은 검사 일관성 및 재현성을 보장한다.
- ● Result는 UI, Storage, CIM이 함께 소비하는 단일 사실 공급원이어야 한다.
- ● 비동기 처리에서는 Buffer 수명, ID 상관관계, Queue 제한을 설계해야 한다.
- ● 좌표 변환에서는 단위, 원점, 축 방향, 회전 부호를 명시해야 한다.

학습 완료 체크

- ☐ 전체 Pipeline을 보지 않고 그릴 수 있다.
- ☐ Vision SW와 Equipment SW의 책임 차이를 설명할 수 있다.
- ☐ Trigger, Exposure, Frame, Buffer를 설명할 수 있다.
- ☐ Align과 Calibration의 차이를 30초 안에 설명할 수 있다.
- ☐ NG와 Error를 분리한 이유를 C++ 구조로 보여줄 수 있다.
- ☐ Recipe/Result Version이 필요한 이유를 설명할 수 있다.
- ☐ Phase 0 미니 프로젝트의 설계 또는 구현을 완료했다.

다음 Chapter 예고

다음 Chapter에서는 Image, Pixel, Gray Scale, RGB, Bit Depth, Dynamic Range를 배우고 Camera Resolution, Image Resolution, Pixel Size, Spatial Resolution, Optical Resolution을 명확히 구분한다.

Chapter 2. Image와 Pixel 기본

Phase 1 · 2일차 · 예상 학습 시간: 8~10시간 · 난이도: 입문 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- Image, Pixel, Width/Height, Pixel Coordinate
- Gray Scale, RGB, Channel
- Bit Depth와 8/10/12/16-bit Image
- Gray Level과 Dynamic Range
- Camera Resolution, Image Resolution, Pixel Size, Spatial Resolution, Optical Resolution의 차이
- Histogram, Saturation, Quantization
- OpenCV `cv::Mat`의 크기, 채널, Depth, Step

실무 목표

이미지의 `2048×2048`, `8-bit`, `3.45 μm pixel`이 각각 무엇을 뜻하는지 구분한다. Camera SDK가 전달한 Image의 Width, Height, Channel, Depth, Row Stride를 검증하고, 실제 길이 또는 검출 가능성을 Pixel 개수만 보고 단정하지 않는다.

2. 선수 지식

- `1 byte = 8 bit`
- `2^8 = 256`, `2^10 = 1024`, `2^12 = 4096`, `2^16 = 65536`
- `1 mm = 1,000 μm`
- 2차원 배열의 행(Row)과 열(Column)
- Chapter 1의 `Camera → Acquisition → Image → Inspection` 흐름

TEXT

빛 → Sensor의 각 Pixel → 전기 신호 → A/D 변환 → 숫자 배열(Image)

빛은 연속적이지만 디지털 Image는 공간과 밝기를 이산적인 값으로 표현한다. 공간을 나누는 단위가 Pixel이고, 밝기를 나누는 단계 수를 결정하는 것이 Bit Depth다.

3. 핵심 개념

3.1 Image와 Pixel

Image(영상)는 Pixel이 Width×Height 형태로 배열된 데이터다. **Pixel(Picture Element)**은 디지털 Image의 한 위치에 저장된 샘플이다.

FORMULA · CALCULATION

Image 4×3

```

      x=0  x=1  x=2  x=3
y=0  [ 12] [ 15] [ 18] [ 20]
y=1  [ 11] [120] [125] [ 19]
y=2  [ 10] [118] [122] [ 18]
```

`image.at<std::uint8_t>(y, x)` 처럼 메모리 접근은 보통 `(row, column) = (y, x)` 순서다. 반면 `cv::Point(x, y)` 와 `cv::Rect(x, y, width, height)` 는 `(x, y)` 순서다.

[!WARNING]

OpenCV에서 `(row, col)` 과 `(x, y)` 를 섞으면 사각형 Image에서는 오류가 바로 드러나지만 정사각형 Image에서는 우연히 정상처럼 보여 더 위험하다.

3.2 Gray Scale과 Gray Level

Gray Scale Image(그레이스케일 영상)는 각 Pixel에 밝기 값 하나를 저장한다. 8-bit라면 보통 `0=검정` , `255=흰색` 이며 그 사이 값은 회색이다.

- Gray Level: 한 Pixel의 수치
- Brightness: 사람이 느끼거나 영상에서 관찰되는 밝기
- Intensity: 문맥에 따라 Pixel 값 또는 빛의 세기를 뜻함

Pixel 값은 물체의 고유 속성만이 아니다. 조명 세기와 각도, 표면 반사, Lens, Exposure, Gain, Sensor Response의 결과다. 따라서 같은 물체도 촬영 조건이 바뀌면 Gray Level이 바뀐다.

3.3 RGB와 Channel

RGB Image는 일반적으로 Pixel마다 Red, Green, Blue 세 값을 저장한다. OpenCV의 기본 Color 순서는 대부분 **BGR**이다.

FORMULA · CALCULATION

한 Pixel: B=20, G=80, R=200

OpenCV Vec3b 접근: `pixel[0]=B`, `pixel[1]=G`, `pixel[2]=R`

산업 검사에서는 Color가 판별 특징이면 RGB/Color Camera를 사용한다. 형상, Edge, 치수, 표면 Contrast가 핵심이라면 Mono Camera가 해상도·감도·처리량 측면에서 유리할 수 있다. 무조건 Gray 변환하기 전에 검사 특징이 색에 있는지 판단한다.

3.4 Bit Depth

Bit Depth는 한 Pixel 또는 한 Channel이 표현할 수 있는 디지털 단계 수다.

Bit Depth	가능한 코드 범위	단계 수	일반적 저장
8-bit	0~255	256	<code>CV_8U</code> , 1 byte
10-bit	0~1023	1,024	보통 16-bit Container 또는 Packed Format
12-bit	0~4095	4,096	보통 16-bit Container 또는 Packed Format
16-bit	0~65535	65,536	<code>CV_16U</code> , 2 byte

10/12-bit Camera Image가 항상 Pixel당 정확히 10/12 bit로 메모리에 놓이는 것은 아니다.

- **Unpacked:** 16-bit Container에 저장. 접근은 쉽지만 여유 bit가 존재한다.
- **Packed:** 여러 Pixel을 bit 단위로 압축. 대역폭은 절약하지만 해제 과정이 필요하다.
- **정렬 방식:** 유효 bit가 상위(MSB) 또는 하위(LSB)에 정렬될 수 있다.

Camera SDK의 Pixel Format, 유효 Bit 수, 정렬 방식을 확인하지 않고 `uint16_t` 최댓값을 65535로 가정하면 Histogram과 Threshold가 틀어진다.

3.5 Dynamic Range와 Bit Depth의 차이

- **Bit Depth:** 디지털 코드가 몇 단계인가?
- **Dynamic Range:** 구분 가능한 가장 큰 신호와 가장 작은 신호의 비율은 얼마인가?

12-bit가 4096단계를 제공해도 Sensor Noise가 크면 모든 하위 단계를 유효하게 구분하지 못한다. 반대로 Dynamic Range가 넓어도 Display나 저장 과정에서 8-bit로 잘못 축소하면 정보가 손실될 수 있다.

TEXT

Bit Depth: 값을 담는 눈금의 개수

Dynamic Range: 실제로 의미 있게 측정 가능한 어두움~밝음의 범위

3.6 Saturation과 Quantization

- **Saturation(포화):** 신호가 표현 가능한 최댓값을 넘어 모두 같은 최대 코드로 잘린 상태
- **Underexposure(노출 부족):** 신호가 Noise Floor 근처에 몰려 특징 대비가 부족한 상태
- **Quantization(양자화):** 연속 신호를 제한된 코드 단계로 반올림하는 과정

포화된 영역은 물체가 더 밝아도 값이 증가하지 않으므로 내부 차이를 복구할 수 없다. 단순히 Contrast를 후처리로 키워도 잃어버린 정보는 돌아오지 않는다.

3.7 Resolution 용어 비교

용어	정확한 의미	대표 단위 / 표현	무엇이 결정하는가?	흔한 오해
Camera Resolution	Camera Sensor의 유효 Pixel 개수	2048×2048, 5 MP	Sensor 설계 및 출력 Mode	실제 1 pixel의 μm 크기라고 오해
Image Resolution	저장·전송된 Image의 Pixel 배열 크기	Width×Height	Camera Mode, ROI/Binning/Resizing	원본 Sensor 크기와 항상 같다고 오해
Pixel Size	Sensor에서 한 Physical Pixel의 Pitch	$3.45 \mu\text{m}/\text{pixel}$	Sensor 설계	물체 $3.45 \mu\text{m}$ 가 항상 1 pixel이라고 오해
Spatial Resolution	Object Space에서 Image 1 pixel이 담당하는 길이 또는 구분 가능한 공간 간격	$\mu\text{m}/\text{pixel}$, lp/mm	FOV/Pixel Count, Magnification 및 광학계	Pixel Size와 같은 값이라고 오해
Optical Resolution	Lens/광학계가 실제로 분리해 전달할 수 있는 세부 정도	μm , lp/mm, MTF	Lens, NA, 파장, Focus, 수차	Sensor Pixel 수가 높으면 자동 향상된다고 오해

꼭 구분할 문장

FORMULA · CALCULATION

Camera Resolution = 센서가 몇 개의 Pixel로 구성되었는가?
 Image Resolution = 지금 처리하는 Image 배열이 몇 Pixel인가?
 Pixel Size = 센서의 한 Pixel Pitch가 몇 μm 인가?
 Spatial Resolution = 물체 공간에서 한 Pixel 또는 구분 간격이 얼마인가?
 Optical Resolution = 광학계가 세부 정보를 얼마나 선명하게 전달하는가?

3.8 Image Size의 두 의미

Image Size 라는 말은 모호하므로 구분해서 말한다.

- Pixel Dimension: 2048×2048 pixel
- Memory Size: 약 4 MiB
- 화면 표시 크기: UI의 800×800 logical pixel
- 실제 FOV: 물체 공간의 10×10 mm

회의에서는 “Image Size가 크다” 대신 어떤 의미인지 단위와 함께 말한다.

3.9 Camera ROI, Binning, Resizing

- **Camera ROI**: Sensor 일부만 읽어 출력 Width/Height와 대역폭을 줄임
- **Binning**: 인접 Sensor Pixel의 신호를 결합. 출력 Pixel 수와 Sampling이 변함
- **Software Crop**: 이미 받은 Image의 일부만 참조/복사. Acquisition 대역폭은 그대로
- **Resize**: 보간으로 Image Pixel 배열을 변경. 새로운 실제 세부 정보가 생기지 않음

4. 그림으로 이해하기

TEXT



TEXT

공간을 나눔

FOV → Pixel Grid
10 mm / 2000 pixel
= 5 μm/pixel

밝기를 나눔

Analog Signal → Digital Code
0~100% → 0~4095 (12-bit)

공간 Sampling과 밝기 Quantization은 서로 다른 축이다. Pixel 수를 늘린다고 Bit Depth가 늘지 않고, Bit Depth를 늘린다고 작은 Defect가 더 많은 Pixel로 표현되지 않는다.

5. 실제 검사 장비에서 어디에 사용하는가?

Threshold 검사

8-bit Image에서 Threshold 128을 사용하던 코드를 12-bit 원본에 그대로 적용하면 거의 모든 Pixel이 흰색이 될 수 있다. 입력 Bit Depth와 유효 범위를 기준으로 Parameter 의미를 정해야 한다.

Review 표시

12/16-bit Image를 화면에 바로 표시하면 매우 어둡게 보일 수 있다. Display용 8-bit 변환과 검사 원본을 분리한다. Review를 보기 좋게 만든 변환 Image로 검사를 다시 수행하면 결과가 달라질 수 있다.

Camera 교체

같은 2048×2048 Camera라도 Pixel Size가 다르면 Sensor Size가 다르고, 동일 Lens에서 FOV 및 광학 조건이 달라질 수 있다. Camera Resolution만 같다는 이유로 동일한 검사 조건을 기대하면 안 된다.

Camera ROI와 Cycle Time

Sensor 전체가 필요하지 않다면 Camera ROI로 전송량을 줄여 Frame Rate를 높일 수 있다. 하지만 Align 범위와 제품 편차를 포함하는 충분한 영역을 확보해야 한다.

Color 검사

빨강과 초록 부품이 Gray 변환 후 비슷한 밝기가 되면 구분이 사라질 수 있다. Channel별 Histogram, HSV/Lab 변환 또는 적절한 광학 Filter와 조명을 검토한다.

6. 숫자로 이해하기

예제 1: 8-bit Gray Image 메모리

조건:

- 2048×2048
- 8-bit Gray, 1 Channel

FORMULA · CALCULATION

$$\begin{aligned} &2048 \times 2048 \times 1 \text{ byte} \\ &= 4,194,304 \text{ byte} \\ &= 4 \text{ MiB} \end{aligned}$$

20 fps이면 Pixel Payload만 약 **80 MiB/s** 다.

예제 2: 12-bit Image 저장 크기

조건:

- 4096×3000
- 12-bit Mono

16-bit Container에 Unpacked 저장하면:

FORMULA · CALCULATION

$$\begin{aligned} &4096 \times 3000 \times 2 \text{ byte} \\ &= 24,576,000 \text{ byte} \\ &\approx 23.44 \text{ MiB/frame} \end{aligned}$$

이상적으로 12-bit Packed라면:

FORMULA · CALCULATION

$$\begin{aligned}
 &4096 \times 3000 \times 12 / 8 \text{ byte} \\
 &= 18,432,000 \text{ byte} \\
 &\approx 17.58 \text{ MiB/frame}
 \end{aligned}$$

Packed가 약 25% 작지만 실제 Row Padding과 Protocol Overhead는 별도다.

예제 3: FOV로 Object Space Pixel Size 계산

조건:

- Image Width: 2000 pixel
- Horizontal FOV: 10 mm = 10,000 μm

FORMULA · CALCULATION

$$\begin{aligned}
 &\text{Object Space Pixel Size} \\
 &= 10,000 \mu\text{m} / 2000 \text{ pixel} \\
 &= 5 \mu\text{m/pixel}
 \end{aligned}$$

이미지에서 Edge 사이 거리가 240 pixel이면 이상적인 길이 계산은:

FORMULA · CALCULATION

$$240 \text{ pixel} \times 5 \mu\text{m/pixel} = 1,200 \mu\text{m} = 1.2 \text{ mm}$$

이는 Calibration Scale이 균일하고 왜곡·Perspective가 무시 가능한 경우의 계산이다.

예제 4: Bit Depth 변환

12-bit 유효 값 3072 를 전체 범위 기준으로 8-bit에 선형 Mapping하면:

FORMULA · CALCULATION

$$3072 / 4095 \times 255 \approx 191.30 \rightarrow 191$$

단순히 하위 8-bit만 취하면 전혀 다른 값이 된다. 변환 목적과 유효 Bit 정렬을 확인해야 한다.

7. C++ 구현

Image 정보와 통계 검사

CPP · 1/3

```
#include <opencv2/opencv.hpp>

#include <algorithm>
#include <cstdint>
#include <iostream>
#include <limits>
#include <stdexcept>
#include <string>

struct ImageStatistics final {
    double minimum{};
    double maximum{};
    double mean{};
    double standardDeviation{};
    double saturatedRatio{};
};

[[nodiscard]] std::string DescribeType(const cv::Mat& image)
{
    const std::string depth = [&image] {
        switch (image.depth()) {
            case CV_8U: return std::string{"CV_8U"};
            case CV_16U: return std::string{"CV_16U"};
            case CV_32F: return std::string{"CV_32F"};
            default: return std::string{"unsupported"};
        }
    }();

    return depth + "C" + std::to_string(image.channels());
}

[[nodiscard]] ImageStatistics CalculateMonoStatistics(
    const cv::Mat& image,
    const double validMaximum)
{
    if (image.empty()) {
        throw std::invalid_argument{"Image is empty"};
    }
    if (image.channels() != 1) {
        throw std::invalid_argument{"Mono image is required"};
    }
    if (validMaximum <= 0.0) {
        throw std::invalid_argument{"validMaximum must be positive"};
    }
}
```

CPP · 2/3

```

    double minimum = 0.0;
    double maximum = 0.0;
    cv::minMaxLoc(image, &minimum, &maximum);

    cv::Scalar mean;
    cv::Scalar standardDeviation;
    cv::meanStdDev(image, mean, standardDeviation);

    cv::Mat saturatedMask;
    cv::compare(image, validMaximum, saturatedMask, cv::CMP_GE);
    const double pixelCount = static_cast<double>(image.total());
    const double saturatedRatio =
        static_cast<double>(cv::countNonZero(saturatedMask)) / pixelCount;

    return {minimum, maximum, mean[0], standardDeviation[0],
            saturatedRatio};
}

[[nodiscard]] cv::Mat Convert12BitTo8BitForDisplay(const cv::Mat& image12)
{
    if (image12.empty() || image12.type() != CV_16UC1) {
        throw std::invalid_argument{"CV_16UC1 image is required"};
    }

    cv::Mat display8;
    constexpr double scale = 255.0 / 4095.0;
    image12.convertTo(display8, CV_8U, scale);
    return display8;
}

int main(int argc, char* argv[])
{
    if (argc != 2) {
        std::cerr << "Usage: image_info <image-path>\n";
        return 1;
    }

    const cv::Mat image = cv::imread(argv[1], cv::IMREAD_UNCHANGED);
    if (image.empty()) {
        std::cerr << "Failed to load image\n";
        return 2;
    }
}

```

CPP · 3/3

```

std::cout << "width=" << image.cols
          << ", height=" << image.rows
          << ", type=" << DescribeType(image)
          << ", element-bytes=" << image.elemSize()
          << ", row-step=" << image.step << '\n';

if (image.channels() == 1) {
    const double validMaximum = image.depth() == CV_8U ? 255.0 : 65535.0;
    const auto stats = CalculateMonoStatistics(image, validMaximum);
    std::cout << "min=" << stats.minimum
              << ", max=" << stats.maximum
              << ", mean=" << stats.mean
              << ", stddev=" << stats.standardDeviation
              << ", saturated=" << stats.saturatedRatio * 100.0
              << "%\n";
}
}

```

코드에서 봐야 할 점

1. `cols=Width` , `rows=Height` 다.
2. `depth()` 는 Channel 하나의 자료형이고 `type()` 은 Depth와 Channel을 결합한다.
3. `elemSize()` 는 한 Pixel의 전체 Byte 수다.
4. `step` 은 한 Row에서 다음 Row까지의 Byte 간격이며 `cols×elemSize()` 보다 클 수 있다.
5. 12-bit가 16-bit Container에 들어 있다면 `validMaximum=4095` 를 Camera Format에 맞게 전달해야 한다.
6. 검사 원본과 Display용 8-bit Image를 분리한다.

간단한 Unit Test

CPP

```
#include <cassert>
#include <cmath>

void TestEightBitStatistics()
{
    const cv::Mat image = (cv::Mat_<std::uint8_t>(2, 2) << 0, 255, 100, 200);
    const auto stats = CalculateMonoStatistics(image, 255.0);

    assert(stats.minimum == 0.0);
    assert(stats.maximum == 255.0);
    assert(std::abs(stats.mean - 138.75) < 1e-9);
    assert(std::abs(stats.saturatedRatio - 0.25) < 1e-9);
}

void TestTwelveBitDisplayMapping()
{
    const cv::Mat image12 =
        (cv::Mat_<std::uint16_t>(1, 3) << 0, 2048, 4095);
    const cv::Mat display8 = Convert12BitTo8BitForDisplay(image12);

    assert(display8.at<std::uint8_t>(0, 0) == 0);
    assert(display8.at<std::uint8_t>(0, 2) == 255);
}
```

[!NOTE]

12-bit 값이 16-bit 상위 Bit에 정렬된 Format이라면 먼저 정렬을 해석해야 한다. 위 함수는 유효 값이 **0~4095** 로 하위 정렬되어 있다는 전제다.

8. 실무에서 발생하는 문제

문제 1: 12-bit를 8-bit로 잘못 읽음

- 증상: 줄무늬, 밝기 왜곡, Width 이상
- 원인: Pixel당 Byte 수와 Packed/Unpacked Format 오해
- 대응: SDK Pixel Format, Stride, 유효 Bit 정렬 확인

문제 2: Row Padding 무시

- 증상: Image가 기울거나 다음 Row 시작점이 어긋남
- 원인: **width × bytesPerPixel** 만으로 Row 주소 계산
- 대응: SDK의 Pitch/Stride 또는 **cv::Mat::step** 사용

문제 3: RGB/BGR Channel 순서 혼동

- 증상: 빨강과 파랑이 뒤바뀌고 Color Threshold 실패
- 원인: SDK RGB와 OpenCV BGR의 차이
- 대응: Interface 경계에서 Pixel Format을 명시하고 변환 테스트

문제 4: 포화 Image에 후처리만 적용

- 증상: 밝은 부위의 Edge/Texture가 사라짐
- 원인: Exposure/Gain 과다로 원본 정보가 Clipping됨
- 대응: Exposure-조명 조정, Histogram과 Saturation Ratio 감시

문제 5: Resize Image로 치수 측정

- 증상: UI 표시 Image에서 측정한 값과 원본 검사값이 다름
- 원인: Display Resize Scale과 원본 좌표를 혼용
- 대응: 검사는 원본 좌표로 수행하고 Overlay만 Display Transform 적용

문제 6: Camera Resolution만으로 작은 결함 검출을 보장

- 증상: Pixel 수는 충분하지만 Defect Contrast가 낮거나 Blur됨
- 원인: Lens MTF, Focus, 조명, Noise와 필요한 Sampling 수 무시
- 대응: Object Space Sampling과 Optical Resolution을 함께 평가

9. 흔한 오해

Image의 Pixel은 샘플 위치다. 실제 물체에서 담당하는 크기는 Lens Magnification/FOV에 따라 달라진다.

1. **“Pixel은 실제 물체의 작은 정사각형이다.”**

3.45 μm 는 Sensor Pixel Pitch다. Object Space 값은 배율을 적용해야 한다.

2. **“3.45 μm Pixel Camera이면 물체 3.45 μm 가 1 Pixel이다.”**

Bit Depth는 밝기 단계이고 공간 Sampling 개수와 다르다.

3. **“12-bit는 8-bit보다 공간 해상도가 높다.”**

10/12-bit 값이 16-bit Container에 저장될 수 있다.

4. **“16-bit Container이면 항상 16-bit 유효 데이터다.”**

Pixel 배열은 커지지만 원본에 없던 광학 세부 정보가 생기지 않는다.

5. **“Resize로 Image를 두 배 키우면 해상도가 두 배 좋아진다.”**

조명, Exposure, Gain, Lens, Sensor 및 후처리의 결과다.

6. **“Pixel 값은 물체의 절대 밝기다.”**

8-bit Mono일 때의 Pixel Payload다. Channel, Container, Stride에 따라 달라진다.

7. **“2048×2048이면 항상 4 MiB다.”**

10. 면접에서 나올 수 있는 질문

Q1. Camera Resolution과 Pixel Size의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Camera Resolution은 센서에 Pixel이 몇 개 있는지이고, Pixel Size는 센서 Pixel 하나의 물리적 간격이 몇 μm 인지입니다.

실무자 답변 Resolution은 `Width×Height` Pixel Count이며 데이터 크기와 Sampling 개수를 좌우합니다. Pixel Size는 Sensor Pixel Pitch로 Sensor Size와 감도·광학 설계에 영향을 줍니다. 물체 기준 $\mu\text{m}/\text{pixel}$ 은 Pixel Size를 광학 배율로 나누거나 FOV를 Image Pixel 수로 나눠 계산합니다.

면접용 30초 답변 "Camera Resolution은 2048×2048처럼 Pixel 개수이고 Pixel Size는 3.45 μm 처럼 Sensor의 물리적 Pixel Pitch입니다. 실제 물체에서 1 pixel이 몇 μm 인지는 Pixel Size만으로 정할 수 없고 Lens Magnification 또는 측정된 FOV가 필요합니다."

Q2. Bit Depth가 높으면 무엇이 좋아지나요?

초보자가 이해할 수 있는 답변 밝기를 더 많은 단계로 나눠 미세한 밝기 차이를 표현할 수 있습니다.

실무자 답변 높은 Bit Depth는 Quantization 단계를 늘려 충분한 Sensor SNR과 Dynamic Range가 있을 때 미세 Contrast 보존에 유리합니다. 하지만 데이터 크기와 처리 비용이 증가하며, Noise가 크거나 포화되면 명목상 Bit 수가 실제 정보 증가로 이어지지 않습니다.

면접용 30초 답변 "Bit Depth는 한 Channel의 밝기 단계 수입입니다. 8-bit는 256단계, 12-bit는 4096단계라 미세 Contrast 보존에 유리하지만 공간 해상도가 증가하는 것은 아닙니다. 실제 이득은 Sensor Noise와 Dynamic Range, Exposure가 받쳐 줄 때 생깁니다."

Q3. Spatial Resolution과 Optical Resolution은 어떻게 다른가요?

초보자가 이해할 수 있는 답변 Spatial Resolution은 물체가 Image Pixel에 얼마나 촘촘히 나뉘는지이고, Optical Resolution은 Lens가 실제 세부 얼마나 선명하게 보여주는지입니다.

실무자 답변 Spatial Sampling은 보통 $\mu\text{m}/\text{pixel}$ 로 계산하지만 Optical Resolution은 Lens MTF, NA, 파장, Focus, 수차에 제한됩니다. Sampling이 촘촘해도 Optical Image가 Blur되면 세부를 복원할 수 없고, 광학이 좋아도 Sampling이 부족하면 Alias가 발생합니다.

면접용 30초 답변 "Spatial Resolution은 FOV/Pixel Count로 얻는 Object Space Sampling이고, Optical Resolution은 렌즈와 조명이 전달할 수 있는 실제 세부 한계입니다. 검사 가능성은 두 조건을 함께 봐야 하며 $\mu\text{m}/\text{pixel}$ 만으로 결함 검출을 단정하지 않습니다."

Q4. OpenCV에서 rows/cols 와 x/y 는 어떻게 대응하나요?

초보자가 이해할 수 있는 답변 `cols` 가 Width와 x 방향이고, `rows` 가 Height와 y 방향입니다. Pixel 배열 접근은 `(y, x)` 순서입니다.

실무자 답변 `cv::Mat::at(row, col)` 은 `(y, x)` 이며 `cv::Point` 와 `cv::Rect` 는 `(x, y)` 다. 비정사각형 Test Image와 경계값 테스트를 사용해 교환 오류를 검출하고, 좌표 타입/함수 이름으로 의미를 분명히 합니다.

면접용 30초 답변 "OpenCV에서 `cols=width=x축`, `rows=height=y축` 입니다. 다만 Mat Pixel 접근은 `at(y, x)` 이고 Point나 Rect 생성자는 `(x, y)` 이므로 혼동하기 쉽습니다. 저는 비정사각형 테스트와 ROI 경계 검증으로 이를 확인합니다."

Q5. 12-bit Image를 8-bit로 변환할 때 무엇을 확인해야 하나요?

초보자가 이해할 수 있는 답변 12-bit 값이 메모리에 어떤 방식으로 저장됐는지 확인하고 0~4095를 0~255 범위로 올바르게 줄여야 합니다.

실무자 답변 Packed/Unpacked, Container 크기, MSB/LSB 정렬, Black Level과 유효 범위를 확인합니다. 검사용 원본은 보존하고 Display/저장 목적에 맞게 고정 범위 또는 Windowing으로 변환합니다. Frame마다 Min-Max Normalize하면 Review 모양과 Threshold 의미가 흔들릴 수 있습니다.

면접용 30초 답변 “먼저 Pixel Format이 Packed인지, 16-bit Container인지, 유효 12-bit가 어느 쪽에 정렬됐는지 확인합니다. 그 후 검사용 원본은 유지하고 Display용으로 0~4095를 0~255에 Mapping합니다. Frame별 자동 Normalize는 영상 간 밝기 비교를 왜곡할 수 있어 목적에 맞게 사용합니다.”

Q6. 높은 Camera Resolution이면 작은 결함을 항상 검출할 수 있나요?

초보자가 이해할 수 있는 답변 아닙니다. Pixel이 많아도 Lens가 흐리거나 조명 Contrast가 부족하면 결함이 보이지 않습니다.

실무자 답변 결함 크기 대비 Object Space Sampling, Lens MTF와 Focus, 조명 Contrast, Sensor Noise, Motion Blur, 알고리즘에 필요한 Pixel 수를 함께 평가해야 합니다. 검출과 안정적 치수 측정에 필요한 Sampling도 다릅니다.

면접용 30초 답변 “아닙니다. Pixel Count는 Sampling 한 요소일 뿐입니다. 실제 검출에는 FOV 기준 $\mu\text{m}/\text{pixel}$, Lens MTF와 Focus, 조명 Contrast, Noise, Motion Blur 및 알고리즘이 요구하는 최소 Pixel 수가 모두 충족되어야 합니다.”

11. 실습 문제

실습 1: Image Inspector 구현

이미지 파일을 입력받아 다음을 출력한다.

- Width, Height, Channel, Depth, Type
- `elemSize` , `step` , 연속 메모리 여부
- Min, Max, Mean, Standard Deviation
- 8-bit Mono일 때 Histogram CSV
- Saturated Pixel 비율

빈 파일, 8-bit Gray, 8-bit BGR, 16-bit Mono로 테스트한다.

실습 2: 메모리와 대역폭 계산

다음은 각각 Unpacked 기준으로 계산한다.

1. 2448×2048, 8-bit Mono, 30 fps
2. 4096×3000, 12-bit Mono를 16-bit Container로 저장, 20 fps
3. 2048×2048, 8-bit BGR, 10 fps, Camera 4대

Pixel Payload와 초당 데이터양을 MiB 단위로 구하고, Image를 3회 Deep Copy할 때 메모리 Traffic이 어떻게 늘어나는지 설명한다.

실습 3: Bit Depth 변환 비교

12-bit Gradient Image를 만들고 다음 방법을 비교한다.

- 고정 범위 `0~4095` → `0~255`
- 특정 검사 범위 `1000~2500` → `0~255`
- Frame별 Min-Max Normalize
- 단순 Bit Shift

각 방법이 Review, Threshold, Frame 간 비교에 미치는 영향을 기록한다.

실습 4: Resolution 용어 설명

아래 사양을 보고 5개 Resolution 용어를 각각 한 문장으로 설명한다.

TEXT

Camera: 2048×2048, Pixel Size 3 μm
 Lens Magnification: 0.5×
 Saved Image: Camera ROI 적용 후 1024×768

아직 계산하지 못하는 값은 “왜 정보가 부족한가?”를 함께 적는다.

Phase 1 미니 프로젝트: Image Metadata & Quality Inspector

C++17/OpenCV로 다음을 구현한다.

TEXT

```
Image Load
  ↓
Metadata Validation
  ↓
Type / Size / Stride Report
  ↓
Histogram / Statistics
  ↓
Underexposure / Saturation Warning
  ↓
Display Conversion
  ↓
CSV + Preview Save
```

필수 조건

- 원본 Image를 수정하지 않는다.
- 8-bit/16-bit Mono와 8-bit BGR을 구분한다.
- 검사 통계와 Display 변환을 별도 함수로 만든다.
- 지원하지 않는 Type은 명확한 Error로 반환한다.
- 정상·빈 Image·포화 Image Unit Test를 작성한다.

12. Chapter 핵심 요약

- ● Image는 Width×Height의 Pixel 배열이며 Pixel은 디지털 샘플이다.
- ● OpenCV에서 `cols=x=Width` , `rows=y=Height` , 배열 접근은 `(y,x)` 다.
- ● Bit Depth는 밝기 단계이며 공간 해상도와 다르다.
- ● 10/12-bit Image는 16-bit Container 또는 Packed Format일 수 있다.
- ● Camera Resolution, Image Resolution, Pixel Size는 서로 다른 사양이다.

- Object Space의 $\mu\text{m}/\text{pixel}$ 은 FOV/Pixel Count 또는 Pixel Size/Magnification으로 계산한다.
- Optical Resolution과 Sampling을 함께 봐야 실제 검출 가능성을 판단할 수 있다.
- **step** 과 SDK Stride를 사용해야 Row Padding을 안전하게 처리한다.
- Saturation으로 손실된 정보는 후처리 Contrast 조정으로 복구되지 않는다.
- 검사 원본과 Display용 변환 Image를 분리한다.

2일차 권장 학습 순서

- ☐ 45분: 3.1~3.6 읽고 Bit Depth 표를 직접 다시 작성
- ☐ 45분: Resolution 5종을 본인 말로 비교
- ☐ 30분: 숫자 예제 1~4를 계산기 없이 다시 계산
- ☐ 60~90분: C++ 예제 실행 또는 코드 리뷰
- ☐ 60분: 실습 1의 Image Inspector 구현
- ☐ 30분: 면접 Q1~Q6의 30초 답변 녹음
- ☐ 20분: 틀린 내용과 질문을 문서 하단에 기록

학습 완료 체크

- ☐ 8/10/12/16-bit의 범위와 저장 차이를 설명할 수 있다.
- ☐ Camera/Image/Spatial/Optical Resolution과 Pixel Size를 구분한다.
- ☐ **2048×2048 8-bit Mono = 4 MiB** 를 계산할 수 있다.
- ☐ OpenCV의 **rows** , **cols** , **channels** , **depth** , **step** 을 설명할 수 있다.
- ☐ 12-bit 원본과 8-bit Display Image를 분리해야 하는 이유를 설명한다.
- ☐ Phase 1 미니 프로젝트를 설계하거나 구현했다.

다음 Chapter 예고

다음 Chapter에서는 **Camera Pixel Size = 3 μm** 의 정확한 의미를 배우고, $0.5 \times 1 \times 2 \times$ Magnification에서 Sensor Size, FOV, Object Space Pixel Resolution을 직접 계산한다.

Chapter 3. Camera Pixel Size와 실제 물체의 관계

Phase 2 · 3일차 · 예상 학습 시간: 10~14시간 · 난이도: 입문→중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- Camera Pixel Size(Pixel Pitch)의 정확한 의미
- $\text{Sensor Size} = \text{Pixel Count} \times \text{Pixel Size}$
- $\text{Object Space Pixel Size} = \text{Camera Pixel Size} \div \text{Magnification}$
- $\text{FOV} = \text{Sensor Size} \div \text{Magnification}$

- Object Space Pixel Size = FOV ÷ Image Pixel Count
- 0.5×, 1×, 2× 배율에서 FOV와 $\mu\text{m}/\text{pixel}$ 변화
- 명목 배율과 실제 배율, Calibration Scale의 차이
- Sampling과 실제 Defect 검출 능력의 차이

실무 목표

Camera Pixel Size = 3 μm 라는 사양을 보고 Sensor의 Physical Pixel Pitch임을 설명한다. Camera Resolution과 Lens Magnification을 이용해 Sensor Size, FOV, Object Space Pixel Resolution을 계산하고, 측정한 FOV로 결과를 교차 검증한다.

[!IMPORTANT]

“3 μm Camera이므로 물체의 3 μm 가 항상 Image 1 Pixel이다”는 틀린 문장이다.

3 μm 는 Sensor 위의 Pixel Pitch다. 물체 기준 $\mu\text{m}/\text{pixel}$ 은 Lens Magnification 또는 실제 FOV가 있어야 정해진다.

2. 선수 지식

- Chapter 2의 Camera Resolution, Image Resolution, Pixel Size 구분
- 1 mm = 1,000 μm
- 배율(Magnification)은 Image Space 크기 ÷ Object Space 크기
- Width와 Height 방향 계산은 각각 독립적임

2.1 이번 Chapter의 기호

기호	의미	단위
N_x, N_y	Sensor/Image Pixel Count	pixel
p	Camera Sensor Pixel Pitch	$\mu\text{m}/\text{pixel}$
S_x, S_y	Sensor Physical Size	μm 또는 mm
M	Optical Magnification	무차원, ×
FOV_x, FOV_y	Object Space에서 보이는 범위	μm 또는 mm
r_o	Object Space Pixel Size/Sampling	$\mu\text{m}/\text{pixel}$

이 Chapter는 정사각형 Pixel, 일정한 배율, 왜곡이 무시 가능한 이상적 모델부터 시작한다. 실제 장비에서는 Calibration으로 위치별 Scale과 왜곡을 확인한다.

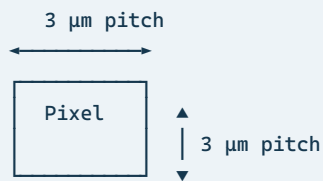
3. 핵심 개념

3.1 Camera Pixel Size = 3 μm 의 의미

Camera 사양의 Pixel Size = 3 $\mu\text{m} \times 3 \mu\text{m}$ 는 보통 Sensor 위에서 인접 Pixel 중심 사이 간격인 Pixel Pitch가 가로·세로 각각 약 3 μm 라는 뜻이다.

TEXT

Camera Sensor



이는 다음과 같은 뜻이 아니다.

- 물체의 3 μm Defect를 반드시 검출한다.
- Image의 1 Pixel이 항상 물체의 3 μm 다.
- Lens가 없어도 물체 크기를 계산할 수 있다.

Sensor Pixel의 실제 광감지 영역(Photosensitive Area)은 Pixel Pitch 전체와 다를 수 있다. Microlens, 회로, Fill Factor도 감도에 영향을 주지만 현재 단계에서는 Pixel Pitch를 Sensor Size 계산에 사용한다.

3.2 Sensor Size 계산

FORMULA · CALCULATION

Sensor Width = Horizontal Pixel Count $\times$ Pixel Size

Sensor Height = Vertical Pixel Count $\times$ Pixel Size

2048 $\times$ 2048, 3 μm Camera라면:

FORMULA · CALCULATION

Sensor Width = 2048 pixel $\times$ 3 $\mu\text{m}/\text{pixel}$
 = 6144 μm = 6.144 mm

Sensor Height = 6.144 mm

2048 $\times$ 2048 은 Image 배열의 Pixel 수이고 6.144 $\times$ 6.144 mm 는 Sensor의 물리적 크기다.

3.3 Optical Magnification

FORMULA · CALCULATION

Magnification M = Image Size on Sensor / Object Size

- $M = 1\times$: 물체의 1 mm가 Sensor 위 1 mm로 결상
- $M = 0.5\times$: 물체의 1 mm가 Sensor 위 0.5 mm로 결상
- $M = 2\times$: 물체의 1 mm가 Sensor 위 2 mm로 결상

배율이 커지면 같은 Sensor에 물체의 더 작은 영역만 보인다. FOV는 줄고 Object Space Sampling은 더 촘촘해진다.

3.4 Object Space Pixel Size

Sensor의 Pixel Pitch p 에 대응하는 물체 공간의 길이는:

FORMULA · CALCULATION

$$\text{Object Space Pixel Size } r_o = p / M$$

Pixel Size	Magnification	Object Space Pixel Size
3 μm	0.5 $\times$	6 $\mu\text{m/pixel}$
3 μm	1 $\times$	3 $\mu\text{m/pixel}$
3 μm	2 $\times$	1.5 $\mu\text{m/pixel}$

0.5 $\times$ 에서는 물체가 Sensor에 절반 크기로 맺히므로 Sensor 1 Pixel이 물체의 더 넓은 6 μm 구간을 담당한다. 2 $\times$ 에서는 물체가 두 배로 확대되어 1 Pixel이 물체의 1.5 μm 구간을 담당한다.

3.5 FOV 계산

FORMULA · CALCULATION

$$\text{FOV} = \text{Sensor Size} / \text{Magnification}$$

6.144 mm Sensor라면:

Magnification	Horizontal FOV	Vertical FOV
0.5 $\times$	12.288 mm	12.288 mm
1 $\times$	6.144 mm	6.144 mm
2 $\times$	3.072 mm	3.072 mm

배율이 2배가 되면 FOV와 $\mu\text{m/pixel}$ 은 절반이 된다. 반대로 배율이 절반이면 FOV와 $\mu\text{m/pixel}$ 은 2배가 된다.

3.6 FOV 방식으로 교차 검산

FORMULA · CALCULATION

$$\text{Object Space Pixel Size} = \text{FOV} / \text{Image Pixel Count}$$

0.5× 예제:

FORMULA · CALCULATION

$$\begin{aligned} &12.288 \text{ mm} / 2048 \text{ pixel} \\ &= 0.006 \text{ mm/pixel} \\ &= 6 \text{ } \mu\text{m/pixel} \end{aligned}$$

이는 $3 \text{ } \mu\text{m} / 0.5 = 6 \text{ } \mu\text{m/pixel}$ 과 같다. 두 방식의 결과가 다르면 다음을 확인한다.

- 사용한 Image가 Full Sensor인지 Camera ROI/Crop인지
- Lens의 명목 배율과 실제 배율이 같은지
- FOV 측정 기준이 Sensor 전체인지
- Pixel Size와 Pixel Count가 해당 Camera Mode의 값인지
- Lens Distortion/Perspective가 큰지

3.7 Sensor Pixel과 Image Pixel은 같은가?

Full Resolution, 1:1 Readout, Software Resize 없음이라면 Sensor Pixel 하나가 출력 Image Pixel 하나에 대응한다고 볼 수 있다. 하지만 다음 경우에는 단순 대응이 깨진다.

- Binning: 여러 Sensor Pixel을 출력 Pixel 하나로 결합
- Decimation/Skipping: 일부 Sensor Pixel만 읽음
- Camera ROI: Sensor 일부만 출력하지만 ROI 내부에서는 1:1일 수 있음
- Debayer: Color Filter Array의 Sensor Sample로 Color Pixel을 보간
- Software Resize: 출력 배열을 보간해 새 Pixel Grid 생성

따라서 SDK의 Pixel Format과 Camera Mode를 확인해야 한다.

3.8 명목 배율과 실제 배율

Lens에 0.5× 라고 적혀 있어도 실제 장착 조건, Working Distance, Focus, Tube Length 등에 따라 유효 배율이 달라질 수 있다. 실무에서는 알려진 길이의 Calibration Target을 촬영해 실제 Scale을 측정한다.

FORMULA · CALCULATION

Known Length = 10.000 mm

Measured Pixel Distance = 1660 pixel

Measured Scale = 10,000 μm / 1660 pixel
 $\approx 6.024 \mu\text{m}/\text{pixel}$

3 μm Pixel 기준 실제 배율은:

FORMULA · CALCULATION

$M_{\text{effective}} = 3 \mu\text{m}/\text{pixel} / 6.024 \mu\text{m}/\text{pixel}$
 $\approx 0.498\times$

명목 0.5 $\times$ 와 가깝지만 측정/Calibration 값이 실제 치수 계산의 기준이 된다.

3.9 Sampling과 실제 Resolution

1.5 $\mu\text{m}/\text{pixel}$ 이라고 해서 1.5 μm Defect를 안정적으로 검출한다는 뜻은 아니다.

FORMULA · CALCULATION

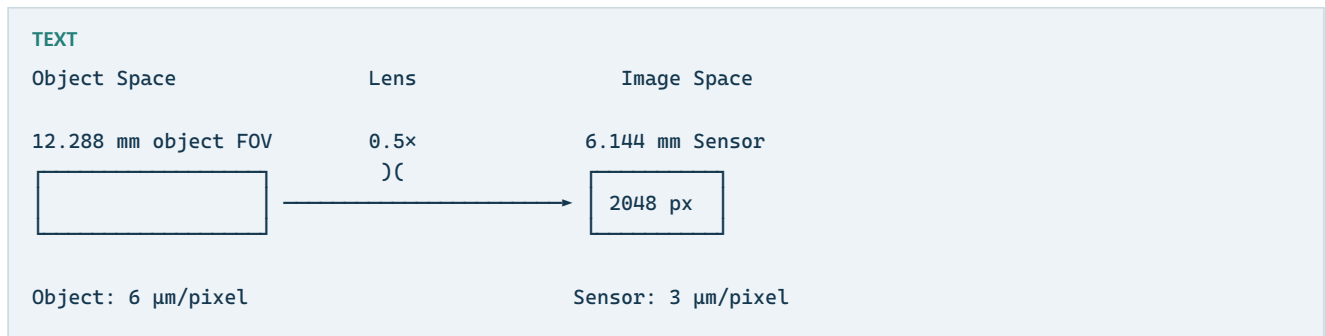
Object Space Sampling

- + Lens MTF / Focus
- + Illumination Contrast
- + Sensor Noise / Exposure
- + Motion Blur
- + Algorithm의 최소 필요 Pixel 수
- = 실제 검사 성능

Defect가 1 Pixel보다 작아도 Pixel 평균값에 영향을 주어 보일 수 있지만 위치·Contrast에 민감하다. 안정적인 검출이나 치수 측정에는 일반적으로 여러 Pixel에 걸친 표현이 필요하며, 필요한 수는 결함과 알고리즘별 실험으로 정해야 한다.

4. 그림으로 이해하기

4.1 Camera-Lens-Object 관계



4.2 배율 변화

FORMULA · CALCULATION		
M = 0.5×	wide FOV, coarse sampling	12.288 mm, 6 μm/px
M = 1.0×	middle	6.144 mm, 3 μm/px
M = 2.0×	narrow FOV, fine sampling	3.072 mm, 1.5 μm/px
Magnification ↑ → FOV ↓ → Object μm/pixel ↓ → 더 촘촘한 Sampling		
Magnification ↓ → FOV ↑ → Object μm/pixel ↑ → 더 넓은 영역		

4.3 한 Pixel까지의 흐름

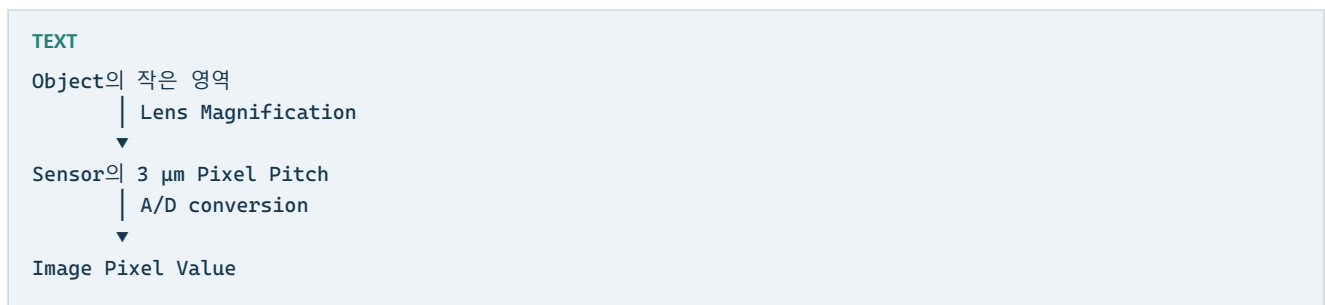


Image Pixel 값은 해당 Object 영역의 “크기 값”이 아니라 그 영역에서 광학계가 전달한 빛을 Sensor가 Sampling한 밝기 값이다.

5. 실제 검사 장비에서 어디에 사용하는가?

Lens 및 Camera 선정

검사 대상 폭이 10 mm이고 양쪽 Margin을 포함해 12 mm FOV가 필요하다면 6.144 mm Sensor에서 약 **0.512×** 이하 배율이 필요하다. 원하는 μm/pixel과 FOV를 동시에 만족하는지 확인해야 한다.

치수 측정

Edge 간 거리가 500 pixel이고 Calibration Scale이 $6\ \mu\text{m}/\text{pixel}$ 이면 이상적인 길이는 3 mm다. 하지만 위치별 Scale과 Sub-pixel Edge 알고리즘의 Bias를 Gauge로 검증해야 한다.

Defect 사양 협의

고객이 $2\ \mu\text{m}$ Defect 검출을 요구할 때 Camera Pixel Size만 보고 가능하다고 답하지 않는다. FOV와 Sampling, Lens MTF, 조명 Contrast, Exposure/Noise, Defect 형상과 검출률 기준을 함께 확인한다.

Recipe와 Calibration 관리

Camera ROI, Binning, Lens, Working Distance 또는 Focus를 바꾸면 Scale이 변할 수 있다. Recipe에 Camera Mode, Optical Configuration ID, Calibration Version을 함께 저장해야 한다.

Stage 이동량 계산

Align 결과가 **+20 pixel** 이고 Calibration Scale이 **$5\ \mu\text{m}/\text{pixel}$** 이라면 크기는 $100\ \mu\text{m}$ 다. 실제 Stage 명령에는 축 방향, Rotation, Offset 및 Camera와 Stage의 관계를 추가 적용한다.

6. 숫자로 이해하기

예제 1: $3\ \mu\text{m}$, 2048×2048 , $1 \times$

FORMULA · CALCULATION

Sensor Size

$$= 2048\ \text{pixel} \times 3\ \mu\text{m}/\text{pixel}$$

$$= 6144\ \mu\text{m} = 6.144\ \text{mm}$$

Object Space Pixel Size

$$= 3\ \mu\text{m}/\text{pixel} \div 1$$

$$= 3\ \mu\text{m}/\text{pixel}$$

FOV

$$= 6.144\ \text{mm} \div 1$$

$$= 6.144\ \text{mm} \times 6.144\ \text{mm}$$

예제 2: 같은 Camera, 0.5×

FORMULA · CALCULATION

Object Space Pixel Size
 $= 3 \mu\text{m}/\text{pixel} \div 0.5$
 $= 6 \mu\text{m}/\text{pixel}$

FOV
 $= 6.144 \text{ mm} \div 0.5$
 $= 12.288 \text{ mm} \times 12.288 \text{ mm}$

물체 6 μm 가 이상적으로 Image 1 Pixel 간격에 대응한다. 이는 6 μm 특징을 안정적으로 검출한다는 보장은 아니다.

예제 3: 같은 Camera, 2×

FORMULA · CALCULATION

Object Space Pixel Size
 $= 3 \mu\text{m}/\text{pixel} \div 2$
 $= 1.5 \mu\text{m}/\text{pixel}$

FOV
 $= 6.144 \text{ mm} \div 2$
 $= 3.072 \text{ mm} \times 3.072 \text{ mm}$

예제 4: 실제 FOV로 Scale 계산

Full Width 2048 pixel에서 실제 Target으로 측정한 FOV가 12.20 mm라면:

FORMULA · CALCULATION

$12.20 \text{ mm} = 12,200 \mu\text{m}$

Measured Scale
 $= 12,200 \mu\text{m} / 2048 \text{ pixel}$
 $\approx 5.95703125 \mu\text{m}/\text{pixel}$

3 μm Camera의 유효 배율:

FORMULA · CALCULATION

$$M_{\text{effective}} = 3 / 5.95703125 \approx 0.5036\times$$

예제 5: 요구사항 역산

조건:

- 검사 FOV Width ≥ 10 mm
- Image Width = 2048 pixel
- 최소 결함이 안정적으로 4 Pixel에 걸쳐야 함

FOV 10 mm일 때 Sampling:

FORMULA · CALCULATION

$$10,000 \mu\text{m} / 2048 \text{ pixel} \approx 4.8828 \mu\text{m/pixel}$$

4 Pixel에 대응하는 물체 크기:

FORMULA · CALCULATION

$$4 \times 4.8828 \mu\text{m} \approx 19.53 \mu\text{m}$$

따라서 이 단순 Sampling 기준에서는 약 20 μm 특징이 4 Pixel을 차지한다. 10 μm Defect를 4 Pixel로 표현하려면 **2.5 $\mu\text{m/pixel}$** 이하가 필요하고, 10 mm FOV에서는 가로 4000 Pixel 이상이 필요하다. 광학 조건은 별도로 만족해야 한다.

7. C++ 구현

Camera-Lens 계산기

CPP · 1/2

```
#include <cmath>
#include <cstdint>
#include <initializer_list>
#include <iostream>
#include <stdexcept>

struct CameraSpec final {
    std::size_t widthPixels{};
    std::size_t heightPixels{};
    double pixelPitchUm{};
};

struct OpticalCalculation final {
    double sensorWidthMm{};
    double sensorHeightMm{};
    double fovWidthMm{};
    double fovHeightMm{};
    double objectUmPerPixel{};
};

[[nodiscard]] OpticalCalculation CalculateOptics(
    const CameraSpec& camera,
    const double magnification)
{
    if (camera.widthPixels == 0 || camera.heightPixels == 0) {
        throw std::invalid_argument{"Pixel count must be positive"};
    }
    if (!std::isfinite(camera.pixelPitchUm) || camera.pixelPitchUm <= 0.0) {
        throw std::invalid_argument{"Pixel pitch must be finite and positive"};
    }
    if (!std::isfinite(magnification) || magnification <= 0.0) {
        throw std::invalid_argument{"Magnification must be finite and positive"};
    }

    constexpr double umPerMm = 1000.0;
    const double sensorWidthMm =
        static_cast<double>(camera.widthPixels) * camera.pixelPitchUm / umPerMm;
    const double sensorHeightMm =
        static_cast<double>(camera.heightPixels) * camera.pixelPitchUm / umPerMm;

    return {
        sensorWidthMm,
        sensorHeightMm,
        sensorWidthMm / magnification,
        sensorHeightMm / magnification,
        camera.pixelPitchUm / magnification
    };
};
```

CPP · 2/2

```

}

[[nodiscard]] double CalculateMeasuredUmPerPixel(
    const double measuredFovMm,
    const std::size_t imagePixels)
{
    if (!std::isfinite(measuredFovMm) || measuredFovMm <= 0.0 ||
        imagePixels == 0) {
        throw std::invalid_argument{"FOV and pixel count must be positive"};
    }
    return measuredFovMm * 1000.0 / static_cast<double>(imagePixels);
}

[[nodiscard]] double CalculateEffectiveMagnification(
    const double sensorPixelPitchUm,
    const double measuredObjectUmPerPixel)
{
    if (sensorPixelPitchUm <= 0.0 || measuredObjectUmPerPixel <= 0.0) {
        throw std::invalid_argument{"Scale values must be positive"};
    }
    return sensorPixelPitchUm / measuredObjectUmPerPixel;
}

int main()
{
    const CameraSpec camera{2048, 2048, 3.0};

    for (const double magnification : {0.5, 1.0, 2.0}) {
        const auto result = CalculateOptics(camera, magnification);
        std::cout << magnification << "x: FOV="
                    << result.fovWidthMm << " x " << result.fovHeightMm
                    << " mm, sampling=" << result.objectUmPerPixel
                    << " um/pixel\n";
    }

    const double measuredScale = CalculateMeasuredUmPerPixel(12.20, 2048);
    const double effectiveMagnification =
        CalculateEffectiveMagnification(camera.pixelPitchUm, measuredScale);
    std::cout << "Measured scale=" << measuredScale
                << " um/pixel, effective M=" << effectiveMagnification << "x\n";
}

```

예상 핵심 출력:

FORMULA · CALCULATION

0.5x: FOV=12.288 x 12.288 mm, sampling=6 um/pixel

1x: FOV=6.144 x 6.144 mm, sampling=3 um/pixel

2x: FOV=3.072 x 3.072 mm, sampling=1.5 um/pixel

간단한 Unit Test

C++

```
#include <cassert>
#include <cmath>

[[nodiscard]] bool NearlyEqual(
    const double lhs,
    const double rhs,
    const double tolerance = 1e-9)
{
    return std::abs(lhs - rhs) <= tolerance;
}

void TestHalfMagnification()
{
    const auto result = CalculateOptics({2048, 2048, 3.0}, 0.5);
    assert(NearlyEqual(result.sensorWidthMm, 6.144));
    assert(NearlyEqual(result.fovWidthMm, 12.288));
    assert(NearlyEqual(result.objectUmPerPixel, 6.0));
}

void TestFovCrossCheck()
{
    const double scale = CalculateMeasuredUmPerPixel(12.288, 2048);
    assert(NearlyEqual(scale, 6.0));
    assert(NearlyEqual(CalculateEffectiveMagnification(3.0, scale), 0.5));
}

void TestInvalidMagnification()
{
    bool thrown = false;
    try {
        static_cast<void>(CalculateOptics({2048, 2048, 3.0}, 0.0));
    } catch (const std::invalid_argument&) {
        thrown = true;
    }
    assert(thrown);
}
```

코드에서 봐야 할 점

1. Pixel Count, Pixel Pitch, Magnification에 단위와 의미가 드러나는 이름을 사용했다.
2. μm 와 mm 변환을 한 지점에서 명시했다.
3. `0x`, 음수, NaN/Infinity를 거부한다.
4. Lens 사양 계산값과 실제 FOV 측정값을 별도 함수로 구분했다.
5. 실제 제품에서는 X/Y Pixel Pitch가 다르거나 Calibration이 위치별로 다를 가능성을 모델에 반영해야 한다.

8. 실무에서 발생하는 문제

문제 1: 명목 배율만 사용해 치수 오차 발생

- 원인: Working Distance, Focus, 장착 공차로 실제 배율 변화
- 대응: Calibration Target으로 실제 $\mu\text{m}/\text{pixel}$ 측정, Calibration Version 관리

문제 2: Camera ROI 이후에도 Full Width로 나눔

- 원인: 1024 Pixel ROI Image에 2048 Pixel을 사용해 Scale 계산
- 대응: FOV와 Pixel Count가 동일한 출력 영역을 기준으로 계산

문제 3: Resize된 Review Image에서 좌표 측정

- 원인: 원본 2048 Pixel을 UI 1024 Pixel로 줄였는데 Display 좌표를 원본 좌표로 해석
- 대응: Display↔Original Transform을 명시하고 검사는 원본 좌표 사용

문제 4: Lens Distortion으로 위치별 Scale이 다름

- 원인: 중심과 가장자리의 배율 차이를 단일 $\mu\text{m}/\text{pixel}$ 로 처리
- 대응: 왜곡 보정, Grid Calibration, 위치별 Residual 확인

문제 5: Pixel Sampling을 실제 검출 Resolution으로 오해

- 원인: Lens MTF, Focus, 조명, Noise, Motion Blur 무시
- 대응: Golden Sample/Defect Sample로 Detection Rate와 False Positive 검증

문제 6: Binning Mode 변경 후 Calibration 재사용

- 원인: 출력 Pixel 하나가 결합하는 Sensor 영역이 변했는데 기존 Scale 사용
- 대응: Camera Mode를 Calibration Key에 포함하고 변경 시 무효화

9. 흔한 오해

1×일 때의 이상적 Sampling과 우연히 같을 뿐이며 배율이 바뀌면 달라진다.

1. **“3 μm Pixel Camera면 물체 3 μm 가 1 Pixel이다.”**

Sampling은 촘촘해지지만 FOV와 DOF가 줄고 정렬·초점·진동 민감도가 커질 수 있다.

2. **“배율이 클수록 언제나 좋다.”**

Sampling 간격일 뿐 실제 검출에는 광학과 Contrast, Noise, 필요한 Pixel 수가 중요하다.

3. **“ $\mu\text{m}/\text{pixel}$ 이 2 μm 면 2 μm Defect를 보장한다.”**

Binning, Decimation, Debayer, Resize가 있으면 달라질 수 있다.

4. **“Sensor Pixel과 Image Pixel은 언제나 1:1이다.”**

Sensor Size와 Lens Magnification의 조합으로 정해진다.

5. **“FOV는 Camera 사양만으로 정해진다.”**

장착 조건에 따라 달라질 수 있으므로 실제 Calibration으로 확인한다.

6. **“Lens에 적힌 0.5×가 정확한 실제 배율이다.”**

같은 Sensor 기술과 조건에서도 감도, Full Well Capacity, Noise, Lens Resolution 요구와 Trade-off가 있다.

7. “Pixel Size가 작으면 무조건 영상이 좋다.”

10. 면접에서 나올 수 있는 질문

Q1. Camera Pixel Size가 3 μm 라는 것은 무슨 의미인가요?

초보자가 이해할 수 있는 답변 Camera Sensor에서 Pixel 하나의 물리적 간격이 약 3 μm 라는 뜻입니다. 물체의 3 μm 가 항상 Image 1 Pixel이라는 뜻은 아닙니다.

실무자 답변 Sensor의 Pixel Pitch가 3 μm 라는 뜻이다. Full Resolution 1:1 Readout에서는 출력 Pixel과 Sensor Pixel이 대응하지만, Object Space Sampling은 3 $\mu\text{m}/\text{M}$ 으로 Lens의 유효 배율에 따라 달라진다. 실제 치수는 FOV 또는 Calibration Target으로 Scale을 검증한다.

면접용 30초 답변 “3 μm 는 물체가 아니라 Sensor의 Physical Pixel Pitch입니다. 실제 물체에서 1 Pixel이 담당하는 길이는 $\text{Pixel Pitch} \div \text{Lens Magnification}$ 입니다. 따라서 0.5×면 6 $\mu\text{m}/\text{pixel}$, 1×면 3 $\mu\text{m}/\text{pixel}$, 2×면 1.5 $\mu\text{m}/\text{pixel}$ 이며 실제 측정은 Calibration으로 확인합니다.”

Q2. 2048 Pixel, 3 μm Camera의 Sensor Width는 얼마인가요?

초보자가 이해할 수 있는 답변 2048에 3 μm 를 곱하면 6144 μm , 즉 6.144 mm입니다.

실무자 답변 정사각 3 μm Pitch와 2048 유효 Pixel이라는 전제에서 $2048 \times 3 \mu\text{m} = 6.144 \text{ mm}$ 다. Data Sheet의 Active Area, Optical Format, 유효 Pixel과 전체 Pixel의 차이를 확인하고 Lens Image Circle이 Sensor를 커버해야 한다.

면접용 30초 답변 “Sensor Width는 Pixel Count×Pixel Pitch이므로 $2048 \times 3 \mu\text{m} = 6144 \mu\text{m} = 6.144 \text{ mm}$ 입니다. 이 물리적 Sensor Width를 배율로 나누면 Object FOV를 구할 수 있습니다.”

Q3. Magnification이 증가하면 FOV와 $\mu\text{m}/\text{pixel}$ 은 어떻게 되나요?

초보자가 이해할 수 있는 답변 배율이 커지면 확대되어 더 좁은 영역을 보고, 물체 기준 한 Pixel의 크기는 작아집니다.

실무자 답변 고정 Sensor에서 $\text{FOV} = S/M$, $\text{Object } \mu\text{m}/\text{pixel} = p/M$ 이므로 둘 다 배율에 반비례한다. 다만 Sampling 개선이 Optical Resolution 개선을 자동 보장하지 않고, DOF와 조립 공차, Focus 민감도도 함께 평가해야 한다.

면접용 30초 답변 “고정 Camera에서는 배율이 증가할수록 FOV와 Object Space $\mu\text{m}/\text{pixel}$ 이 배율에 반비례해 감소합니다. 예를 들어 1×에서 3 $\mu\text{m}/\text{pixel}$ 이면 2×에서 1.5 $\mu\text{m}/\text{pixel}$ 이지만 FOV는 절반이 됩니다.”

Q4. Object Space Pixel Size를 두 가지 방식으로 계산해 보세요.

초보자가 이해할 수 있는 답변 Camera Pixel Size를 배율로 나누거나, 실제 FOV를 Image Pixel 수로 나눕니다.

실무자 답변 이상적 설계값은 p/M , 실제 측정 기반 값은 FOV/N 이다. 둘의 차이는 명목/유효 배율, Camera ROI, Distortion, FOV 측정 오차를 찾는 단서다. 치수 검사는 Calibration Target 기반 값을 우선한다.

면접용 30초 답변 “설계 단계에서는 $\text{Sensor Pixel Pitch} \div \text{Magnification}$, 장비에서는 $\text{측정 FOV} \div \text{Image Pixel Count}$ 로 구합니다. 두 값을 교차 검산하고 차이가 있으면 실제 배율, ROI Mode, 왜곡과 Calibration 조건을 확인합니다.”

Q5. 3 μm Pixel Camera로 2 μm Defect를 검출할 수 있나요?

초보자가 이해할 수 있는 답변 Pixel Size만으로는 알 수 없습니다. Lens 배율, 선명도, 조명 Contrast와 Noise를 함께 봐야 합니다.

실무자 답변 먼저 Object Space Sampling을 계산하고 Defect가 몇 Pixel에 걸리는지 본다. 이후 Lens MTF/Focus, 조명에 따른 Defect Contrast, Sensor SNR, Motion Blur, Sub-pixel 위치에 따른 응답, 목표 Detection/False Positive Rate를 Sample로 검증해야 한다.

면접용 30초 답변 "Pixel Size 하나로는 판단할 수 없습니다. 배율을 적용한 Object $\mu\text{m}/\text{pixel}$, Lens MTF와 Focus, 조명 Contrast, Noise 및 알고리즘의 최소 Sampling 요구를 확인해야 합니다. 최종 판단은 실제 Defect Sample의 검출률과 오검률로 검증합니다."

Q6. 명목 배율 대신 Calibration이 필요한 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 실제로 Lens를 장착하면 거리와 초점 때문에 사양서 배율과 조금 달라질 수 있기 때문입니다.

실무자 답변 Working Distance, Focus, 장착 공차, Sensor/Lens 자세, Distortion으로 실제 Scale과 좌표 방향이 달라진다. 알려진 Target으로 Scale, Rotation, Offset 및 필요 시 위치별 왜곡을 추정하고 Residual을 확인해야 측정 추적성을 확보할 수 있다.

면접용 30초 답변 "Lens의 명목 배율은 설계값이고 실제 장착에서는 WD, Focus, 공차와 왜곡 때문에 Scale이 달라집니다. Calibration Target으로 실제 $\mu\text{m}/\text{pixel}$ 과 좌표 변환을 구하고 Version을 Recipe/Result에 남겨야 치수와 위치 결과를 재현할 수 있습니다."

11. 실습 문제

실습 1: 배율별 계산표 만들기

Camera가 4096×3000 , Pixel Size가 3.45 μm 일 때 다음 배율을 계산한다.

- 0.25×
- 0.5×
- 1×
- 2×

각각 Sensor Width/Height, FOV Width/Height, Object $\mu\text{m}/\text{pixel}$ 을 손 계산하고 C++ 결과와 비교한다.

실습 2: 요구사항에서 Camera Pixel Count 역산

조건:

- FOV Width = 20 mm
- 최소 Defect Width = 25 μm
- Defect가 최소 5 Pixel에 걸쳐야 한다는 내부 기준

필요한 최대 $\mu\text{m}/\text{pixel}$ 과 최소 Horizontal Pixel Count를 계산한다. 이후 Lens MTF와 조명을 확인하지 않으면 안 되는 이유도 작성한다.

실습 3: 실제 Scale 측정

Grid 또는 자를 촬영한 Image를 사용한다.

1. 멀리 떨어진 두 기준점의 실제 거리를 기록한다.
2. 두 점 사이 Pixel 거리를 측정한다.
3. $\mu\text{m}/\text{pixel}$ 과 유효 배율을 계산한다.
4. Image 중앙과 가장자리에서 반복한다.
5. 차이를 왜곡 또는 측정 오차 관점에서 분석한다.

실습 4: Camera Mode 변경 영향 분석

Full Image 2048×2048에서 다음 Mode로 바뀔 때 Pixel Count, FOV, $\mu\text{m}/\text{output-pixel}$ 및 대역폭이 어떻게 달라지는지 설명한다.

- 중앙 Camera ROI 1024×1024, 1:1 Readout
- 2×2 Binning으로 1024×1024 출력
- Software Resize로 1024×1024 저장

세 Mode가 같은 **1024×1024 Image**를 만들더라도 의미가 다른 이유를 적는다.

Phase 2 미니 프로젝트: Camera-Lens Calculator

C++17로 다음 기능을 구현한다.

TEXT

Camera Spec 입력

└ Width/Height Pixel
└ Pixel Pitch μm

Lens Magnification 입력

↓

Sensor Size / FOV / $\mu\text{m-per-pixel}$ 계산

↓

Measured FOV 또는 Calibration Distance 입력

↓

Measured Scale / Effective Magnification 계산

↓

설계값과 측정값 오차율 출력

필수 검증

- 0/음수/NaN 입력 거부
- $\mu\text{m}/\text{mm}$ 단위 변환 Unit Test
- 0.5×/1×/2× Golden Value Test
- **p/M** 과 **FOV/N** 교차 계산
- 설계값 대비 측정값 오차가 기준을 넘으면 Warning

12. Chapter 핵심 요약

- Camera Pixel Size는 Sensor의 Physical Pixel Pitch다.
- $\text{Sensor Size} = \text{Pixel Count} \times \text{Pixel Pitch}$ 다.
- $\text{Object } \mu\text{m/pixel} = \text{Pixel Pitch} \div \text{Magnification}$ 이다.
- $\text{FOV} = \text{Sensor Size} \div \text{Magnification}$ 이다.
- 실제 FOV가 있으면 $\text{FOV} \div \text{Image Pixel Count}$ 로 Scale을 교차 계산한다.
- 3 μm Camera에서 $0.5\times/1\times/2\times$ 는 각각 6/3/1.5 $\mu\text{m/pixel}$ 이다.
- 배율이 증가하면 FOV와 $\mu\text{m/pixel}$ 은 감소한다.
- $\mu\text{m/pixel}$ 은 Sampling이며 실제 검출 가능성을 단독으로 보장하지 않는다.
- Binning, ROI, Resize가 Sensor Pixel과 Image Pixel 관계를 바꿀 수 있다.
- 실제 치수와 좌표에는 명목 배율보다 검증된 Calibration을 사용한다.

3일차 권장 학습 순서

- ☐ 40분: Pixel Pitch와 Object Space Pixel Size 차이를 그림으로 설명
- ☐ 50분: $0.5\times/1\times/2\times$ 계산을 손으로 반복
- ☐ 30분: p/M 과 FOV/N 방식으로 교차 계산
- ☐ 60~90분: C++ Camera-Lens Calculator 구현
- ☐ 60분: 요구사항 역산 실습 1~2
- ☐ 30분: 면접 Q1~Q6을 30초 답변으로 연습
- ☐ 20분: 실제 업무 Camera/Lens 사양에서 모르는 항목 목록 작성

학습 완료 체크

- ☐ 3 μm Camera 의 뜻을 오해 없이 설명할 수 있다.
- ☐ Sensor Size, FOV, Object $\mu\text{m/pixel}$ 을 단위와 함께 계산한다.
- ☐ 배율 변화의 장점과 FOV Trade-off를 설명한다.
- ☐ Camera ROI/Binning/Resize의 차이를 설명한다.
- ☐ Sampling과 실제 검출 Resolution의 차이를 설명한다.
- ☐ Calculator 또는 계산표를 완성했다.

다음 Chapter 예고

다음 Chapter에서는 Camera와 Lens를 하나의 Optical System으로 보고 Focal Length, Magnification, Working Distance, FOV, Object/Image Space, Depth of Field, Distortion의 관계를 학습한다.

Chapter 4. Camera + Lens + Optical System

Phase 3 · 4일차 · 예상 학습 시간: 12~16시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- Camera, Sensor, Lens, Object, Image의 관계
- Focal Length(초점거리)와 Magnification(배율)의 차이
- Working Distance(WD), Object Distance, Image Distance 구분
- Sensor Size, Magnification, FOV 관계
- Object Space와 Image Space
- Focus, Depth of Field(DOF), Defocus
- Distortion과 치수·좌표 오차
- F-number, Aperture, Diffraction, Exposure의 Trade-off
- 일반 Lens와 Telecentric Lens의 차이
- Lens Resolution/MTF와 Sensor Sampling의 조합

실무 목표

검사 요구사항을 FOV → 필요한 $\mu\text{m}/\text{pixel}$ → Sensor/Lens 조합 → WD/DOF → 조명/Exposure → Calibration 순서로 검토한다. “같은 Camera인데 Lens를 바꾸면 왜 검사 결과가 달라지는가?”를 FOV, Sampling, Contrast, Blur, Distortion 관점에서 설명한다.

2. 선수 지식

- Chapter 3의 다음 관계식

FORMULA · CALCULATION

Sensor Size = Pixel Count × Pixel Pitch

FOV = Sensor Size ÷ Magnification

Object Space Pixel Size = Pixel Pitch ÷ Magnification

- 단위: $1 \text{ mm} = 1,000 \mu\text{m}$
- 비율과 역수
- Lens 사양의 명목값과 장착 후 Calibration 값은 다를 수 있음

2.1 기호

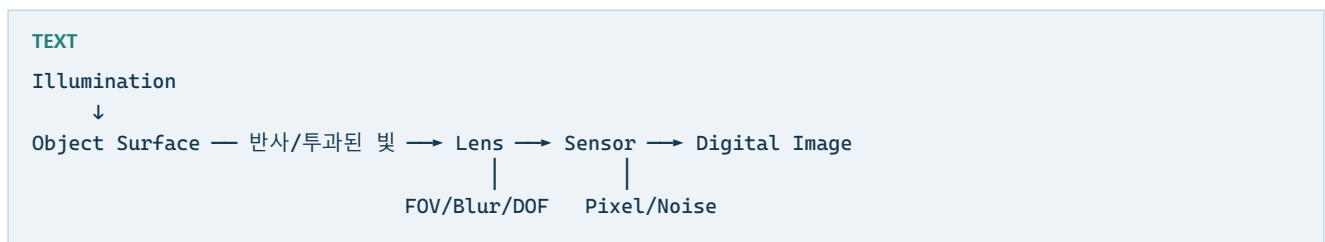
기호	의미	단위
f	Focal Length	mm
u	Object Distance(주평면 기준 이상 모델)	mm
v	Image Distance(주평면 기준 이상 모델)	mm
M	Magnification의 절댓값 v/u	x
N	F-number	무차원
c	허용 Circle of Confusion	mm

[!WARNING]

실제 제품의 **Working Distance**는 보통 Lens의 기계적 전면 또는 제조사가 정한 기준면부터 Object까지의 거리다. Thin Lens 식의 u 는 광학 주평면(Principal Plane) 기준이므로 두 값을 같은 것으로 취급하면 안 된다.

3. 핵심 개념

3.1 Camera만으로 Image가 만들어지지 않는다



Camera Sensor는 도달한 빛을 Sampling한다. Lens는 Object의 어느 범위를 Sensor에 어떤 배율과 선명도로 맺을지 결정한다. 조명은 검사 특징이 Lens에 들어오는 Contrast를 만든다.

같은 Camera라도 Lens가 달라지면 다음이 바뀔 수 있다.

- FOV와 Object Space $\mu\text{m}/\text{pixel}$
- Working Distance와 장비 배치
- Optical Resolution과 Contrast
- DOF와 Focus 민감도
- Distortion과 위치·치수 오차
- Sensor 전체를 덮는 Image Circle
- 필요한 조명 배치와 Exposure

3.2 Object Space와 Image Space

- **Object Space**: 실제 검사 대상이 존재하는 공간. mm, μm 등 실제 단위를 사용한다.
- **Image Space**: Lens가 Sensor 위에 형성한 Optical Image 공간. Sensor mm 또는 Pixel 좌표로 표현한다.

TEXT

Object 10 mm — 0.5× Lens → Sensor Image 5 mm

Object 5 mm — 1.0× Lens → Sensor Image 5 mm

Object 2.5 mm — 2.0× Lens → Sensor Image 5 mm

Sensor의 3 μm Pixel은 Image Space의 Sampling Pitch다. Object Space에서는 배율을 적용해 각각 6, 3, 1.5 μm/pixel이 된다.

3.3 Focal Length와 Magnification

Focal Length(초점거리)는 Lens의 광학적 특성으로, 무한대에 가까운 평행광을 맺는 기준 거리와 관련된다.

Magnification(배율)은 특정 Object 거리와 Image 거리에서 Object 크기가 Sensor 위에 맺힌 크기의 비율이다.

FORMULA · CALCULATION

Magnification = Image Size / Object Size

따라서 **50 mm Lens = 0.5× Lens** 가 아니다. 같은 Focal Length도 Working Distance와 Extension/Focus 조건에 따라 배율이 달라진다. Fixed Magnification Telecentric Lens처럼 특정 WD에서 배율을 사양으로 제공하는 제품도 있다.

이상적인 Thin Lens 모델

FORMULA · CALCULATION

$$1/f = 1/u + 1/v$$

$$M = |v/u|$$

배율 **M** 이 주어지면 이상 모델에서:

FORMULA · CALCULATION

$$u = f \times (1 + 1/M)$$

$$v = f \times (1 + M)$$

이 식은 개념과 1차 추정에 유용하지만 실제 Compound Lens의 주평면 위치, Internal Focusing, Mount Flange Distance, 제조사 WD 정의를 반영하지 않는다. Lens 선정은 제조사 Datasheet와 Lens Calculator를 우선한다.

3.4 Working Distance

Working Distance는 Lens와 Object 사이에 실제로 확보되는 작업 거리다. 다음을 함께 결정한다.

- Stage, Robot, Nozzle, Cover와의 기계 간섭
- Ring/Coaxial/Dome/Back Light를 설치할 공간
- 진동과 높이 편차에 대한 민감도
- 안전 및 유지보수 접근성
- 원하는 FOV와 배율을 만들 수 있는 Lens 조합

WD를 임의로 바꾸고 Focus만 다시 맞추면 배율과 FOV가 달라질 수 있으므로 Calibration을 재검토해야 한다.

3.5 FOV와 Sensor Coverage

고정 배율 Lens의 1차 계산:

FORMULA · CALCULATION

Horizontal FOV = Sensor Width / M
Vertical FOV = Sensor Height / M

Lens의 **Image Circle**이 Sensor 대각선을 충분히 덮어야 한다. 작으면 모서리가 어두워지는 Vignetting 또는 심한 화질 저하가 생길 수 있다.

FORMULA · CALCULATION

Sensor Diagonal = $\sqrt{\text{Width}^2 + \text{Height}^2}$

Sensor Format의 "1/2 inch" 같은 관용 표기는 실제 Sensor 대각선과 같지 않을 수 있으므로 mm 단위 Active Area를 확인한다.

3.6 Aperture와 F-number

FORMULA · CALCULATION

F-number N = Focal Length / Entrance Pupil Diameter

- 작은 F-number(예: F/2.8): 더 많은 빛, 짧은 Exposure 가능, DOF가 얇을 수 있음
- 큰 F-number(예: F/8): 빛 감소, DOF 증가 경향, 너무 조이면 Diffraction 증가

조리개를 조이면 무조건 선명해지는 것이 아니다. 수차가 줄다가 Diffraction이 증가하는 최적 구간이 있으며, Sensor Pixel Pitch와 요구 MTF에서 실제 Test해야 한다.

3.7 Focus와 Depth of Field

Focus는 관심 Plane의 Detail이 Sensor에 가장 선명하게 맺히는 상태다. **Depth of Field(DOF)**는 허용 가능한 선명도로 보이는 Object 방향 깊이 범위다.

DOF는 일반적으로 다음 경향을 가진다.

- F-number 증가 → DOF 증가 경향
- Magnification 증가 → DOF 급격히 감소
- 허용 Blur 기준이 느슨함 → DOF 증가
- Object 높이 편차가 큼 → 더 큰 DOF 필요

Macro 영역에서 사용하는 대칭 근사식 중 하나:

FORMULA · CALCULATION

$$\text{Total DOF} \approx 2 \times N \times c \times (1 + M) / M^2$$

이는 근사적 비교용이다. **c** 선택, Pupil Magnification, 비대칭 전후 DOF, Diffraction과 Lens 설계를 생략하므로 최종 사양은 제조사 DOF 자료와 실제 Target으로 검증한다.

3.8 Distortion

Distortion은 Object의 위치가 이상적인 선형 투영 위치와 다르게 맷히는 기하 오차다.

- **Barrel Distortion:** 직선이 바깥쪽으로 볼록하게 휘어 보임
- **Pincushion Distortion:** 직선이 안쪽으로 당겨진 것처럼 보임
- **Mustache/Wave Distortion:** 위치에 따라 복합 형태

Distortion은 Blur와 다르다. Edge가 선명해도 위치가 틀릴 수 있다. 치수·위치 검사에서는 Lens 사양의 Distortion 값뿐 아니라 Working Distance, Focus, 실제 FOV에서 Grid Calibration Residual을 확인한다.

3.9 Perspective Error와 Telecentric Lens

일반 Lens에서는 Object가 Lens에 가까워지면 더 크게 보인다. 높이 편차가 있는 부품 치수 검사에서 배율 오차가 발생한다.

Object-side Telecentric Lens는 일정 범위 내에서 Object 거리 변화에 따른 배율 변화를 줄이고 Perspective를 억제한다.

장점:

- 높이 변화에 따른 치수 변화 감소
- 원통/단차 부품의 가장자리 관찰 안정성
- 낮은 Distortion 제품을 통한 정밀 측정

Trade-off:

- 크고 비싸며 무거울 수 있음
- 제한된 FOV, WD, Telecentric Range
- 큰 Sensor/FOV에서 Lens 직경 증가
- Telecentric Lens도 Calibration과 Focus 검증이 필요

3.10 Optical Resolution과 MTF

Lens Resolution은 단순히 “몇 μm 까지 보인다” 한 숫자로 끝나지 않는다. **MTF(Modulation Transfer Function)**는 공간 주파수별 Contrast 전달 능력을 나타낸다.

TEXT

Object Detail

- └ Lens가 Contrast를 얼마나 전달하는가? → MTF
- └ Sensor가 얼마나 촘촘히 Sampling하는가? → $\mu\text{m}/\text{pixel}$
 - └ Noise/조명/알고리즘이 구분 가능한가? → 실제 검사 성능

높은 해상도 Camera를 저해상도 Lens에 연결하면 Pixel 수만 늘고 실제 Detail은 Blur될 수 있다. 반대로 좋은 Lens라도 Object Sampling이 너무 거칠면 Alias가 발생한다.

4. 그림으로 이해하기

4.1 Optical System

TEXT

Object Space

Image Space

Object

Lens

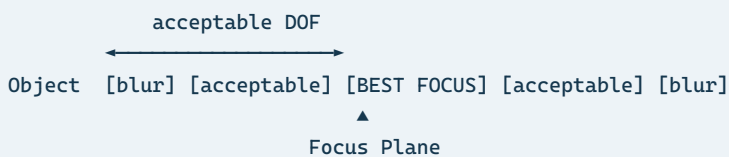
Sensor



$$\text{Object FOV} / \text{Sensor Size} = 1 / \text{Magnification}$$

4.2 Focus와 DOF

TEXT



4.3 일반 Lens와 Telecentric Lens

TEXT	
General Lens	Object-side Telecentric
 Lens	 Lens
높이 변화 → 배율 변화	지정 범위에서 배율 변화 억제

5. 실제 검사 장비에서 어디에 사용하는가?

치수 검사

높이가 ± 0.5 mm 달라지는 부품의 직경을 일반 Lens로 측정하면 Focus뿐 아니라 Perspective Magnification도 달라질 수 있다. 공차가 작다면 Telecentric Lens, 제품 높이 고정, Autofocus 또는 높이별 Calibration을 검토한다.

표면 Defect 검사

Lens MTF가 낮거나 Defocus가 있으면 Scratch의 Contrast가 줄어 Threshold/Edge 결과가 불안정해진다. Camera Pixel 수를 늘리기 전에 Focus Map과 조명 조건을 확인한다.

장비 기구 설계

원하는 FOV만 보고 Lens를 선택했는데 WD에 조명을 설치할 공간이 없을 수 있다. Optics, Lighting, Mechanical Layout을 함께 검토해야 한다.

Multi-Camera Matching

같은 Model Lens라도 장착 높이와 Focus, 조리개가 다르면 FOV/밝기/MTF가 달라진다. Camera별 Calibration과 Golden Image 통계를 관리한다.

Align

FOV 가장자리에서 Distortion이 크면 Pattern 위치와 회전 추정에 Bias가 생긴다. 기준 Pattern 위치를 고정하거나 Distortion Correction 후 Align하고 Calibration Residual을 확인한다.

6. 숫자로 이해하기

예제 1: Sensor와 배율로 FOV 계산

조건:

- Camera: 2448×2048
- Pixel Pitch: 3.45 μm
- Lens Magnification: 0.5×

Sensor Size:

FORMULA · CALCULATION

Width = $2448 \times 3.45 \mu\text{m} = 8445.6 \mu\text{m} = 8.4456 \text{ mm}$
 Height = $2048 \times 3.45 \mu\text{m} = 7065.6 \mu\text{m} = 7.0656 \text{ mm}$

FOV:

FORMULA · CALCULATION

FOV Width = $8.4456 / 0.5 = 16.8912 \text{ mm}$
 FOV Height = $7.0656 / 0.5 = 14.1312 \text{ mm}$

Object Sampling:

FORMULA · CALCULATION

$3.45 \mu\text{m} / 0.5 = 6.9 \mu\text{m/pixel}$

교차 검산:

FORMULA · CALCULATION

$16.8912 \text{ mm} / 2448 \text{ pixel} = 0.0069 \text{ mm/pixel} = 6.9 \mu\text{m/pixel}$

예제 2: Thin Lens 개념 계산

이상적인 Thin Lens에서:

- Focal Length $f = 50 \text{ mm}$
- Magnification $M = 0.1\times$

FORMULA · CALCULATION

$$\begin{aligned}
 u &= f \times (1 + 1/M) \\
 &= 50 \times (1 + 10) \\
 &= 550 \text{ mm}
 \end{aligned}$$

$$\begin{aligned}
 v &= f \times (1 + M) \\
 &= 50 \times 1.1 \\
 &= 55 \text{ mm}
 \end{aligned}$$

검산:

FORMULA · CALCULATION

$$\begin{aligned}
 1/50 &= 1/550 + 1/55 = 0.001818... + 0.018181... = 0.02 \\
 M &= v/u = 55/550 = 0.1
 \end{aligned}$$

$u=550 \text{ mm}$ 는 Lens 주평면 기준 이상값이지 Datasheet의 Mechanical WD와 같다고 단정할 수 없다.

예제 3: DOF 경향 비교

근사 조건:

- $N = 8$
- 허용 Circle of Confusion $c = 0.01 \text{ mm}$

$M=0.5\times$:

FORMULA · CALCULATION

$$\begin{aligned}
 \text{DOF} &\approx 2 \times 8 \times 0.01 \times (1 + 0.5) / 0.5^2 \\
 &= 0.96 \text{ mm}
 \end{aligned}$$

$M=1\times$:

FORMULA · CALCULATION

$$\begin{aligned}
 \text{DOF} &\approx 2 \times 8 \times 0.01 \times (1 + 1) / 1^2 \\
 &= 0.32 \text{ mm}
 \end{aligned}$$

다른 조건이 같다면 배율 증가로 DOF가 크게 감소한다. 이 값은 비교용 근사이며 Lens Datasheet와 실제 Target Test로 확정한다.

예제 4: Sensor 대각선과 Image Circle

Sensor 크기가 8.4456×7.0656 mm 라면:

FORMULA · CALCULATION

$$\begin{aligned}\text{Diagonal} &= \sqrt{8.4456^2 + 7.0656^2} \\ &\approx 11.01 \text{ mm}\end{aligned}$$

Lens의 유효 Image Circle은 최소한 이 대각선을 덮어야 하며, 모서리 MTF와 Vignetting도 별도로 확인한다.

7. C++ 구현

Optical System 1차 계산기

CPP · 1/2

```
#include <cmath>
#include <cstdint>
#include <stdexcept>

struct SensorSpec final {
    std::size_t widthPixels{};
    std::size_t heightPixels{};
    double pixelPitchUm{};
};

struct OpticalResult final {
    double sensorWidthMm{};
    double sensorHeightMm{};
    double sensorDiagonalMm{};
    double fovWidthMm{};
    double fovHeightMm{};
    double objectUmPerPixel{};
};

struct ThinLensResult final {
    double objectDistanceMm{}; // principal-plane based ideal value
    double imageDistanceMm{}; // principal-plane based ideal value
};

[[nodiscard]] bool IsPositiveFinite(const double value) noexcept
{
    return std::isfinite(value) && value > 0.0;
}

[[nodiscard]] OpticalResult CalculateOpticalSystem(
    const SensorSpec& sensor,
    const double magnification)
{
    if (sensor.widthPixels == 0 || sensor.heightPixels == 0 ||
        !IsPositiveFinite(sensor.pixelPitchUm) ||
        !IsPositiveFinite(magnification)) {
        throw std::invalid_argument{"Invalid sensor or magnification"};
    }

    constexpr double umPerMm = 1000.0;
    const double widthMm = static_cast<double>(sensor.widthPixels) *
        sensor.pixelPitchUm / umPerMm;
    const double heightMm = static_cast<double>(sensor.heightPixels) *
        sensor.pixelPitchUm / umPerMm;
```

CPP · 2/2

```

    return {
        widthMm,
        heightMm,
        std::hypot(widthMm, heightMm),
        widthMm / magnification,
        heightMm / magnification,
        sensor.pixelPitchUm / magnification
    };
}

[[nodiscard]] ThinLensResult CalculateIdealThinLens(
    const double focalLengthMm,
    const double magnification)
{
    if (!IsPositiveFinite(focalLengthMm) ||
        !IsPositiveFinite(magnification)) {
        throw std::invalid_argument{"Invalid focal length or magnification"};
    }

    return {
        focalLengthMm * (1.0 + 1.0 / magnification),
        focalLengthMm * (1.0 + magnification)
    };
}

[[nodiscard]] double CalculateApproximateMacroDofMm(
    const double fNumber,
    const double circleOfConfusionMm,
    const double magnification)
{
    if (!IsPositiveFinite(fNumber) ||
        !IsPositiveFinite(circleOfConfusionMm) ||
        !IsPositiveFinite(magnification)) {
        throw std::invalid_argument{"Invalid DOF parameter"};
    }

    return 2.0 * fNumber * circleOfConfusionMm *
        (1.0 + magnification) /
        (magnification * magnification);
}

```

Unit Test

C++

```
#include <cassert>
#include <cmath>

[[nodiscard]] bool NearlyEqual(
    const double lhs,
    const double rhs,
    const double tolerance = 1e-9)
{
    return std::abs(lhs - rhs) <= tolerance;
}

void TestHalfMagnificationFov()
{
    const auto result = CalculateOpticalSystem({2448, 2048, 3.45}, 0.5);
    assert(NearlyEqual(result.sensorWidthMm, 8.4456));
    assert(NearlyEqual(result.sensorHeightMm, 7.0656));
    assert(NearlyEqual(result.fovWidthMm, 16.8912));
    assert(NearlyEqual(result.fovHeightMm, 14.1312));
    assert(NearlyEqual(result.objectUmPerPixel, 6.9));
}

void TestIdealThinLens()
{
    const auto result = CalculateIdealThinLens(50.0, 0.1);
    assert(NearlyEqual(result.objectDistanceMm, 550.0));
    assert(NearlyEqual(result.imageDistanceMm, 55.0));
}

void TestDofTrend()
{
    const double dofAtHalfX = CalculateApproximateMacroDofMm(8.0, 0.01, 0.5);
    const double dofAtOneX = CalculateApproximateMacroDofMm(8.0, 0.01, 1.0);
    assert(NearlyEqual(dofAtHalfX, 0.96));
    assert(NearlyEqual(dofAtOneX, 0.32));
    assert(dofAtOneX < dofAtHalfX);
}
```

코드에서 봐야 할 점

1. 실제 Camera/Lens 선정 계산과 Thin Lens 교육용 계산을 함수로 분리했다.
2. `std::hypot` 으로 Sensor 대각선을 계산한다.
3. 단위가 변수명에 포함되어 $\mu\text{m}/\text{mm}$ 혼용을 줄인다.
4. NaN, Infinity, 0, 음수 입력을 거부한다.
5. DOF 함수 이름에 `ApproximateMacro` 를 넣어 결과의 한계를 드러낸다.
6. 제품 코드에서는 Lens Datasheet 값과 실측 Calibration 결과를 별도 구조체로 관리한다.

[!CAUTION]

Thin Lens와 DOF 함수는 교육 및 1차 비교용이다. 이 계산값만으로 Lens를 구매하거나 장비 공차를 확정하지 않는다.

8. 실무에서 발생하는 문제

문제 1: Sensor보다 작은 Image Circle

- 증상: 모서리 어두움, 가장자리 Resolution 저하
- 대응: Sensor 대각선 Coverage와 Corner MTF 확인

문제 2: WD 변경 후 기존 Calibration 사용

- 증상: 치수 Scale과 Align 보정량이 달라짐
- 대응: 광학/기구 변경을 Calibration 무효화 조건으로 관리

문제 3: 제품 높이 편차가 DOF를 초과

- 증상: Lot 또는 위치별 Edge/Match Score 변화
- 대응: F-number, 배율, Lens, 제품 평탄도, Autofocus 및 Telecentric 적용 검토

문제 4: 조리개를 과도하게 조임

- 증상: Exposure가 길어지고 Motion Blur 또는 Diffraction으로 Detail 감소
- 대응: 조명 세기, Exposure, F-number의 실험적 최적점 결정

문제 5: Distortion을 단일 Scale로 처리

- 증상: 중앙 치수는 맞지만 가장자리에서 오차 증가
- 대응: Grid Calibration, 왜곡 보정, 위치별 Residual Map 확인

문제 6: Focus 조정으로 배율도 바뀐다

- 증상: 선명도는 회복했지만 FOV와 측정값 변화
- 대응: Focus Lock, 장착 기준 관리, Focus 변경 후 Calibration 검증

문제 7: Lens 표면 오염

- 증상: 국부 Contrast 저하, Flare, 얼룩 모양의 False Defect
- 대응: 청소 기준, 보호 Glass 영향 확인, Reference Image/Histogram 감시

9. 흔한 오해

Focal Length는 Lens 특성이고 배율은 Object/Image 거리 조건과 함께 정해진다.

1. **“Focal Length가 곧 Magnification이다.”**

기준면이 다르므로 동일하지 않을 수 있다.

2. **“Working Distance는 Thin Lens의 Object Distance다.”**

DOF는 늘지만 빛이 줄고 Diffraction이 증가할 수 있다.

3. “조리개를 조일수록 항상 선명하다.”

배율, Distortion, Perspective, Calibration Residual도 중요하다.

4. “Focus만 맞으면 치수가 정확하다.”

배율 변화와 Perspective를 줄이지만 장착 Rotation, Scale, Offset, 잔여 Distortion은 검증해야 한다.

5. “Telecentric Lens면 Calibration이 필요 없다.”

Image Circle Coverage와 Corner MTF/Vignetting은 별도 조건이다.

6. “Lens가 Sensor를 덮으면 전체 화질도 같다.”

허용 선명도 기준 내라는 뜻이며 Contrast와 측정 Bias가 완전히 같다는 뜻은 아니다.

7. “DOF 안이면 모든 높이에서 동일 성능이다.”

10. 면접에서 나올 수 있는 질문

Q1. 같은 Camera라도 Lens에 따라 검사 성능이 달라지는 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 Lens가 보이는 범위와 확대 정도, 선명도와 왜곡을 결정하기 때문입니다.

실무자 답변 Lens의 배율이 FOV와 Object Sampling을 결정하고, MTF/수차/Focus가 Defect Contrast를, Distortion/Perspective가 위치·치수 정확도를 결정한다. WD와 DOF는 기구 배치와 높이 공차에도 영향을 준다.

면접용 30초 답변 “Camera는 Sensor Sampling을 담당하지만 Lens는 FOV, 배율, MTF, DOF와 Distortion을 결정합니다. 그래서 같은 Camera라도 Lens에 따라 $\mu\text{m}/\text{pixel}$ 과 결함 Contrast, 치수 오차가 달라집니다. 실제 선정 시 FOV뿐 아니라 WD, Corner MTF, DOF, 왜곡을 함께 검증합니다.”

Q2. Focal Length와 Magnification의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Focal Length는 Lens 자체의 광학 특성이고 Magnification은 특정 거리에서 물체가 Sensor에 몇 배 크기로 맺히는지입니다.

실무자 답변 Focal Length는 Lens의 주평면과 초점 관계를 나타내며 화각과 거리 조건에 관여한다. Magnification은 Image/Object 크기 비이고 Focus/Working 조건에 따라 달라질 수 있다. Fixed Magnification Lens는 지정 WD에서 배율 사양을 제공한다.

면접용 30초 답변 “Focal Length는 mm로 표현하는 Lens의 광학 특성이고 Magnification은 Image Size/Object Size라는 무차원 비율입니다. 같은 50 mm Lens도 Object 거리와 Extension에 따라 배율이 달라질 수 있으므로 둘을 같은 사양으로 보면 안 됩니다.”

Q3. Working Distance가 중요한 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 Lens와 제품 사이에 조명과 기구를 설치할 공간을 정하고, 거리 변화가 배율과 초점에도 영향을 주기 때문입니다.

실무자 답변 WD는 광학 배율/FOV 조건뿐 아니라 조명 배치, Motion 간섭, 유지보수, 높이 공차와 진동 민감도를 제약한다. WD 또는 Focus를 변경하면 FOV와 Calibration Scale이 변할 수 있어 재검증이 필요하다.

면접용 30초 답변 “WD는 단순 초점 거리가 아니라 Lens와 Object 사이의 실제 작업 공간입니다. 원하는 FOV를 만족하면서 조명·Stage와 간섭이 없어야 하고, WD 변경은 배율과 Focus, Calibration에 영향을 줄 수 있어 기구와 광학을 함께 검토합니다.”

Q4. DOF를 늘리려면 어떻게 해야 하나요?

초보자가 이해할 수 있는 답변 조리개를 조이거나 배율을 낮추면 DOF가 늘어나는 경향이 있지만 다른 영향도 확인해야 합니다.

실무자 답변 F-number 증가와 Magnification 감소가 일반적인 방법이다. 다만 F-number 증가는 광량 저하와 Diffraction, Exposure 증가를 만들고 Magnification 감소는 Sampling을 거칠게 한다. 조명 강화, 제품 높이 안정화, Telecentric/특수 Lens, Focus 전략을 함께 검토한다.

면접용 30초 답변 “일반적으로 F-number를 키우거나 배율을 낮추면 DOF가 증가합니다. 하지만 조리개를 과도하게 조이면 Diffraction과 Exposure 증가가 생기고, 배율을 낮추면 $\mu\text{m}/\text{pixel}$ 이 나빠집니다. 그래서 조명, 높이 공차, Sampling과 함께 실험적으로 최적화합니다.”

Q5. Distortion이 검사에 어떤 문제를 만드나요?

초보자가 이해할 수 있는 답변 Image 가장자리에서 직선과 위치가 휘어 보여 실제 거리와 좌표가 틀릴 수 있습니다.

실무자 답변 Distortion은 위치에 따라 이상 투영 좌표에서 벗어나는 기하 오차다. 단일 $\mu\text{m}/\text{pixel}$ 로 치수를 계산하면 위치별 Bias가 발생하고 Align/좌표 변환에도 영향을 준다. Grid Calibration과 왜곡 보정 후 Residual을 검사 공차와 비교한다.

면접용 30초 답변 “Distortion은 Blur가 아니라 위치 오차입니다. 중앙과 가장자리의 Scale이 달라져 치수와 Machine 좌표 변환에 Bias를 만들 수 있습니다. 따라서 Grid Target으로 왜곡을 보정하고 보정 후 Residual이 검사 공차보다 충분히 작은지 확인합니다.”

Q6. Telecentric Lens는 언제 사용하나요?

초보자가 이해할 수 있는 답변 제품 높이가 조금 달라도 크기가 변해 보이지 않게 해야 하는 정밀 치수 검사에 사용합니다.

실무자 답변 높이 공차나 단차로 Perspective/Magnification Error가 문제 되는 치수·위치 검사에 Object-side Telecentric Lens를 고려한다. 요구 FOV, Sensor Size, WD, Telecentric Range, Distortion, Resolution과 비용/크기를 함께 평가한다.

면접용 30초 답변 “Telecentric Lens는 Object 높이 변화에 따른 배율과 Perspective 오차를 줄여야 하는 정밀 치수 검사에 사용합니다. 다만 지정 Telecentric Range와 FOV 안에서만 장점이 있고 크기와 비용이 증가하므로 공차 예산과 Calibration 결과로 필요성을 판단합니다.”

11. 실습 문제

실습 1: Lens 선정표 작성

다음 요구사항을 사용한다.

- Object 검사 영역: 14×10 mm
- Margin: 각 방향 10%
- Camera: 2448×2048, 3.45 μm
- 최소 WD: 100 mm

필요 FOV, 최대 허용 배율, $\mu\text{m}/\text{pixel}$ 을 계산하고 후보 Lens의 배율, WD, Image Circle, Distortion, MTF 확인표를 만든다.

실습 2: 높이 편차와 DOF

제품 높이가 기준면에서 $\pm 0.4 \text{ mm}$ 변한다. DOF 근사 결과가 0.6 mm인 시스템과 1.2 mm인 시스템을 비교한다.

1. 전체 높이 범위를 계산한다.
2. 단순 DOF 기준에서 어느 시스템이 범위를 포함하는가?
3. DOF 경계에서 Measurement Bias를 별도 검증해야 하는 이유는 무엇인가?
4. F-number 증가 외의 개선 방법을 제안한다.

실습 3: Distortion 측정 설계

Grid Target을 사용해 다음 절차를 설계한다.

- 중앙 및 네 모서리의 Point 검출
- 이상적인 Grid Position과 측정 Position 비교
- Pixel Residual과 μm Residual 계산
- 보정 전/후 최대-RMS Residual 비교
- 허용 측정 공차 대비 Calibration 합격 기준 정의

실습 4: C++ Calculator 확장

Chapter 3 Calculator에 다음을 추가한다.

- Sensor Diagonal
- Thin Lens 교육용 u/v 계산
- Approximate DOF 비교
- Lens Image Circle Coverage 판정
- 여러 Lens 후보를 FOV/WD/DOF/Distortion 기준으로 정렬

Phase 3 미니 프로젝트: Optical Configuration Evaluator

TEXT

Inspection Requirement

- Required FOV
- Minimum Feature
- Height Variation
- Minimum WD
- Measurement Tolerance

↓

Camera + Lens Candidate 입력

↓

FOV / Sampling / Sensor Coverage / DOF 비교

↓

Risk Warning

- FOV 부족
- Sampling 부족
- Image Circle 부족
- DOF 부족
- Calibration 필수 조건

결과에는 계산값뿐 아니라 왜 해당 후보가 탈락 또는 보류인지 근거를 남긴다. MTF, Distortion, WD처럼 Datasheet/실측이 필요한 항목을 단순 계산으로 만들어내지 않는다.

12. Chapter 핵심 요약

- Camera, Lens, 조명, Object를 하나의 Optical System으로 봐야 한다.
- Focal Length와 Magnification은 서로 다른 개념이다.
- Object Space와 Image Space의 단위와 의미를 구분한다.
- FOV와 Object $\mu\text{m}/\text{pixel}$ 은 Sensor Size와 배율로 1차 계산한다.
- Working Distance는 광학뿐 아니라 조명·기구 배치를 제약한다.
- 배율이 증가하면 Sampling은 촘촘해지지만 FOV와 DOF가 줄어든다.
- Distortion은 선명도 문제가 아니라 위치·치수의 기하 오차다.
- 조리개를 조이면 DOF는 늘지만 광량 저하와 Diffraction을 고려해야 한다.
- Telecentric Lens는 높이 변화에 따른 Perspective/배율 오차를 줄인다.
- 계산값은 Datasheet와 Calibration Target 실측으로 검증해야 한다.

4일차 권장 학습 순서

- ☐ 40분: Object/Image Space와 Focal Length/Magnification 구분
- ☐ 50분: FOV, Sensor Coverage, Sampling 계산
- ☐ 40분: WD, Focus, DOF 관계와 Trade-off 정리
- ☐ 30분: Distortion과 Telecentric Lens 비교
- ☐ 60~90분: C++ Optical Calculator 확장
- ☐ 60분: Lens 선정표 실습
- ☐ 30분: 면접 Q1~Q6 30초 답변 연습

학습 완료 체크

- ☐ Focal Length와 Magnification을 구분해 설명한다.
- ☐ Working Distance와 Thin Lens의 Object Distance가 다를 수 있음을 안다.
- ☐ FOV, Sensor Diagonal, Object $\mu\text{m}/\text{pixel}$ 을 계산한다.
- ☐ DOF를 늘릴 때의 Trade-off를 설명한다.
- ☐ Distortion과 Blur의 차이를 설명한다.
- ☐ Telecentric Lens가 필요한 검사 조건을 설명한다.
- ☐ Optical Configuration Evaluator를 설계하거나 구현했다.

다음 Chapter 예고

다음 Chapter에서는 FOV, Pixel Count, Pixel Size, Sensor Size, Magnification, Object Resolution을 하나의 계산 체계로 묶고 $0.25\times/0.5\times/1\times/2\times$ 비교 및 검사 요구사항 역산을 집중 연습한다.

Chapter 5. FOV · Resolution · Magnification 집중 계산

Phase 4 · 5일차 · 예상 학습 시간: 10~14시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- FOV, Sensor Size, Pixel Count, Pixel Pitch, Magnification의 연결
- Object Space Sampling($\mu\text{m}/\text{pixel}$)과 Feature의 Pixel 점유 수
- $0.25\times/0.5\times/1\times/2\times$ 배율 비교
- 검사 FOV와 Sampling을 동시에 만족하는 최소 Pixel Count 역산
- Margin을 포함한 실제 요구 FOV 산정
- 설계값과 Calibration 측정값 교차 검산
- Width/Height/Diagonal 조건의 독립 검토
- Camera ROI, Binning, Resize가 계산에 미치는 영향
- Sampling 요구와 Optical Resolution 요구의 분리

실무 목표

Camera와 Lens 사양을 받아 FOV와 $\mu\text{m}/\text{pixel}$ 을 계산하는 데서 끝나지 않고, 검사 대상 크기와 최소 Feature 요구사항으로부터 필요한 Image Pixel Count와 배율을 역산한다. 서로 동시에 만족할 수 없는 요구사항을 계산으로 찾아내고 Camera, Lens, Multi-shot 또는 검사 사양을 조정할 수 있어야 한다.

2. 선수 지식

- Chapter 3의 Pixel Pitch와 Object Space 관계
- Chapter 4의 Camera-Lens-Object System
- 단위 변환: $1\text{ mm} = 1,000\text{ }\mu\text{m}$
- 비율, 반비례, 올림(`ceil`)

2.1 이번 Chapter의 기본식

FORMULA · CALCULATION

Sensor Size = Pixel Count $\times$ Pixel Pitch

FOV = Sensor Size $\div$ Magnification

Object Sampling = Pixel Pitch $\div$ Magnification

Object Sampling = FOV $\div$ Image Pixel Count

Feature Pixels = Feature Size $\div$ Object Sampling

Required Pixel Count = Required FOV $\div$ Required Sampling

길이 단위를 먼저 통일해야 한다. FOV가 mm이면 μm 로 변환한 후 $\mu\text{m}/\text{pixel}$ 과 계산한다.

[!WARNING]

이 Chapter의 **Resolution** 은 문맥상 주로 **Object Space Sampling($\mu\text{m}/\text{pixel}$)**을 뜻한다. Optical Resolution, 반복 정밀도, 측정 정확도와 같은 말이 아니다.

3. 핵심 개념

3.1 FOV란 무엇인가?

FOV(Field of View)는 한 Frame에서 실제 Object Space로 보이는 범위다. 보통 Width×Height를 mm로 표현한다.

FOV는 다음을 모두 포함해야 한다.

- 검사 대상의 최대 크기
- 제품 투입 위치 편차
- Align Pattern과 검사 ROI
- 회전 시 Corner가 움직이는 범위
- 기구 및 Calibration 오차 Margin

제품 폭이 10 mm라고 FOV를 정확히 10 mm로 잡으면 가장자리 Feature가 잘리거나 Align이 실패할 수 있다.

3.2 FOV와 Sampling의 직접 관계

FORMULA · CALCULATION

$$\text{Object Sampling} = \text{FOV} / \text{Pixel Count}$$

같은 Camera Pixel Count에서 FOV가 넓어지면 한 Pixel이 담당하는 Object 영역이 커진다. 즉 더 넓게 보지만 Sampling은 거칠어진다.

TEXT

같은 2048 pixel Camera

FOV 10 mm → 4.8828 $\mu\text{m}/\text{pixel}$

FOV 20 mm → 9.7656 $\mu\text{m}/\text{pixel}$

3.3 배율과 Trade-off

고정 Sensor에서:

TEXT

Magnification ↑ → FOV ↓ → $\mu\text{m}/\text{pixel}$ ↓ → Feature Pixel 수 ↑

Magnification ↓ → FOV ↑ → $\mu\text{m}/\text{pixel}$ ↑ → Feature Pixel 수 ↓

배율을 높이면 작은 특징이 더 많은 Pixel로 나타나지만, 제품 전체 또는 Align Pattern이 FOV 밖으로 나갈 수 있다. 또한 DOF, Focus 민감도 및 조명 배치도 영향을 받는다.

3.4 배율별 기준표

Camera:

- 2048×2048
- Pixel Pitch = 3 μm
- Sensor = 6.144×6.144 mm

Magnification	FOV Width×Height	Object Sampling	24 μm Feature 점유
0.25×	24.576×24.576 mm	12 $\mu\text{m}/\text{pixel}$	2 pixel
0.5×	12.288×12.288 mm	6 $\mu\text{m}/\text{pixel}$	4 pixel
1×	6.144×6.144 mm	3 $\mu\text{m}/\text{pixel}$	8 pixel
2×	3.072×3.072 mm	1.5 $\mu\text{m}/\text{pixel}$	16 pixel

이는 Sampling 비교다. 24 μm Feature의 실제 Contrast와 형태가 2/4/8/16 Pixel에 이상적으로 보인다는 보장은 없다.

3.5 Feature가 몇 Pixel을 차지하는가?

FORMULA · CALCULATION

Feature Pixels = Feature Size / $\mu\text{m-per-pixel}$

20 μm Defect와 5 $\mu\text{m}/\text{pixel}$ 이라면:

FORMULA · CALCULATION

20 μm / 5 $\mu\text{m}/\text{pixel}$ = 4 pixel

하지만 실제 Scratch Width, Particle Diameter, Edge Transition은 다른 방식으로 Pixel에 나타난다. “최소 4 Pixel” 같은 규칙은 알고리즘과 품질 목표에 맞춰 검증된 내부 기준이어야 한다.

3.6 요구사항 역산

FOV와 필요한 Sampling이 주어졌다면 최소 Pixel Count는:

FORMULA · CALCULATION

Required Pixel Count = Required FOV / Maximum Allowed $\mu\text{m-per-pixel}$

12 mm FOV에서 3 $\mu\text{m}/\text{pixel}$ 이하가 필요하다면:

FORMULA · CALCULATION

$$12 \text{ mm} = 12,000 \text{ } \mu\text{m}$$

$$12,000 / 3 = 4,000 \text{ pixel}$$

따라서 Width가 최소 4000 Pixel인 Camera가 필요하다. Height 방향도 별도로 계산하고 실제 Camera Resolution 후보로 올림한다.

3.7 배율 역산

Camera Pixel Pitch **p** 와 목표 Object Sampling **r** 로 필요한 최소 배율을 구할 수 있다.

FORMULA · CALCULATION

$$M = p / r$$

3 μm Pixel Camera에서 2 $\mu\text{m}/\text{pixel}$ 이 필요하다면:

FORMULA · CALCULATION

$$M = 3 / 2 = 1.5\times$$

그러나 이 배율에서 필요한 FOV도 만족하는지 반드시 확인한다. 4096 Pixel Sensor라면 Sensor Width는 12.288 mm이고 FOV는 $12.288 / 1.5 = 8.192 \text{ mm}$ 다. 12 mm FOV 요구와 동시에 만족하지 못한다.

3.8 FOV와 Sampling 동시 만족 조건

Camera Pixel Pitch와 배율은 FOV와 Sampling 양쪽에 들어가지만 다음 항등식이 핵심이다.

FORMULA · CALCULATION

$$\text{FOV} / \text{Sampling} = \text{Pixel Count}$$

따라서 FOV 12 mm와 2 $\mu\text{m}/\text{pixel}$ 을 동시에 요구하면:

FORMULA · CALCULATION

$$12,000 \text{ } \mu\text{m} / 2 \text{ } \mu\text{m}/\text{pixel} = 6,000 \text{ pixel}$$

가로 최소 6000 Pixel이 필요하다. 배율만 변경해 Pixel Count 부족을 해결할 수 없다.

대안:

- 더 높은 Resolution Camera
- FOV 축소
- 허용 Sampling 완화
- Camera 여러 대 또는 Multi-shot/Tiling
- 검사 항목별 Camera 분리

3.9 Margin 산정

제품 Width가 10 mm이고 좌우 위치 편차가 각각 최대 ± 0.5 mm라면 최소 FOV는 단순히 10 mm가 아니다.

FORMULA · CALCULATION

Minimum FOV Width = Product Width + 2 × Position Variation + Safety Margin

양쪽 Safety Margin을 0.25 mm씩 추가하면:

FORMULA · CALCULATION

$10 + 2 \times 0.5 + 2 \times 0.25 = 11.5 \text{ mm}$

Rotation이 있다면 Axis-aligned Bounding Box가 더 커진다. Width **W**, Height **H** 의 Rectangle을 **θ** 만큼 회전했을 때 필요한 Bounding Box는:

FORMULA · CALCULATION

Rotated Width = $|W \cos\theta| + |H \sin\theta|$

Rotated Height = $|W \sin\theta| + |H \cos\theta|$

3.10 설계값과 Calibration 값

설계 계산:

TEXT

Pixel Pitch / Nominal Magnification

실측 계산:

TEXT

Known Object Distance / Measured Pixel Distance

실제 Measurement에는 Calibration 값을 사용한다. 설계값은 후보 선정과 예상 성능에, 실측값은 장비 좌표·치수 변환에 사용한다.

3.11 Width와 Height를 따로 확인

가로 조건만 만족하고 세로 FOV가 부족할 수 있다. Camera Resolution과 Sensor Size는 비율이 다를 수 있고 제품 Rotation이 Height 요구를 키울 수 있다.

TEXT

Horizontal: FOV_x / N_x

Vertical: FOV_y / N_y

정사각 Pixel이고 왜곡이 작다면 X/Y Scale이 비슷해야 하지만 Calibration에서 각각 확인한다.

3.12 Camera ROI, Binning, Resize

- Camera ROI: Sensor 일부를 읽으므로 FOV가 줄지만 1:1 Readout이면 $\mu\text{m}/\text{pixel}$ 은 유지
- 2×2 Binning: 출력 Pixel 수가 절반이고 Output Pixel당 Object 영역은 보통 2배
- Software Crop: FOV는 줄지만 원본 $\mu\text{m}/\text{pixel}$ 유지
- Resize: 출력 배열 Sampling 표현이 변하지만 새 Optical Detail은 생성되지 않음

같은 1024×1024 출력이라도 어떤 과정으로 만들어졌는지 알아야 올바르게 계산한다.

4. 그림으로 이해하기

4.1 설계 흐름

TEXT

Inspection Requirement

└ Product Size + Position/Rotation Margin → Required FOV
└ Minimum Feature + Required Pixels → Required $\mu\text{m}/\text{pixel}$



Required Pixel Count = $FOV / \mu\text{m-per-pixel}$



Camera Resolution Candidate



Pixel Pitch / Magnification → Lens Candidate



WD / DOF / MTF / Distortion / Light 검증



Calibration + Sample Test

4.2 FOV와 Detail의 줄다리기

TEXT	
Wide FOV	Narrow FOV
product + margin small feature = 2 px	detail = 8 px
낮은 배율, 거친 Sampling	높은 배율, 촘촘한 Sampling

4.3 요구사항 충돌

TEXT

FOV 12 mm / 2 $\mu\text{m-per-pixel}$ = 6000 horizontal pixels required

4096 px Camera

- FOV를 12 mm로 맞추면 2.93 $\mu\text{m/px}$
- 2 $\mu\text{m/px}$ 로 맞추면 FOV는 8.192 mm

배율 변경만으로 두 조건을 동시에 만족할 수 없음

5. 실제 검사 장비에서 어디에 사용하는가?

Camera/Lens 사양 협의

“5 MP Camera”보다 필요 FOV , 목표 $\mu\text{m/pixel}$, Feature당 최소 Pixel , WD , DOF 를 먼저 제시해야 비전 엔지니어가 적절한 조합을 검토할 수 있다.

Align ROI 확보

검사 ROI만 가까스로 FOV에 들어오더라도 제품이 이동·회전하면 Fiducial이 잘릴 수 있다. Align Search Range와 회전 Bounding Box를 FOV Margin에 포함한다.

Recipe 변경

제품 크기가 커진 새 Recipe가 기존 FOV를 초과할 수 있다. Parameter만 바꾸기 전에 Optical Configuration의 적용 가능 범위를 검사한다.

Multi-shot 검사

한 Frame에서 넓은 FOV와 미세 Sampling을 동시에 만족할 수 없으면 Stage 이동 후 여러 Frame을 촬영한다. Stitching 오차, Cycle Time, Overlap, 좌표 Calibration을 추가 고려한다.

Camera ROI 최적화

검사 영역이 Sensor 일부에만 있고 제품 편차가 작다면 Camera ROI로 Frame Rate와 대역폭을 개선할 수 있다. 단, $\mu\text{m/pixel}$ 이 좋아지는 것은 아니며 FOV만 줄어든다.

6. 숫자로 이해하기

예제 1: 0.25×~2× 전체 비교

Camera: 2048×2048 , 3 μm

FORMULA · CALCULATION

Sensor Width = 2048 × 3 μm = 6.144 mm

M	FOV	μm/pixel	30 μm Feature
0.25×	24.576 mm	12	2.5 pixel
0.5×	12.288 mm	6	5 pixel
1×	6.144 mm	3	10 pixel
2×	3.072 mm	1.5	20 pixel

예제 2: FOV로 Sampling과 Feature Pixel 계산

조건:

- Horizontal FOV = 10 mm
- Image Width = 2048 pixel
- Feature Width = 20 μm

FORMULA · CALCULATION

Sampling = 10,000 μm / 2048 pixel
= 4.8828125 μm/pixel

Feature Pixels = 20 μm / 4.8828125 μm/pixel
= 4.096 pixel

예제 3: 최소 Pixel Count 역산

조건:

- Required FOV = 18 mm
- Feature = 30 μm
- 내부 기준: Feature가 최소 5 Pixel

허용 Sampling:

FORMULA · CALCULATION

$$30 \mu\text{m} / 5 \text{ pixel} = 6 \mu\text{m/pixel}$$

필요 Pixel Count:

FORMULA · CALCULATION

$$18,000 \mu\text{m} / 6 \mu\text{m/pixel} = 3000 \text{ pixel}$$

가로 3000 Pixel 이상의 Camera가 필요하다. 실제 후보가 3072 또는 4096 Pixel이면 그 후보의 Height/FOV도 별도로 확인한다.

예제 4: Rotation Margin

제품 크기 $W=10 \text{ mm}$, $H=6 \text{ mm}$, 최대 회전 $\theta=5^\circ$ 라면:

FORMULA · CALCULATION

$$\begin{aligned} \text{Rotated Width} \\ &= 10 \times \cos 5^\circ + 6 \times \sin 5^\circ \\ &\approx 10.485 \text{ mm} \end{aligned}$$

$$\begin{aligned} \text{Rotated Height} \\ &= 10 \times \sin 5^\circ + 6 \times \cos 5^\circ \\ &\approx 6.849 \text{ mm} \end{aligned}$$

여기에 Translation 편차와 Safety Margin을 추가해야 한다.

예제 5: 설계값과 실측값 오차

설계 Sampling이 $6 \mu\text{m/pixel}$, Calibration Target 실측이 $6.08 \mu\text{m/pixel}$ 이면:

FORMULA · CALCULATION

$$\begin{aligned} \text{Relative Difference} \\ &= |6.08 - 6.00| / 6.00 \times 100 \\ &\approx 1.333\% \end{aligned}$$

이 차이를 무시하고 1000 Pixel을 6 mm로 계산하면 실제 Scale 기준으로는 6.08 mm여서 0.08 mm 차이가 난다.

7. C++ 구현

FOV 요구사항 후보 평가기

CPP · 1/2

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <stdexcept>

struct AxisRequirement final {
    double requiredFovMm{};
    double minimumFeatureUm{};
    double requiredPixelsPerFeature{};
};

struct AxisDesign final {
    std::size_t availablePixels{};
    double pixelPitchUm{};
    double magnification{};
};

struct AxisEvaluation final {
    double sensorSizeMm{};
    double actualFovMm{};
    double objectUmPerPixel{};
    double featurePixels{};
    std::size_t minimumRequiredPixels{};
    bool fovSatisfied{};
    bool samplingSatisfied{};
};

[[nodiscard]] bool IsPositiveFinite(const double value) noexcept
{
    return std::isfinite(value) && value > 0.0;
}

[[nodiscard]] AxisEvaluation EvaluateAxis(
    const AxisRequirement& requirement,
    const AxisDesign& design)
{
    if (!IsPositiveFinite(requirement.requiredFovMm) ||
        !IsPositiveFinite(requirement.minimumFeatureUm) ||
        !IsPositiveFinite(requirement.requiredPixelsPerFeature) ||
        design.availablePixels == 0 ||
        !IsPositiveFinite(design.pixelPitchUm) ||
        !IsPositiveFinite(design.magnification)) {
        throw std::invalid_argument{"All dimensions must be positive"};
    }
}
```

CPP · 2/2

```

const double maximumUmPerPixel =
    requirement.minimumFeatureUm /
    requirement.requiredPixelsPerFeature;
const double requiredFovUm = requirement.requiredFovMm * 1000.0;
const auto minimumRequiredPixels = static_cast<std::size_t>{
    std::ceil(requiredFovUm / maximumUmPerPixel)};

const double sensorSizeMm =
    static_cast<double>(design.availablePixels) *
    design.pixelPitchUm / 1000.0;
const double actualFovMm = sensorSizeMm / design.magnification;
const double objectUmPerPixel =
    design.pixelPitchUm / design.magnification;
const double featurePixels =
    requirement.minimumFeatureUm / objectUmPerPixel;

constexpr double tolerance = 1e-12;
return {
    sensorSizeMm,
    actualFovMm,
    objectUmPerPixel,
    featurePixels,
    minimumRequiredPixels,
    actualFovMm + tolerance >= requirement.requiredFovMm,
    featurePixels + tolerance >= requirement.requiredPixelsPerFeature
};
}

[[nodiscard]] double CalculateRotatedBoundingWidthMm(
    const double widthMm,
    const double heightMm,
    const double angleDegrees)
{
    if (!IsPositiveFinite(widthMm) || !IsPositiveFinite(heightMm) ||
        !std::isfinite(angleDegrees)) {
        throw std::invalid_argument{"Invalid rectangle or angle"};
    }

    constexpr double pi = 3.14159265358979323846;
    const double radians = angleDegrees * pi / 180.0;
    return std::abs(widthMm * std::cos(radians)) +
        std::abs(heightMm * std::sin(radians));
}

```

Unit Test

CPP

```
#include <cassert>
#include <cmath>

[[nodiscard]] bool NearlyEqual(
    const double lhs,
    const double rhs,
    const double tolerance = 1e-9)
{
    return std::abs(lhs - rhs) <= tolerance;
}

void TestHalfMagnificationCandidate()
{
    const AxisRequirement requirement{12.0, 30.0, 5.0};
    const AxisDesign design{2048, 3.0, 0.5};
    const auto result = EvaluateAxis(requirement, design);

    assert(NearlyEqual(result.sensorSizeMm, 6.144));
    assert(NearlyEqual(result.actualFovMm, 12.288));
    assert(NearlyEqual(result.objectUmPerPixel, 6.0));
    assert(NearlyEqual(result.featurePixels, 5.0));
    assert(result.minimumRequiredPixels == 2000);
    assert(result.fovSatisfied);
    assert(result.samplingSatisfied);
}

void TestRequirementConflict()
{
    // 12 mm FOV at 2 um/pixel requires at least 6000 pixels.
    const AxisRequirement requirement{12.0, 10.0, 5.0};
    const AxisDesign design{4096, 3.0, 1.5};
    const auto result = EvaluateAxis(requirement, design);

    assert(result.minimumRequiredPixels == 6000);
    assert(NearlyEqual(result.objectUmPerPixel, 2.0));
    assert(result.samplingSatisfied);
    assert(!result.fovSatisfied); // actual FOV = 8.192 mm
}

void TestRotatedWidth()
{
    const double width = CalculateRotatedBoundingWidthMm(10.0, 6.0, 5.0);
    assert(std::abs(width - 10.485) < 0.001);
}
```

코드에서 봐야 할 점

1. 요구사항과 Camera/Lens 설계값을 별도 구조체로 나눴다.
2. 최소 Pixel Count는 부족하지 않도록 `std::ceil` 로 올림한다.
3. FOV 만족과 Sampling 만족을 별도 Boolean으로 반환한다.
4. 한 조건만 만족하는 후보를 합격 처리하지 않는다.
5. 단위가 변수명에 포함되어 mm/μm 혼용을 줄인다.
6. 실제 시스템에서는 X/Y 축을 각각 평가하고 WD, DOF, MTF, Distortion을 추가한다.

8. 실무에서 발생하는 문제

문제 1: 제품 크기만 FOV에 반영

- 증상: 위치 편차나 회전 시 제품/Align Pattern이 잘림
- 대응: Translation, Rotation Bounding Box, Safety Margin 포함

문제 2: 가로만 계산하고 세로 FOV 누락

- 증상: Landscape 제품은 들어오지만 회전 시 Height 방향이 잘림
- 대응: X/Y 요구사항과 Sensor Aspect Ratio를 독립 검토

문제 3: 최소 Feature가 1 Pixel이면 된다고 가정

- 증상: Defect 위치와 조명 변화에 따라 검출률 급변
- 대응: 알고리즘별 필요 Sampling을 Sample Test로 정의

문제 4: 배율만 올려 요구사항을 해결하려 함

- 증상: Sampling은 만족하지만 FOV 부족
- 대응: $\text{Required Pixels} = \text{FOV} / \text{Sampling}$ 으로 Pixel Count부터 검토

문제 5: Camera ROI와 Binning을 동일하게 취급

- 증상: Output Image 크기는 같지만 Scale 계산이 틀림
- 대응: Sensor Readout Mode와 Output Pixel 의미를 Recipe/Calibration에 기록

문제 6: 명목 FOV로 치수 계산

- 증상: 장비별 측정값이 일정 비율로 다름
- 대응: Calibration Target 기반 실측 Scale과 Residual 사용

문제 7: Multi-shot 선택 후 Cycle Time 누락

- 증상: 광학 요구는 만족하지만 생산성 미달
- 대응: Stage Move/Settle/Trigger/Processing/Stitching 시간 예산 포함

9. 흔한 오해

고정 Pixel Count에서는 FOV와 $\mu\text{m/pixel}$ 이 직접 연결되므로 동시 요구는 Camera Pixel Count를 결정한다.

1. “FOV가 넓고 Resolution도 좋은 Lens를 고르면 된다.”

Sampling은 좋아지지만 FOV와 DOF는 줄어든다.

2. “배율을 높이면 모든 요구사항이 좋아진다.”

Sampling 간격일 뿐이며 Optical Blur와 Pixel Aperture로 에너지가 분산된다.

3. “2 $\mu\text{m}/\text{pixel}$ 이면 2 μm Defect가 1 Pixel로 정확히 보인다.”

1:1 Crop이면 Scale은 유지되고 FOV와 데이터량만 줄어든다.

4. “Camera ROI를 줄이면 $\mu\text{m}/\text{pixel}$ 이 좋아진다.”

배열 크기는 늘지만 Optical Information은 늘지 않는다.

5. “Image를 2배 Resize하면 Resolution이 2배다.”

위치, Orientation, Blur, Threshold에 따라 실제 표현은 달라진다.

6. “계산상 4.1 Pixel이면 언제나 4 Pixel로 보인다.”

실제 장착 후 Calibration Scale을 사용해야 한다.

7. “설계 배율로 치수를 계산해도 충분하다.”

10. 면접에서 나올 수 있는 질문

Q1. FOV와 Resolution의 관계를 설명해 주세요.

초보자가 이해할 수 있는 답변 같은 Pixel 수에서 더 넓은 영역을 보면 한 Pixel이 담당하는 실제 크기가 커져 작은 특징 표현이 거칠어집니다.

실무자 답변 Object Sampling은 $\text{FOV}/\text{Pixel Count}$ 다. 고정 Camera에서 FOV가 증가하면 $\mu\text{m}/\text{pixel}$ 이 증가한다. 요구 FOV와 허용 $\mu\text{m}/\text{pixel}$ 로 최소 Pixel Count를 먼저 역산하고 Lens 배율, WD, MTF와 DOF를 검토한다.

면접용 30초 답변 “같은 Camera에서는 $\mu\text{m}/\text{pixel} = \text{FOV}/\text{Pixel Count}$ 이므로 FOV를 넓히면 Sampling이 거칠어집니다. 그래서 제품 크기와 위치 Margin으로 FOV를, 최소 Feature와 필요한 Pixel 수로 $\mu\text{m}/\text{pixel}$ 을 정한 뒤 두 조건으로 최소 Camera Resolution을 계산합니다.”

Q2. Magnification이 0.5×에서 1×가 되면 무엇이 바뀌나요?

초보자가 이해할 수 있는 답변 배율이 두 배가 되므로 보이는 범위와 물체 기준 $\mu\text{m}/\text{pixel}$ 은 절반이 됩니다.

실무자 답변 고정 Sensor에서 FOV와 Object Sampling은 배율에 반비례한다. Feature Pixel 수는 두 배가 되지만 DOF 감소, Focus 민감도, WD/조명 배치 변화를 함께 평가한다.

면접용 30초 답변 “0.5×에서 1×로 올리면 같은 Sensor의 FOV와 $\mu\text{m}/\text{pixel}$ 은 절반이 되고 Feature가 차지하는 Pixel 수는 두 배가 됩니다. 대신 제품 전체가 FOV에 들어오는지와 DOF, Focus, WD 조건을 다시 확인해야 합니다.”

Q3. 12 mm FOV에서 2 $\mu\text{m}/\text{pixel}$ 이 필요하면 최소 몇 Pixel인가요?

초보자가 이해할 수 있는 답변 12 mm는 12,000 μm 이므로 2로 나누면 가로 6000 Pixel이 필요합니다.

실무자 답변 이상적 최소 Width는 $\text{ceil}(12,000/2)=6000 \text{ pixel}$ 이다. 실제 후보 Resolution, Margin, Camera ROI, Optical MTF와 Feature별 최소 Sampling을 추가 검증한다.

면접용 30초 답변 “ $\text{Required Pixels} = \text{FOV}/\mu\text{m-per-pixel}$ 이므로 $12,000/2=6000 \text{ pixel}$ 입니다. 이는 Sampling의 최소 조건이고 실제 선택에서는 FOV Margin, Sensor Height, Lens MTF와 검출률을 추가 확인합니다.”

Q4. Camera ROI와 Binning의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 ROI는 Sensor 일부만 잘라 읽는 것이고 Binning은 여러 Sensor Pixel 신호를 합쳐 출력 Pixel 하나로 만드는 것입니다.

실무자 답변 1:1 Camera ROI는 FOV와 데이터량을 줄이지만 원래 $\mu\text{m}/\text{pixel}$ 을 유지한다. 2×2 Binning은 출력 Pixel Count를 절반으로 만들며 Output Pixel의 Object Sampling이 대략 두 배가 되고 감도/Noise 특성도 달라질 수 있다.

면접용 30초 답변 “Camera ROI는 Sensor 일부를 1:1로 읽는 Crop이라 일반적으로 $\mu\text{m}/\text{pixel}$ 은 유지되고 FOV만 줄어듭니다. Binning은 인접 Pixel 신호를 합쳐 출력 Pixel 수와 Sampling 의미가 바뀌므로 기존 Calibration을 그대로 쓰면 안 됩니다.”

Q5. 작은 Defect가 최소 몇 Pixel이어야 하나요?

초보자가 이해할 수 있는 답변 한 가지 고정 답은 없습니다. 결함 형태와 Lens, 조명, Noise 및 알고리즘에 따라 실제 Sample로 정해야 합니다.

실무자 답변 Detection과 Measurement의 필요 Sampling이 다르다. Point-like Particle, Scratch Width, Edge 간격은 영상 형성이 다르므로 MTF/PSF, Contrast, SNR, Orientation, Sub-pixel 위치와 목표 Detection/False Positive Rate를 포함해 내부 기준을 검증한다.

면접용 30초 답변 “고정된 정답은 없습니다. 먼저 알고리즘 후보에 필요한 Pixel 수를 가정해 광학계를 설계하고, Lens MTF·조명·Noise·Defect 위치를 포함한 실제 Sample로 검출률과 오검률을 측정해 최소 Sampling 기준을 확정합니다.”

Q6. 설계 $\mu\text{m}/\text{pixel}$ 과 Calibration $\mu\text{m}/\text{pixel}$ 이 다른 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 Lens 장착 거리와 Focus, 제조 공차 때문에 실제 배율이 사양값과 조금 달라질 수 있습니다.

실무자 답변 명목 배율, WD, Focus, Camera/Lens 자세, Distortion 및 Target 측정 오차가 차이를 만든다. 설계값은 후보 선정에 사용하고 실제 Measurement에는 Traceable Target으로 얻은 Calibration Transform과 Residual을 사용한다.

면접용 30초 답변 “설계값은 Pixel Pitch/명목 배율이지만 실제 장착에서는 WD, Focus, 공차와 Distortion으로 유효 배율이 달라집니다. 따라서 치수와 좌표 변환에는 Calibration Target으로 얻은 실제 Scale과 보정 후 Residual을 사용합니다.”

11. 실습 문제

실습 1: 배율 비교표 작성

Camera 4096×3000 , Pixel Pitch 3.45 μm 에 대해 0.25×/0.5×/1×/2×의 다음 값을 계산한다.

- Sensor Width/Height
- FOV Width/Height
- X/Y $\mu\text{m}/\text{pixel}$
- 20 μm 와 50 μm Feature의 Pixel 점유 수

실습 2: 요구사항 충돌 찾기

조건:

- Required FOV: 25×18 mm
- 최소 Defect: 30 μm
- 내부 Sampling 기준: 최소 6 Pixel
- 후보 Camera: 4096×3000

필요 $\mu\text{m}/\text{pixel}$, 최소 Width/Height Pixel Count를 구하고 후보 Camera의 합격 여부와 대안을 제안한다.

실습 3: Rotation Margin 계산

제품 20×8 mm, 위치 편차 X/Y 각각 ±0.5 mm, 회전 ±3°, 양쪽 Safety Margin 0.3 mm일 때 필요한 최대 FOV Width/Height를 계산한다.

실습 4: Calculator 구현

C++ 후보 평가기에 다음을 추가한다.

- X/Y 동시 평가
- 여러 Camera/Lens 후보 CSV 입력
- FOV/Sampling/Pixel Count 합격 사유 출력
- Calibration 실측 Scale과 설계값 오차율 Warning
- Invalid Unit 및 Overflow 방지 Test

Phase 4 미니 프로젝트: Vision Requirement Calculator

TEXT

Product Geometry + Position/Rotation Margin

↓

Required FOV

Minimum Feature + Required Feature Pixels

↓

Required $\mu\text{m-per-pixel}$

Required FOV / $\mu\text{m-per-pixel}$

↓

Minimum Camera Pixel Count

↓

Camera/Lens Candidate Evaluation

↓

Pass / Fail / Need Measurement

필수 출력

- X/Y Required FOV
- X/Y Required Pixel Count
- 후보별 실제 FOV와 $\mu\text{m/pixel}$
- Feature Pixel 수
- FOV/Sampling 각각의 합격 여부
- WD/DOF/MTF/Distortion처럼 계산만으로 확정할 수 없는 항목

12. Chapter 핵심 요약

- $\mu\text{m}/\text{pixel} = \text{FOV}/\text{Pixel Count}$ 다.
- FOV가 넓어지면 같은 Pixel Count에서 Sampling은 거칠어진다.
- 배율이 증가하면 FOV와 $\mu\text{m}/\text{pixel}$ 은 감소하고 Feature Pixel 수는 증가한다.
- $\text{Feature Pixels} = \text{Feature Size}/\mu\text{m-per-pixel}$ 이다.
- $\text{Required Pixel Count} = \text{Required FOV}/\text{Required Sampling}$ 이다.
- FOV와 Sampling 동시 요구는 최소 Camera Pixel Count를 결정한다.
- 제품 크기뿐 아니라 Translation, Rotation, Align, Safety Margin을 FOV에 포함한다.
- X/Y 방향과 Sensor Aspect Ratio를 각각 확인한다.
- Camera ROI, Binning, Resize는 같은 출력 크기라도 의미가 다르다.
- Sampling 계산 후에도 MTF, DOF, 조명, Noise 및 실제 검출률을 검증한다.

5일차 권장 학습 순서

- ☐ 30분: 기본식 6개를 보지 않고 작성
- ☐ 60분: $0.25\times\sim 2\times$ 비교표 손 계산
- ☐ 60분: 요구사항 역산 예제 2~5 풀이
- ☐ 60~90분: C++ 후보 평가기 구현 및 Unit Test
- ☐ 45분: Rotation Margin 실습
- ☐ 30분: 면접 Q1~Q6 30초 답변 연습
- ☐ 20분: 현재 업무 장비의 FOV/Pixel Count/Scale 조사 목록 작성

학습 완료 체크

- ☐ FOV, Pixel Count, $\mu\text{m}/\text{pixel}$ 을 양방향으로 계산한다.
- ☐ 배율별 FOV와 Feature Pixel 수를 비교한다.
- ☐ 요구 FOV와 Sampling으로 최소 Camera Resolution을 역산한다.
- ☐ 배율 변경만으로 해결할 수 없는 요구사항을 찾는다.
- ☐ Translation/Rotation/Safety Margin을 FOV에 반영한다.
- ☐ 설계값과 Calibration 값의 사용 목적을 구분한다.
- ☐ Vision Requirement Calculator를 설계하거나 구현했다.

다음 Chapter 예고

다음 Chapter에서는 Sampling 계산만으로 실제 검사 Resolution을 보장할 수 없는 이유를 Lens MTF, Nyquist, Blur, Focus, Motion, Illumination과 연결해 학습한다.

Chapter 6. Optical Resolution과 실제 검사 Resolution

Phase 5 · 6일차 · 예상 학습 시간: 12~16시간 · 난이도: 중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- Pixel Sampling과 실제 검출 능력의 차이
- Nyquist Frequency와 Alias
- Optical Resolution과 Lens MTF
- PSF, Blur, Focus, Defocus
- Motion Blur와 Exposure Time
- Illumination Contrast, Noise, SNR/CNR
- 2 μm Defect와 3 μm Pixel Camera 사례 분석
- Diffraction, Numerical Aperture(NA), F-number
- Detection과 Measurement의 요구 조건 차이
- Resolution Budget과 실제 Sample 검증

실무 목표

Object Sampling = 3 $\mu\text{m}/\text{pixel}$ 이라는 계산만으로 3 μm Defect 검출 가능 이라고 결론 내리지 않는다.

Sampling, Optical Transfer, Motion, Contrast/Noise, 알고리즘 조건을 분리해 검토하고, 검출률과 오검률로 최종 성능을 검증하는 절차를 설계한다.

2. 선수 지식

- Chapter 5의 FOV/Sampling 계산
- Object $\mu\text{m}/\text{pixel}$ = Pixel Pitch/Magnification
- 주기(Period)와 공간 주파수(Spatial Frequency)의 역수 관계
- 평균, 표준편차, Contrast의 기초

2.1 용어와 단위

용어	의미	대표 단위
Sampling Pitch	인접 Image Sample 간 Object 거리	$\mu\text{m}/\text{pixel}$
Spatial Frequency	길이당 밝기 패턴 반복 수	lp/mm, cycles/mm
MTF	공간 주파수별 Contrast 전달 비율	0~1 또는 %
PSF	점 Object가 Image에 퍼지는 모양	Pixel 또는 μm
SNR	Signal 크기 대비 Noise	비율, dB
CNR	Defect와 Background 차이 대비 Noise	비율

Line Pair(lp)는 밝은 선 하나와 어두운 선 하나의 한 쌍이다. 100 lp/mm 이면 한 주기의 길이는 $1/100 \text{ mm} = 10 \text{ }\mu\text{m}$ 이고 이상적으로 밝고 어두운 선 하나씩은 약 $5 \text{ }\mu\text{m}$ 폭이다.

3. 핵심 개념

3.1 검사 성능은 여러 단계의 곱이다

TEXT

Defect의 물리적 형상/반사 특성
 ↓ Illumination이 Contrast 생성
 Lens/Optics가 공간 정보를 전달(MTF/PSF)
 ↓
 Sensor가 공간과 밝기를 Sampling
 ↓ Exposure/Gain/Noise/ADC
 Digital Image
 ↓ Preprocessing/Algorithm
 Detection / Measurement / Judgement

어느 한 단계에서 정보가 사라지면 뒤 단계가 완전히 복구할 수 없다. Sharpness Filter는 이미 존재하는 Edge Contrast를 강조할 수 있지만 Optical Blur로 없어진 세부를 새로 만들지는 못한다.

3.2 Sampling Pitch와 Resolution

$5 \text{ }\mu\text{m/pixel}$ 은 Object Space에서 Image Sample 중심 간격이 $5 \text{ }\mu\text{m}$ 라는 뜻이다. 이는 다음과 같지 않다.

- $5 \text{ }\mu\text{m}$ Defect를 항상 검출
- $5 \text{ }\mu\text{m}$ 치수를 $5 \text{ }\mu\text{m}$ 정확도로 측정
- 1 Pixel보다 작은 Feature는 절대 영향이 없음

Sub-pixel Feature도 한 Pixel이 모으는 빛의 평균을 바꿀 수 있다. 하지만 Feature 위치, Fill Factor, PSF, Contrast와 Noise에 따라 신호가 달라져 안정성이 낮을 수 있다.

3.3 Nyquist Sampling

주기적인 신호를 Alias 없이 Sampling하려면 이상적으로 한 주기당 최소 2 Sample이 필요하다.

FORMULA · CALCULATION

Sensor Nyquist Frequency = $1 / (2 \times \text{Pixel Pitch})$

$3 \text{ }\mu\text{m}$ Pixel은 0.003 mm 이므로 Image Space Nyquist는:

FORMULA · CALCULATION

$1 / (2 \times 0.003 \text{ mm}) = 166.67 \text{ cycles/mm} \approx 166.67 \text{ lp/mm}$

Object Space Sampling이 $6 \text{ }\mu\text{m/pixel}$ 이면 Object Space Nyquist는:

FORMULA · CALCULATION

$$1 / (2 \times 0.006 \text{ mm}) = 83.33 \text{ lp/mm}$$

[!WARNING]

Nyquist의 2 Sample 기준은 이상적인 주기 신호를 복원하기 위한 이론적 하한이다. 단일 Particle, Scratch, Edge의 안정적 검출·치수 측정에 “2 Pixel이면 충분하다”는 뜻이 아니다.

3.4 Alias

Sampling 가능한 주파수보다 높은 Detail은 사라지지만 하는 것이 아니라 낮은 주파수의 거짓 패턴으로 보일 수 있다.

TEXT

Fine periodic pattern + insufficient sampling



Moiré / false spacing / unstable brightness

반복 회로 Pattern, Mesh, Display Pixel, Wafer Pattern 검사에서 특히 주의한다. Lens Blur나 Optical Low-pass 특성이 일부 고주파를 줄이지만 검사 Detail도 함께 약해질 수 있다.

3.5 MTF(Modulation Transfer Function)

Object Pattern의 Contrast:

FORMULA · CALCULATION

$$\text{Contrast} = (I_{\text{max}} - I_{\text{min}}) / (I_{\text{max}} + I_{\text{min}})$$

MTF는 특정 공간 주파수에서 Image Contrast가 Object Contrast 대비 얼마나 남는지 나타낸다.

FORMULA · CALCULATION

$$\text{MTF}(f) = \text{Image Contrast}(f) / \text{Object Contrast}(f)$$

예를 들어 Object Contrast가 0.8이고 광학계 MTF가 0.25라면 이상적인 Image Contrast는 약 $0.8 \times 0.25 = 0.2$ 다. Sensor와 Noise를 거치면 실제 검출 가능한 Contrast는 더 낮아질 수 있다.

Lens의 MTF Chart를 볼 때 확인할 항목:

- 공간 주파수(lp/mm)
- Image Center와 Corner 위치
- Sagittal/Tangential 방향
- 사용 F-number와 파장
- 대상 Sensor Size와 Pixel Pitch
- Object-side인지 Image-side인지

3.6 Image Space와 Object Space 주파수

배율 M 에서 Object의 길이 T_o 는 Image에서 $T_i=M \times T_o$ 가 된다. 따라서:

FORMULA · CALCULATION

Image-space frequency = Object-space frequency / M

Object-space frequency = $M \times$ Image-space frequency

0.5× Lens에서 Object Pattern이 50 lp/mm라면 Image Space에서는 $50/0.5=100$ lp/mm 가 필요하다. Lens MTF Chart 가 Image-side 기준이라면 100 lp/mm 성능을 확인한다.

3.7 PSF와 Blur

PSF(Point Spread Function)는 이상적인 점이 Image에서 얼마나 퍼지는지를 나타낸다. 실제 Image는 Object Detail이 PSF와 Convolution된 결과로 볼 수 있다.

Blur 원인:

- Lens 수차와 낮은 MTF
- Defocus
- Diffraction
- Motion Blur
- 진동
- Sensor Pixel Aperture
- 보호 Glass 또는 오염

서로 다른 Blur는 방향과 형태가 다르다. Motion Blur는 이동 방향으로 길어지고, Defocus는 대체로 여러 방향으로 퍼진다.

3.8 Focus와 Defocus

Focus가 벗어나면 Edge Transition 폭이 넓어지고 Gradient Peak가 낮아진다.

TEXT

In focus: 0 0 0 10 240 255 255
Defocused: 0 5 25 80 160 225 250

Threshold 위치와 Edge 측정값도 변할 수 있다. 선명도 Score 하나만 보지 말고 FOV 전체 Focus Map, 제품 높이 편차, Lens Tilt와 DOF를 확인한다.

3.9 Diffraction

빛의 파동 특성 때문에 완벽한 Lens도 점을 무한히 작게 맺을 수 없다. Image Space Airy Disk의 첫 번째 영점 지름 근사:

FORMULA · CALCULATION

$$\text{Airy Diameter} \approx 2.44 \times \text{wavelength} \times \text{F-number}$$

파장 $\lambda=0.55 \mu\text{m}$, $F/8$ 이면:

FORMULA · CALCULATION

$$2.44 \times 0.55 \times 8 \approx 10.74 \mu\text{m}$$

3 μm Pixel 기준 약 **3.58 Pixel** 지름이다. 이 값은 회절 크기 비교용이며 실제 Lens MTF, 파장 대역, 유효 F-number, 배율과 Pupil Magnification을 추가 고려해야 한다.

Object Space의 회절 한계를 직관적으로 볼 때 사용하는 Rayleigh 근사:

FORMULA · CALCULATION

$$d \approx 0.61 \times \text{wavelength} / \text{NA}$$

$\lambda=0.55 \mu\text{m}$, $\text{NA}=0.10$ 이면 약 **3.36 μm** , $\text{NA}=0.25$ 이면 약 **1.34 μm** 다. 실제 산업 영상의 검출/측정 한계와 동일한 숫자로 사용하지 말고 광학 설계 비교용으로 본다.

3.10 Motion Blur

Exposure 중 Object Image가 이동한 거리는 Blur가 된다.

FORMULA · CALCULATION

$$\text{Object Motion}(\mu\text{m}) = \text{Velocity}(\text{mm/s}) \times \text{Exposure}(\text{ms})$$

$$\text{Motion Blur}(\text{pixel}) = \text{Object Motion}(\mu\text{m}) / \text{Object Sampling}(\mu\text{m}/\text{pixel})$$

수치상 $\text{mm/s} \times \text{ms} = \mu\text{m}$ 이므로 단위 변환이 간단하지만 단위를 반드시 적는다.

50 mm/s, 2 ms Exposure, 5 $\mu\text{m}/\text{pixel}$ 이면:

FORMULA · CALCULATION

$$\text{Motion} = 50 \times 2 = 100 \mu\text{m}$$

$$\text{Blur} = 100 / 5 = 20 \text{ pixel}$$

정밀 Edge 검사에서 허용 Blur를 0.5 Pixel로 정하면 허용 Exposure는 훨씬 짧아진다.

3.11 Illumination Contrast와 Noise

같은 크기의 Defect도 조명 방식에 따라 거의 보이지 않거나 강한 Contrast로 나타난다.

- Dark Field: Scratch/Edge 산란광 강조
- Bright Field: 반사율 및 평탄한 표면 차이
- Back Light: 외곽 Silhouette와 Hole
- Coaxial: 평탄 반사면의 얼룩/Pattern

단순 Contrast만 아니라 Noise 대비 차이를 본다. 두 영역의 평균과 표준편차를 이용한 간단한 CNR:

FORMULA · CALCULATION

$$\text{CNR} = |\mu_{\text{defect}} - \mu_{\text{background}}| / \sqrt{\sigma_{\text{defect}}^2 + \sigma_{\text{background}}^2}$$

CNR이 높을수록 단순 Threshold로 분리하기 쉽지만, 위치별 Background 변화와 구조 Pattern도 False Positive를 만들 수 있다.

3.12 Detection과 Measurement

- **Detection:** Feature가 존재하는가?
- **Localization:** 어디에 있는가?
- **Measurement:** 크기·거리·폭이 얼마인가?

Feature가 약하게 한두 Pixel 값에 영향을 주면 Detection은 가능할 수 있지만 정확한 Width Measurement는 불가능할 수 있다. 검사 요구사항에서 무엇을 보장해야 하는지 구분한다.

3.13 3 μm Camera로 2 μm Defect를 검사할 수 있는가?

정답은 Pixel Size만으로 판단할 수 없다.

1× 배율

FORMULA · CALCULATION

$$\text{Object Sampling} = 3 \mu\text{m/pixel}$$

$$2 \mu\text{m Defect의 기하 점유} = 2/3 \approx 0.67 \text{ pixel}$$

Defect가 Pixel 평균값에 영향을 줄 수는 있지만 위치와 Contrast에 매우 민감하다. 안정적인 형상 측정은 어렵다.

2× 배율

FORMULA · CALCULATION

Object Sampling = $1.5 \mu\text{m}/\text{pixel}$

2 μm Defect의 기하 점유 $\approx 1.33 \text{ pixel}$

Sampling은 개선되지만 여전히 Lens가 Object의 2 μm Detail Contrast를 전달하는지, 조명이 Defect를 강조하는지, Noise 보다 신호가 큰지 확인해야 한다.

내부 기준이 4 Pixel이라면

FORMULA · CALCULATION

Required Sampling = $2 \mu\text{m} / 4 = 0.5 \mu\text{m}/\text{pixel}$

Required Magnification = $3 \mu\text{m} / 0.5 \mu\text{m} = 6\times$

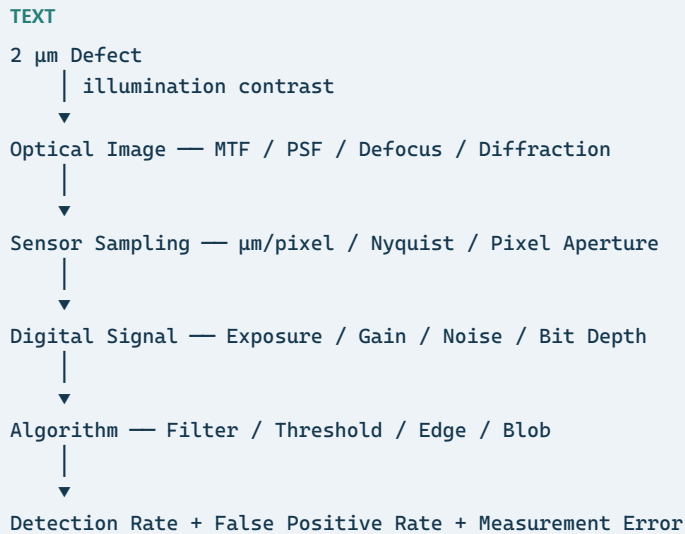
6×에서는 FOV와 DOF가 크게 줄고 높은 Object-side Optical Resolution이 필요하다. 일반적인 구성으로 가능한지 단정하지 말고 Microscope Objective/고NA 광학, 진동, Focus, 조명, Cycle Time을 함께 검토해야 한다.

올바른 판단 절차

1. Defect를 Detection할지 정확히 측정할지 정의한다.
2. FOV와 Sampling으로 Camera Pixel Count/배율을 계산한다.
3. Lens의 Object/Image-side MTF와 NA를 확인한다.
4. 조명으로 Defect CNR을 확보한다.
5. Exposure와 Motion Blur Budget을 정한다.
6. 실제 Defect Sample에서 위치·방향·Lot 변화를 포함해 검출률/오검률을 측정한다.

4. 그림으로 이해하기

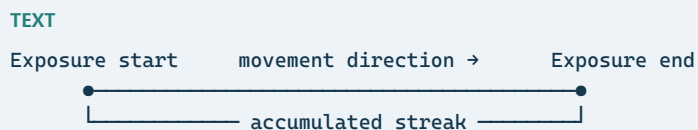
4.1 Resolution Chain



4.2 Sampling은 촘촘하지만 Blur가 큰 경우



4.3 Motion Blur



5. 실제 검사 장비에서 어디에 사용하는가?

Scratch 검사

Scratch 폭만으로 Camera를 선택하지 않는다. Dark Field가 Scratch 산란광을 충분히 만들고, Scratch 방향별 Contrast와 Lens MTF, Focus, Motion Blur를 확인한다.

Edge 치수 측정

Edge가 검출된다는 사실과 치수 정확도는 다르다. Edge Spread, Threshold/Gradient 위치 Bias, Distortion, Calibration 및 반복 정밀도를 Gauge로 검증한다.

이동 중 촬영

Line Scan 또는 Area Scan의 On-the-fly 촬영에서는 Exposure 동안의 이동량을 Pixel Blur Budget과 비교한다. 광량을 늘려 Exposure를 줄이거나 Strobe를 사용한다.

Focus 유지보수

장비 온도, Lens Mount, Stage 높이, 제품 Warpage로 Focus가 변할 수 있다. Golden Target의 Edge Width/Sharpness와 위치별 Focus Map을 예방보전 지표로 사용할 수 있다.

Camera 고해상도 Upgrade

기존 Lens가 새 Pixel Pitch에서 필요한 MTF를 전달하는지 확인한다. 처리량, Exposure, Noise, Sensor 크기와 Lens Image Circle도 함께 검토한다.

6. 숫자로 이해하기

예제 1: Sensor와 Object Space Nyquist

Camera Pixel Pitch $3\ \mu\text{m}$, Magnification $0.5\times$:

FORMULA · CALCULATION

Image-space Nyquist
 $= 1 / (2 \times 0.003\ \text{mm})$
 $= 166.67\ \text{lp/mm}$

Object Sampling
 $= 3 / 0.5 = 6\ \mu\text{m/pixel} = 0.006\ \text{mm/pixel}$

Object-space Nyquist
 $= 1 / (2 \times 0.006\ \text{mm})$
 $= 83.33\ \text{lp/mm}$

주파수 변환으로도:

FORMULA · CALCULATION

Object Nyquist = $M \times \text{Image Nyquist}$
 $= 0.5 \times 166.67$
 $= 83.33\ \text{lp/mm}$

예제 2: MTF와 Contrast

Object Pattern의 $I_{\text{max}}=180$, $I_{\text{min}}=20$ 이라면:

FORMULA · CALCULATION

$$\text{Object Contrast} = (180-20)/(180+20) = 0.8$$

해당 주파수에서 Optical MTF가 0.25라면 단순 선형 모델의 Image Contrast:

FORMULA · CALCULATION

$$0.8 \times 0.25 = 0.20$$

실제 Sensor Noise와 Processing을 포함하기 전의 1차 예상이다.

예제 3: Motion Blur Budget

조건:

- Stage Velocity = 50 mm/s
- Sampling = 5 $\mu\text{m}/\text{pixel}$
- 허용 Motion Blur = 0.5 pixel

허용 Object 이동:

FORMULA · CALCULATION

$$5 \mu\text{m}/\text{pixel} \times 0.5 \text{ pixel} = 2.5 \mu\text{m}$$

최대 Exposure:

FORMULA · CALCULATION

$$\begin{aligned} 2.5 \mu\text{m} / 50,000 \mu\text{m}/\text{s} &= 0.00005 \text{ s} \\ &= 50 \mu\text{s} = 0.05 \text{ ms} \end{aligned}$$

예제 4: Airy Diameter 비교

$\lambda=0.55 \mu\text{m}$ 일 때:

FORMULA · CALCULATION

$$F/4: 2.44 \times 0.55 \times 4 = 5.368 \mu\text{m}$$

$$F/8: 2.44 \times 0.55 \times 8 = 10.736 \mu\text{m}$$

$$F/16: 2.44 \times 0.55 \times 16 = 21.472 \mu\text{m}$$

3 μm Pixel 기준 Image Space 지름은 약 1.79/3.58/7.16 Pixel이다. 조리개를 조이면 DOF가 증가하지만 Diffraction Blur가 커지는 Trade-off를 보여준다.

예제 5: CNR

FORMULA · CALCULATION

Defect: mean=120, stddev=5

Background: mean=100, stddev=4

$$\begin{aligned} \text{CNR} &= |120-100| / \sqrt{5^2+4^2} \\ &= 20 / \sqrt{41} \\ &\approx 3.12 \end{aligned}$$

다른 조명에서 평균 차이가 10으로 줄고 Noise가 같다면 CNR은 약 1.56으로 절반이 된다.

7. C++ 구현

검출 가능성 1차 계산기

CPP · 1/2

```
#include <cmath>
#include <stdexcept>

struct ResolutionInput final {
    double sensorPixelPitchUm{};
    double magnification{};
    double featureSizeUm{};
    double velocityMmPerSecond{};
    double exposureMs{};
    double wavelengthUm{};
    double fNumber{};
};

struct ResolutionEstimate final {
    double objectUmPerPixel{};
    double featurePixels{};
    double imageNyquistLpPerMm{};
    double objectNyquistLpPerMm{};
    double motionUm{};
    double motionBlurPixels{};
    double airyDiameterImageUm{};
    double airyDiameterImagePixels{};
};

[[nodiscard]] bool IsPositiveFinite(const double value) noexcept
{
    return std::isfinite(value) && value > 0.0;
}

[[nodiscard]] ResolutionEstimate EstimateResolution(
    const ResolutionInput& input)
{
    if (!IsPositiveFinite(input.sensorPixelPitchUm) ||
        !IsPositiveFinite(input.magnification) ||
        !IsPositiveFinite(input.featureSizeUm) ||
        !std::isfinite(input.velocityMmPerSecond) ||
        input.velocityMmPerSecond < 0.0 ||
        !std::isfinite(input.exposureMs) || input.exposureMs < 0.0 ||
        !IsPositiveFinite(input.wavelengthUm) ||
        !IsPositiveFinite(input.fNumber)) {
        throw std::invalid_argument{"Invalid resolution parameter"};
    }
}
```

CPP · 2/2

```

const double objectUmPerPixel =
    input.sensorPixelPitchUm / input.magnification;
const double pixelPitchMm = input.sensorPixelPitchUm / 1000.0;
const double imageNyquist = 1.0 / (2.0 * pixelPitchMm);
const double objectNyquist = input.magnification * imageNyquist;

// Numerically, mm/s * ms equals um.
const double motionUm =
    input.velocityMmPerSecond * input.exposureMs;
const double airyDiameterUm =
    2.44 * input.wavelengthUm * input.fNumber;

return {
    objectUmPerPixel,
    input.featureSizeUm / objectUmPerPixel,
    imageNyquist,
    objectNyquist,
    motionUm,
    motionUm / objectUmPerPixel,
    airyDiameterUm,
    airyDiameterUm / input.sensorPixelPitchUm
};
}

[[nodiscard]] double CalculateCnr(
    const double defectMean,
    const double backgroundMean,
    const double defectStdDev,
    const double backgroundStdDev)
{
    if (!std::isfinite(defectMean) || !std::isfinite(backgroundMean) ||
        !std::isfinite(defectStdDev) || !std::isfinite(backgroundStdDev) ||
        defectStdDev < 0.0 || backgroundStdDev < 0.0) {
        throw std::invalid_argument{"Invalid CNR parameter"};
    }

    const double noise = std::hypot(defectStdDev, backgroundStdDev);
    if (noise == 0.0) {
        throw std::invalid_argument{"CNR is undefined for zero noise"};
    }
    return std::abs(defectMean - backgroundMean) / noise;
}

```

Unit Test

C++

```
#include <cassert>
#include <cmath>

[[nodiscard]] bool NearlyEqual(
    const double lhs,
    const double rhs,
    const double tolerance = 1e-6)
{
    return std::abs(lhs - rhs) <= tolerance;
}

void TestNyquistAndMotionBlur()
{
    const ResolutionInput input{
        3.0, 0.5, 24.0, 50.0, 2.0, 0.55, 8.0
    };
    const auto result = EstimateResolution(input);

    assert(NearlyEqual(result.objectUmPerPixel, 6.0));
    assert(NearlyEqual(result.featurePixels, 4.0));
    assert(NearlyEqual(result.imageNyquistLpPerMm, 166.666667));
    assert(NearlyEqual(result.objectNyquistLpPerMm, 83.333333));
    assert(NearlyEqual(result.motionUm, 100.0));
    assert(NearlyEqual(result.motionBlurPixels, 16.666667));
    assert(NearlyEqual(result.airyDiameterImageUm, 10.736));
}

void TestCnr()
{
    const double cnr = CalculateCnr(120.0, 100.0, 5.0, 4.0);
    assert(std::abs(cnr - 3.123475) < 1e-6);
}
```

코드에서 봐야 할 점

1. Sampling, Nyquist, Motion Blur, Diffraction 비교값을 서로 분리한다.
2. $\text{mm/s} \times \text{ms} = \mu\text{m}$ 관계를 주석과 변수명으로 명시한다.
3. `featurePixels` 를 최종 합격 판정으로 사용하지 않는다.
4. Airy Diameter는 Image Space 비교용 근사값이다.
5. 실제 합격 판정에는 Lens MTF Data, CNR, Focus/Distortion 및 Sample 검증 결과가 추가로 필요하다.

[!CAUTION]

이 계산기는 광학계의 검출 가능성을 보증하지 않는다. 불가능하거나 위험한 조건을 조기에 찾는 1차 Engineering Tool이다.

8. 실무에서 발생하는 문제

문제 1: Pixel 수는 충분하지만 Lens MTF 부족

- 증상: 작은 Pattern이 여러 Pixel에 걸려도 Contrast가 낮음
- 대응: 사용 주파수·F-number·Image Height에서 MTF 확인, 실제 Target 비교

문제 2: FOV 중앙만 Focus가 맞음

- 증상: 모서리 Match Score와 Edge Width 악화
- 원인: Lens Tilt, Field Curvature, Sensor Tilt
- 대응: FOV Focus Map, Mount 평행도, Corner MTF/Calibration 점검

문제 3: Exposure가 길어 Motion Blur 발생

- 증상: 이동 방향 Edge가 퍼지고 치수 Bias 증가
- 대응: Strobe/광량 증가, Exposure 단축, Stage 정지 촬영, 진동 억제

문제 4: 조리개를 조여 DOF만 확보

- 증상: Exposure 증가 또는 Diffraction으로 작은 Defect Contrast 저하
- 대응: DOF-Diffraction-광량을 함께 최적화

문제 5: 조명 변화로 CNR 저하

- 증상: Threshold 안정 구간이 사라지고 False Positive 증가
- 대응: 조명 Geometry, 전류/온도 안정화, Exposure Monitor, Adaptive 방법 검토

문제 6: Resize/Sharpen Image로 성능 평가

- 증상: Review에서는 선명해 보이나 원본 검출 성능과 불일치
- 대응: Raw/원본 기준 평가, Display Processing 분리

문제 7: Golden Sample만으로 검증

- 증상: 실제 Lot, 방향, 위치 변화에서 검출률 하락
- 대응: 정상/불량 Population, 경계 Defect, 위치·방향·높이·온도 변화 포함

9. 흔한 오해

Sampling Pitch일 뿐 Optical Resolution과 검출률을 포함하지 않는다.

1. “1 $\mu\text{m}/\text{pixel}$ 이면 1 μm Resolution이다.”

2 Sample 기준은 이상적인 주기 신호의 이론적 하한이다.

2. “Nyquist이므로 2 Pixel이면 Defect를 안정 검출한다.”

Sensor Sampling, Noise, Pixel Aperture와 Exposure가 함께 작동한다.

3. “Lens MTF가 높으면 Camera는 상관없다.”

Pixel 평균을 바꿀 수 있지만 위치-Contrast 의존성이 커 안정성 검증이 필요하다.

4. “Sub-pixel Defect는 절대 보이지 않는다.”

Diffraction과 광량 저하, Motion Blur가 증가할 수 있다.

5. “조리개를 조이면 Focus 문제를 모두 해결한다.”

Detection, Localization, Measurement는 요구 Sampling과 오차 예산이 다르다.

6. “검출되면 정확히 측정할 수도 있다.”

일부 Contrast를 강조할 수 있지만 소실된 정보와 불확실성을 없애지 못한다.

7. “후처리 Sharpening으로 Optical Blur를 복구할 수 있다.”

10. 면접에서 나올 수 있는 질문

Q1. 3 μm Pixel Camera로 2 μm Defect를 검사할 수 있나요?

초보자가 이해할 수 있는 답변 Pixel Size만으로는 알 수 없습니다. Lens 배율과 선명도, 조명이 만드는 Contrast, Noise와 흔들림을 함께 확인해야 합니다.

실무자 답변 먼저 배율을 적용해 Object $\mu\text{m}/\text{pixel}$ 과 Feature Pixel 수를 계산한다. 이후 해당 Object 주파수의 Lens MTF/NA, Focus, Illumination CNR, Sensor Noise, Motion Blur, Defect 위치·방향을 검토하고 목표 Detection Rate와 False Positive Rate로 Sample Test한다.

면접용 30초 답변 “3 μm 는 Sensor Pixel Pitch이므로 그것만으로 판단할 수 없습니다. 배율을 적용한 Object Sampling, Lens MTF와 Focus, 조명 CNR, Noise와 Motion Blur를 확인해야 합니다. 최종 가능 여부는 실제 2 μm Defect Sample의 검출률과 오검률로 결정합니다.”

Q2. Pixel Resolution과 Optical Resolution의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Pixel Resolution은 Image를 얼마나 촘촘히 나누는 지이고 Optical Resolution은 Lens가 실제 세부를 얼마나 선명하게 전달하는 지입니다.

실무자 답변 Object Sampling은 FOV/Pixel Count의 이산 간격이다. Optical Resolution은 MTF/PSF, NA, 파장, 수차와 Focus에 의해 제한된다. 전체 System Resolution은 Optical Transfer와 Sensor Sampling, SNR의 조합으로 평가한다.

면접용 30초 답변 “Pixel Resolution은 $\mu\text{m}/\text{pixel}$ 로 표현하는 Sampling 간격이고 Optical Resolution은 Lens가 공간 주파수 Contrast를 전달하는 능력입니다. Sampling이 촘촘해도 Lens가 Blur하면 Detail은 없고, Lens가 좋아도 Sampling이 부족하면 Alias가 생깁니다.”

Q3. Nyquist 기준을 검사에 어떻게 사용하나요?

초보자가 이해할 수 있는 답변 한 주기 패턴을 표현하려면 이론적으로 최소 2 Pixel이 필요하다는 하한으로 사용합니다.

실무자 답변 Sensor Nyquist $1/(2p)$ 와 Lens MTF 주파수를 비교해 Sampling 병목과 Alias Risk를 본다. 다만 단일 Defect Detection/Measurement에는 Contrast, PSF와 필요한 Pixel 수가 다르므로 Nyquist만으로 합격 판정하지 않는다.

면접용 30초 답변 “Nyquist는 $1/(2 \times \text{Sampling Pitch})$ 으로 이상적 주기 신호의 Alias-free 상한을 제공합니다. Lens MTF와 Sensor Sampling의 균형을 볼 때 사용하지만, 결함 검출에 2 Pixel이면 충분하다는 기준은 아니며 실제 CNR과 검출률을 추가 검증합니다.”

Q4. Motion Blur를 어떻게 계산하고 줄이나요?

초보자가 이해할 수 있는 답변 Exposure 동안 제품이 움직인 실제 거리를 $\mu\text{m}/\text{pixel}$ 로 나누면 Pixel Blur가 됩니다. Exposure를 줄이거나 제품을 멈추면 줄어듭니다.

실무자 답변 $\text{Blur pixel} = \text{velocity} \times \text{exposure} / \text{object sampling}$ 으로 1차 Budget을 계산한다. Strobe와 광량 증가, Exposure 단축, Trigger 동기화, Stage Settle, 진동 억제 및 이동 방향과 검사 Edge 방향을 함께 최적화한다.

면접용 30초 답변 "Object 이동량은 수치상 $\text{mm/s} \times \text{ms} = \mu\text{m}$ 이고 이를 $\mu\text{m}/\text{pixel}$ 로 나누면 Blur Pixel입니다. 허용 Blur에서 최대 Exposure를 역산한 뒤 Strobe, 조명 강화, Stage 정지 촬영과 Trigger 동기화로 맞춥니다."

Q5. 조리개를 조이면 어떤 Trade-off가 있나요?

초보자가 이해할 수 있는 답변 DOF는 늘어나는 경향이 있지만 Image가 어두워지고 너무 조이면 Diffraction으로 Detail이 흐려질 수 있습니다.

실무자 답변 F-number 증가로 수차와 Defocus 민감도가 줄 수 있지만 광량 감소로 Exposure/Gain이 증가하고 Diffraction MTF가 낮아진다. 유효 F-number와 Pixel Pitch, 목표 주파수에서 최적 Aperture를 실험한다.

면접용 30초 답변 "조리개를 조이면 DOF는 증가하지만 광량이 줄어 Exposure나 Gain이 필요하고, 너무 조이면 Diffraction Blur가 커집니다. 따라서 Focus 범위만 보지 않고 목표 공간 주파수의 MTF와 Motion Blur까지 포함해 최적 F-number를 정합니다."

Q6. 검사 가능성을 어떻게 검증하나요?

초보자가 이해할 수 있는 답변 계산 후 실제 정상·불량 Sample을 다양한 위치와 조건에서 촬영해 얼마나 잘 찾고 얼마나 자주 잘못 찾는지 확인합니다.

실무자 답변 FOV/Sampling과 Optical Budget을 계산하고 MTF, Focus Map, CNR, Motion Blur를 계측한다. 경계 Defect와 정상 변동을 포함한 Dataset에서 Detection Rate, False Positive Rate, Repeatability, Gauge R&R 및 환경 변화 Robustness를 평가한다.

면접용 30초 답변 "먼저 Sampling·MTF·CNR·Motion Budget으로 가능성을 선별하고, 실제 경계 Defect와 정상 Variation을 위치·방향·높이·속도 별로 촬영합니다. 그 결과의 검출률, 오검률, 반복 정밀도와 측정 Bias가 요구 기준을 만족하는지 확인합니다."

11. 실습 문제

실습 1: Nyquist와 MTF 연결

Camera Pixel Pitch $2.74 \mu\text{m}$, Lens 배율 $0.5\times$ 에 대해:

1. Image/Object Space Nyquist를 계산한다.
2. Object 60 lp/mm Pattern이 Image Space에서 몇 lp/mm인지 계산한다.
3. Lens MTF Chart에서 어떤 주파수와 Image Height를 확인해야 하는지 적는다.

실습 2: Motion Blur 역산

다음 조건에서 허용 Exposure를 구한다.

- Sampling: $3.5 \mu\text{m}/\text{pixel}$
- Stage 속도: 120 mm/s
- 허용 Blur: 0.3 pixel

계산 결과를 만족시키기 위한 조명/Trigger/기구 대안을 3개 제안한다.

실습 3: $2 \mu\text{m}$ Defect 검토표

다음 후보를 비교한다.

- Camera A: 3 μm Pixel, 1×
- Camera B: 3 μm Pixel, 2×
- Camera C: 2.2 μm Pixel, 4×

각 후보의 Sampling, 2 μm Feature Pixel 수, FOV Trade-off를 계산한다. 계산만으로 알 수 없는 MTF, NA, CNR, DOF, Motion 조건을 별도 열로 만든다.

실습 4: Blur 실험

Sharp Edge Image에 Gaussian Blur와 방향성 Motion Blur를 각각 적용한다.

- Edge Spread Width
- Sobel Gradient Peak
- Threshold Edge Position
- Canny 결과

Blur 크기별 변화를 기록하고 Detection과 Measurement 영향이 다른지 분석한다.

Phase 5 미니 프로젝트: Resolution Budget Analyzer

TEXT

Feature Requirement

- Size / Shape / Orientation
- Detection or Measurement
- Required Detection Rate

↓

Sampling / Nyquist

↓

Lens MTF / Diffraction / Focus

↓

Motion Blur / CNR / Noise

↓

Risk Report + Required Sample Tests

필수 기능

- Object $\mu\text{m}/\text{pixel}$ 과 Feature Pixel 계산
- Image/Object Nyquist 계산
- Motion Blur 및 허용 Exposure 역산
- Airy Diameter 비교값
- CNR 계산
- 계산만으로 합격을 반환하지 않고 **Need Optical Data** , **Need Sample Test** 상태 제공

12. Chapter 핵심 요약

- $\mu\text{m}/\text{pixel}$ 은 Sampling Pitch이며 실제 검출 Resolution과 다르다.
- Nyquist는 주기 신호의 이론적 하한이지 Defect 2 Pixel 보장 규칙이 아니다.
- Lens MTF와 PSF는 작은 Detail의 Contrast와 Blur를 결정한다.
- Sampling과 Optical Resolution 중 약한 쪽이 System Detail을 제한한다.
- Focus, Defocus, Diffraction, Motion은 서로 다른 Blur를 만든다.
- 조명이 Defect Contrast를 만들고 Noise 대비 CNR이 검출 안정성을 좌우한다.
- Detection 가능과 정확한 Measurement 가능을 구분한다.
- 3 μm Pixel Camera의 2 μm Defect 가능 여부는 Pixel Size만으로 판단할 수 없다.
- 조리개는 DOF, 광량, Diffraction의 Trade-off다.
- 최종 성능은 실제 정상/불량 Variation의 검출률·오검률·반복성으로 검증한다.

6일차 권장 학습 순서

- ☐ 40분: Sampling과 Optical Resolution 차이 설명
- ☐ 45분: Nyquist와 MTF 예제 계산
- ☐ 40분: PSF/Focus/Diffraction 정리
- ☐ 40분: Motion Blur와 최대 Exposure 역산
- ☐ 60~90분: C++ Resolution Analyzer 구현
- ☐ 60분: 2 μm Defect 후보 비교 실습
- ☐ 30분: 면접 Q1~Q6 30초 답변 연습

학습 완료 체크

- ☐ $\mu\text{m}/\text{pixel}$ 과 실제 Resolution 차이를 설명한다.
- ☐ Image/Object Space Nyquist를 계산한다.
- ☐ MTF와 PSF의 실무 의미를 설명한다.
- ☐ Motion Blur Pixel과 최대 Exposure를 계산한다.
- ☐ 조리개와 DOF/Diffraction Trade-off를 설명한다.
- ☐ 2 μm Defect 질문에 조건부로 답한다.
- ☐ Resolution Budget Analyzer를 설계하거나 구현했다.

다음 Chapter 예고

다음 Chapter에서는 Back/Ring/Coaxial/Dome/Dark Field/Bright Field 등 조명 Geometry가 어떤 특징의 Contrast를 만드는지 검사 알고리즘과 연결한다.

Chapter 7. Illumination: 조명으로 검사 특징 만들기

Phase 6 · 7일차 · 예상 학습 시간: 10~14시간 · 난이도: 입문→중급 · 중요도: ● 반드시 알아야 함

1. 이 Chapter에서 배우는 것

핵심 개념

- 조명은 Image를 밝게 하는 장치가 아니라 검사 특징의 Contrast를 만드는 광학계
- Bright Field와 Dark Field의 입사·반사 Geometry
- Back Light, Ring Light, Coaxial Light, Dome Light의 선택 기준
- Diffuse Light와 Directional Light의 차이
- 조명 방식과 Threshold/Edge/Blob/Pattern Matching의 연결
- Exposure, Gain, Saturation, Motion Blur의 Trade-off
- 조명 각도, Working Distance, 균일도, 편광(Polarization)
- 파장(Color), Strobe, LED 온도·열화와 재현성
- Histogram, Contrast, CNR을 이용한 후보 조명 비교

실무 목표

검사 대상과 결함의 반사 특성을 보고 조명 후보를 선택하며, “밝아 보이는 Image”가 아니라 검사 특징과 Background가 안정적으로 분리되는 Image를 고른다. 후보 조명을 동일한 Camera 조건과 위치에서 촬영해 CNR, Saturation, 균일도 및 알고리즘 결과로 비교한다.

2. 선수 지식

- Chapter 6의 MTF, Blur, CNR, Exposure
- Gray Level, Histogram, Saturation
- 반사, 투과, 산란의 직관적 의미
- Threshold, Edge, Blob, Pattern Matching의 역할

2.1 먼저 구분할 두 가지 분류

조명 장치 형태와 광학 Geometry는 같은 분류가 아니다.

- 장치 형태: Ring, Bar, Dome, Coaxial, Back Light
- Geometry: Bright Field, Dark Field, Directional, Diffuse

예를 들어 Ring Light도 설치 각도에 따라 Bright Field 또는 Dark Field에 가까운 효과를 만들 수 있다. “Ring Light니까 어떤 결함에 좋다”라고 장치 이름만으로 결정하지 않는다.

3. 핵심 개념

3.1 조명이 검사 알고리즘보다 먼저다

TEXT

Object의 물리적 차이
 | 조명 Geometry/파장/편광
 ▼
 반사·투과·산란 차이
 | Lens + Camera
 ▼
 Gray/Color Contrast
 | Algorithm
 ▼
 Threshold / Edge / Blob / Matching

정상과 불량에 Image에서 같은 Gray Level로 보이면 복잡한 알고리즘도 안정적으로 분리하기 어렵다. 반대로 조명으로 결함만 밝거나 어둡게 만들면 단순 Threshold와 Blob만으로 강력한 검사가 가능하다.

3.2 정반사, 난반사, 산란, 투과

- **Specular Reflection(정반사)**: 거울처럼 입사각과 반사각이 대응하는 방향으로 강하게 반사
- **Diffuse Reflection(난반사)**: 거친 표면에서 여러 방향으로 분산
- **Scattering(산란)**: Scratch, Particle, Edge 등에서 진행 방향이 다양하게 바뀜
- **Transmission(투과)**: 빛이 Object를 지나감

검사 핵심은 Camera가 어떤 빛을 받게 하고 어떤 빛을 받지 않게 할지 설계하는 것이다.

3.3 Bright Field

정상 표면의 주 반사광이 Lens로 들어오도록 조명과 Camera를 배치한다.

TEXT

Light → Surface — specular reflection → Camera

일반적 영상:

- 정상 평탄면: 밝음
- 빛을 산란시키는 Scratch/거칠기/Particle: 어둡거나 Contrast 발생

잘 맞는 대상:

- 평탄 금속/Glass의 얼룩과 반사율 차이
- 인쇄 Pattern, 표면 색/명암
- 밝은 Background 위의 Dark Defect

잘 맞는 알고리즘:

- Gray Threshold
- Background Difference
- Pattern Matching
- Local Contrast/Stain 검사

3.4 Dark Field

정상 표면의 주 반사광은 Lens 밖으로 보내고 Scratch, Edge, Particle에서 산란된 빛만 Lens에 들어오게 한다.

TEXT

Low-angle Light → flat surface → away from Camera
 ▲
 scratch/edge
 └ scattered light → Camera

일반적 영상:

- 정상 평탄면: 어두움
- Scratch, Particle, Edge: 밝음

잘 맞는 대상:

- 평탄 표면의 미세 Scratch
- Particle, 먼지, Burr
- 양각/음각 문자와 Edge

잘 맞는 알고리즘:

- Bright Blob Detection
- Top-hat/Local Threshold
- Line/Edge Detection
- 방향성 Filter

[!WARNING]

방향성 Dark Field는 조명 방향과 평행한 Scratch가 약하게 보일 수 있다. 여러 방향 Bar Light, Segment Ring 또는 촬영 다중화가 필요할 수 있다.

3.5 Back Light

Object 뒤에서 Camera 방향으로 빛을 비춘다. 표면 Texture보다 Silhouette를 만든다.

TEXT

Back Light → Object → Camera
 bright background / dark silhouette

강조 특징:

- 외곽 Edge
- Hole, Gap, Profile
- Width, Height, Diameter, Presence
- 투명체의 일부 내부 구조(조건에 따라)

일반적 영상:

- Background: 밝고 균일
- 불투명 Object: 어두움
- 경계: 높은 Contrast

잘 맞는 알고리즘:

- Threshold + Binary Image
- Edge Position
- Contour/Blob
- Diameter/Width/Gap Measurement

한계:

- Object 표면 Scratch/얼룩은 잘 보이지 않음
- Lens/조명 크기와 Object-Lamp 간격에 따라 Edge가 퍼질 수 있음
- Telecentric Back Light 또는 Collimated Light가 치수 정확도에 유리할 수 있음

3.6 Ring Light

Lens 주위를 둘러싼 원형 조명이다. 설치 높이와 LED 각도에 따라 효과가 달라진다.

- Low-angle Ring: Dark Field 효과, Edge/Scratch 강조
- High-angle Ring: 비교적 Bright Field, 일반 표면 관찰
- Diffuse Ring: Hot Spot 완화
- Segment Ring: 방향별 점등으로 형상 방향성 분석

잘 맞는 대상:

- 원형 또는 방향성이 다양한 부품
- 문자, 조립 상태, 일반 Presence
- 낮은 각도에서 Burr/Particle

주의:

- 반사면 중앙에 Ring 모양 Hot Spot
- 3D 형상에서 Shadow와 방향별 밝기 차이
- Lens FOV/WD 변화에 따른 균일도 변화

3.7 Coaxial Light

Beam Splitter를 통해 Lens Optical Axis와 거의 같은 방향으로 조명한다.

TEXT

```

LED → Beam Splitter ↓
      ↓ optical axis
      Flat Object
      ↑ reflection
      ↑ Camera

```

강조 특징:

- 평탄한 반사면의 Pattern, Mark, 얼룩
- PCB Pad, Wafer, Glass/Film 표면
- 정면 반사에서 생기는 밝기 차이

일반적 영상:

- Camera에 수직인 평탄면: 밝음
- Tilt, Scratch, 거친 영역: 빛이 벗어나 어둡거나 Contrast 발생

잘 맞는 알고리즘:

- Pattern Matching/OCR 전처리
- Threshold
- Background Difference
- Stain/Non-uniformity 검사

한계:

- 높이 차와 경사면은 어두워질 수 있음
- Beam Splitter 광손실
- 큰 FOV에서 균일도 확보가 어려울 수 있음

3.8 Dome Light

반구형 확산면의 여러 방향에서 Object를 비춘다. 반사면의 Hot Spot과 Shadow를 줄여 균일한 영상을 만든다.

강조 특징:

- 곡면, 주름진 포장, 광택 Plastic
- 불규칙 형상의 Color/Print
- 일반 조명에서 반사가 심한 표면의 균일한 외관

일반적 영상:

- 넓고 부드러운 반사
- Shadow 감소
- 미세한 방향성 Scratch Contrast는 약해질 수 있음

잘 맞는 알고리즘:

- Color/Gray Segmentation
- Pattern Matching/OCR
- 인쇄 누락/색상 검사

3.9 Diffuse Light와 Directional Light

구분	Diffuse Light	Directional Light
빛의 방향	여러 방향으로 분산	특정 방향이 강함
장점	Hot Spot/Shadow 완화, 균일도	Edge, 높이, Scratch 방향 강조
단점	미세 형상 Contrast가 약해질 수 있음	방향 의존, Shadow, 균일도 문제
대표 장치	Dome, Diffuser Panel	Bar, Low-angle Ring, Collimated Back Light
잘 맞는 검사	Print/Color/Pattern	Scratch/Burr/Edge/Height 변화

3.10 조명 선택 비교표

조명/Geometry	주 대상	강조되는 특징	예상 Image	잘 맞는 알고리즘
Back Light	외곽, Hole, Gap	Silhouette Edge	밝은 배경+어두운 Object	Threshold, Contour, Edge
High-angle Ring	일반 부품	표면/문자/조립	비교적 균일한 전면	Matching, Threshold
Low-angle Ring	평탄 표면	Scratch, Burr, Particle	어두운 배경+밝은 결함	Blob, Line, Edge
Coaxial	평탄 반사면	Mark, 얼룩, Tilt	수직면 밝음	Matching, Stain, Threshold
Dome	곡면/광택면	Print, Color, 전체 외관	Hot Spot/Shadow 감소	Segmentation, OCR, Matching
Directional Bar	형상/방향 결함	특정 방향 Edge/Scratch	방향성 Contrast/Shadow	Gradient, Line, Edge
Diffuse Panel	반사/표면 검사	넓은 균일도	부드러운 Contrast	Background, Threshold

3.11 Exposure, Gain, 조명 세기

- 조명 세기 증가: Signal 증가, Exposure 단축 가능
- Exposure 증가: Signal 증가, Motion Blur와 Cycle Time 위험
- Gain 증가: 밝아지지만 Noise도 증폭되고 Dynamic Range가 줄 수 있음
- Aperture 개방: 광량 증가, DOF/수차 변화

조명 후보를 비교할 때 Auto Exposure/Auto Gain을 끄고 동일 조건을 유지해야 한다. 그렇지 않으면 조명 Geometry가 아니라 Camera 자동 보정 결과를 비교하게 된다.

3.12 Saturation과 밝기 목표

8-bit Image에서 정상부가 255에 몰리면 더 밝은 차이를 잃는다. 반대로 Defect와 Background가 모두 0 근처면 Noise에 묻힌다.

실무에서는 다음을 함께 본다.

- Defect/Background 평균 차이
- 각 영역 표준편차와 CNR
- Saturated Pixel 비율
- FOV 균일도
- Lot/위치/방향 변화에서 분포 간격

3.13 Polarization

Light와 Lens에 Polarizer를 사용하고 방향을 교차시키면 직접 정반사를 줄일 수 있다.

- Cross Polarization: Glare 억제, Plastic/Film 아래 Print 관찰
- Parallel/특정 각도: 반사 특징 유지 또는 응력/재질 효과 관찰

한계:

- 광량 감소로 Exposure 증가
- 금속 반사와 비금속 표면에서 효과가 다름
- 곡면/복굴절 재질에서 예상과 다를 수 있음

3.14 파장(Color)

물체 색과 조명 파장의 조합으로 Contrast를 바꿀 수 있다.

- 같은 계열 파장: 해당 색이 상대적으로 밝게 보이는 경향
- 보색 계열: 해당 색이 상대적으로 어둡게 보이는 경향
- Blue: 짧은 파장으로 미세 산란/표면 특징에 유리할 수 있으나 재질 의존
- Red/IR: 일부 재질 투과, 표면 Pattern 억제 등에 유리할 수 있음

Color Camera를 쓰기 전에 Mono Camera+파장 선택으로 더 단순하고 강한 Contrast를 만들 수 있는지 검토한다.

3.15 Strobe, 온도, 열화

Strobe는 짧은 시간 강하게 발광해 Motion Blur를 줄일 수 있다. 확인할 항목:

- Pulse Width와 Exposure의 Timing
- Trigger Delay/Jitter
- Peak Current와 허용 Duty Cycle
- LED Controller의 Overdrive Limit
- 열에 따른 밝기와 파장 변화
- 장기 열화와 Camera 간 편차

조명 Parameter와 Controller Version을 Recipe에 저장하고, Warm-up 및 기준 Image 점검 절차를 둔다.

4. 그림으로 이해하기

4.1 Bright Field와 Dark Field

TEXT	
Bright Field	Dark Field
Light incidence surface reflection Camera 정상면 밝음, 결함이 어둡거나 변화	low-angle Light →→→ flat surface scratch \ scattered ↑ reflected normal light → away Camera receives scratch scattering 정상면 어두움, 결함/Edge 밝음

4.2 조명에서 알고리즘까지

TEXT
Geometry 선택 ↓ Defect Contrast 생성 ↓ Exposure/Gain으로 유효 범위 확보 ↓ Histogram/CNR/Uniformity 비교 ↓ Threshold / Edge / Blob / Matching ↓ Detection Rate / False Positive / Repeatability

4.3 방향성 Scratch

TEXT
Light A →→→ Scratch ——— 약한 산란 가능 Light B ↓↓↓ Scratch ——— 강한 산란 가능 한 방향 영상만으로 모든 Scratch 방향을 보장하지 않는다.

5. 실제 검사 장비에서 어디에 사용하는가?

외곽 치수 검사

Back Light로 고Contrast Silhouette를 만들고 Threshold/Edge로 Width와 Diameter를 측정한다. 광원 크기, Object와 Light 거리, Telecentricity가 Edge 위치 Bias에 영향을 줄 수 있다.

금속 표면 Scratch

Low-angle Dark Field로 Scratch 산란광을 밝게 만든다. 여러 방향 조명을 순차 점등하거나 Segment를 합성해 방향 의존성을 줄인 뒤 Line/Blob 검사를 수행한다.

PCB/반도체 Mark Align

Coaxial Light로 평탄 Pad/Mark Contrast를 만들고 Pattern Matching을 적용한다. 높이/기울기 변화로 Mark 밝기가 달라지는지 Recipe 범위에서 검증한다.

광택 포장 인쇄 검사

Dome+편광으로 Hot Spot을 줄이고 Color/Gray Segmentation 및 OCR을 적용한다. 주름과 제품 위치 변화까지 포함해 Background 균일성을 평가한다.

투명 Film Particle

Dark Field 또는 Back/Diffuse 조명 후보를 비교한다. Particle의 산란, Film Texture, Dust와 실제 Defect를 CNR/Blob 특징으로 분리한다.

6. 숫자로 이해하기

예제 1: 조명 후보 CNR 비교

동일한 Defect ROI와 Background ROI를 측정한다.

후보	Defect 평균/표준편차	Background 평균/표준편차	평균 차이	CNR
A	140 / 6	110 / 5	30	$30/\sqrt{61} \approx 3.84$
B	180 / 12	100 / 10	80	$80/\sqrt{244} \approx 5.12$
C	250 / 4	200 / 4	50	$50/\sqrt{32} \approx 8.84$

C가 CNR은 높지만 Defect나 정상부의 Saturation이 발생하는지 별도로 확인해야 한다. CNR 하나만으로 최종 선택하지 않는다.

예제 2: Exposure와 Motion Blur

조건:

- Object 속도: 40 mm/s
- Object Sampling: 4 $\mu\text{m}/\text{pixel}$
- 기존 Exposure: 1.5 ms

FORMULA · CALCULATION

$$\text{Motion} = 40 \text{ mm/s} \times 1.5 \text{ ms} = 60 \mu\text{m}$$

$$\text{Blur} = 60 \mu\text{m} / 4 \mu\text{m}/\text{pixel} = 15 \text{ pixel}$$

Strobe와 광량 증가로 Effective Exposure를 0.05 ms로 줄이면:

FORMULA · CALCULATION

$$\text{Motion} = 40 \times 0.05 = 2 \mu\text{m}$$

$$\text{Blur} = 2 / 4 = 0.5 \text{ pixel}$$

예제 3: Saturation 비율

2048×2048 Image에서 Gray 250 이상 Pixel이 83,886개라면:

FORMULA · CALCULATION

$$\text{Total} = 2048 \times 2048 = 4,194,304 \text{ pixel}$$

$$\text{High-level ratio} = 83,886 / 4,194,304 \times 100 \approx 2.00\%$$

이 Pixel들이 검사 ROI/기준면에 집중되어 있다면 전체 2%라도 문제가 될 수 있다. 위치 Mask와 함께 본다.

예제 4: Strobe Duty Cycle

Pulse Width **100 μs** , 촬영 빈도 **50 Hz** 라면:

FORMULA · CALCULATION

$$\begin{aligned} \text{Duty Cycle} &= 100 \mu\text{s} \times 50 / 1,000,000 \mu\text{s} \times 100 \\ &= 0.5\% \end{aligned}$$

허용 Peak Current와 Duty Cycle은 LED/Controller Datasheet를 따라야 하며 단순 계산만으로 Overdrive를 허용하지 않는다.

7. C++ 구현

조명 후보 영상 평가기

CPP · 1/2

```
#include <opencv2/opencv.hpp>

#include <cmath>
#include <stdexcept>

struct RegionStatistics final {
    double mean{};
    double standardDeviation{};
};

struct LightingMetrics final {
    RegionStatistics feature;
    RegionStatistics background;
    double absoluteContrast{};
    double cnr{};
    double lowClippingRatio{};
    double highClippingRatio{};
};

[[nodiscard]] RegionStatistics CalculateRegionStatistics(
    const cv::Mat& image,
    const cv::Mat& mask)
{
    if (image.empty() || image.type() != CV_8UC1) {
        throw std::invalid_argument{"8-bit mono image is required"};
    }
    if (mask.empty() || mask.type() != CV_8UC1 ||
        mask.size() != image.size() || cv::countNonZero(mask) == 0) {
        throw std::invalid_argument{"Non-empty 8-bit mask is required"};
    }

    cv::Scalar mean;
    cv::Scalar standardDeviation;
    cv::meanStdDev(image, mean, standardDeviation, mask);
    return {mean[0], standardDeviation[0]};
}

[[nodiscard]] double CalculateMaskedRatio(
    const cv::Mat& conditionMask,
    const cv::Mat& regionMask)
{
    cv::Mat intersection;
    cv::bitwise_and(conditionMask, regionMask, intersection);
    return static_cast<double>(cv::countNonZero(intersection)) /
        static_cast<double>(cv::countNonZero(regionMask));
}
```

CPP · 2/2

```

[[nodiscard]] LightingMetrics EvaluateLighting(
    const cv::Mat& image,
    const cv::Mat& featureMask,
    const cv::Mat& backgroundMask,
    const int lowClipLevel = 5,
    const int highClipLevel = 250)
{
    if (image.empty() || image.type() != CV_8UC1) {
        throw std::invalid_argument{"8-bit mono image is required"};
    }
    if (lowClipLevel < 0 || highClipLevel > 255 ||
        lowClipLevel >= highClipLevel) {
        throw std::invalid_argument{"Invalid clipping levels"};
    }

    const auto feature = CalculateRegionStatistics(image, featureMask);
    const auto background = CalculateRegionStatistics(image, backgroundMask);
    const double noise = std::hypot(feature.standardDeviation,
                                     background.standardDeviation);

    cv::Mat lowMask;
    cv::Mat highMask;
    cv::compare(image, lowClipLevel, lowMask, cv::CMP_LE);
    cv::compare(image, highClipLevel, highMask, cv::CMP_GE);

    cv::Mat evaluationMask;
    cv::bitwise_or(featureMask, backgroundMask, evaluationMask);

    return {
        feature,
        background,
        std::abs(feature.mean - background.mean),
        noise > 0.0 ? std::abs(feature.mean - background.mean) / noise : 0.0,
        CalculateMaskedRatio(lowMask, evaluationMask),
        CalculateMaskedRatio(highMask, evaluationMask)
    };
}

```

Unit Test

C++

```
#include <cassert>
#include <cmath>

void TestSeparatedRegions()
{
    cv::Mat image(100, 100, CV_8UC1, cv::Scalar(100));
    const cv::Rect featureRoi{20, 20, 20, 20};
    image(featureRoi).setTo(150);

    cv::Mat featureMask = cv::Mat::zeros(image.size(), CV_8UC1);
    cv::Mat backgroundMask = cv::Mat::zeros(image.size(), CV_8UC1);
    featureMask(featureRoi).setTo(255);
    backgroundMask(cv::Rect{60, 60, 20, 20}).setTo(255);

    const auto metrics = EvaluateLighting(image, featureMask, backgroundMask);
    assert(std::abs(metrics.absoluteContrast - 50.0) < 1e-9);
    assert(metrics.lowClippingRatio == 0.0);
    assert(metrics.highClippingRatio == 0.0);
    // Both synthetic regions have zero noise, so CNR is reported as 0/undefined.
    assert(metrics.cnr == 0.0);
}

void TestSaturationDetection()
{
    cv::Mat image(10, 10, CV_8UC1, cv::Scalar(100));
    image(cv::Rect{0, 0, 2, 10}).setTo(255); // 20% high clipping
    const cv::Mat mask(image.size(), CV_8UC1, cv::Scalar(255));

    const auto metrics = EvaluateLighting(image, mask, mask);
    assert(std::abs(metrics.highClippingRatio - 0.2) < 1e-9);
}
```

코드에서 봐야 할 점

1. Feature와 Background ROI를 동일 Image에서 명시적으로 분리한다.
2. 평균 차이뿐 아니라 각 영역 Noise를 포함한 CNR을 계산한다.
3. 전체 Image가 아니라 평가 ROI 내부의 Clipping 비율을 구한다.
4. CNR이 높아도 Saturation과 위치 균일도가 나쁘면 탈락할 수 있다.
5. 실제 비교에서는 동일 Exposure/Gain/Aperture와 동일 Sample 위치를 유지한다.
6. 정상/불량 여러 장의 분포와 알고리즘 검출률을 추가해야 한다.

8. 실무에서 발생하는 문제

문제 1: 조명 방향에 따라 Scratch가 사라짐

- 원인: 방향성 산란과 단일 Bar/Segment 사용
- 대응: 다방향 촬영, Segment 순차 점등, 방향별 결과 결합

문제 2: Auto Exposure로 후보 비교

- 원인: Camera가 각 조명 밝기를 자동 보정
- 대응: Exposure/Gain/Aperture 고정 후 Geometry와 CNR 비교

문제 3: LED 온도 상승으로 밝기 변화

- 증상: 장비 가동 직후와 안정화 후 Threshold 분포 변화
- 대응: Warm-up, 정전류 Controller, 온도/광량 Monitoring, 기준 Image

문제 4: 반사 Hot Spot과 Saturation

- 원인: Specular Surface와 조명/Camera Geometry
- 대응: Dome/Diffuser/Polarizer, 각도 변경, Exposure 조정

문제 5: FOV 중심과 가장자리 균일도 차이

- 원인: 조명 크기/거리, Vignetting, Coaxial/Dome 정렬
- 대응: Flat-field 평가, 조명 크기/WD 개선, Shading Correction 검증

문제 6: Strobe Timing 불일치

- 증상: Frame마다 밝기 변화 또는 일부 Frame 암전
- 대응: Exposure Active와 Pulse Timing 측정, Delay/Jitter/Width 검증

문제 7: 조명 교체 후 기존 Recipe 사용

- 원인: 같은 Model이라도 광량/파장/배치 편차 존재
- 대응: Optical Configuration ID, 재Calibration/재검증, 변경 이력

9. 흔한 오해

Defect와 Background의 분리, Saturation, Noise와 균일도가 더 중요하다.

1. **“Image가 밝을수록 검사하기 좋다.”**

LED 각도와 WD에 따라 Bright/Dark Field 효과가 달라진다.

2. **“Ring Light는 항상 같은 효과다.”**

Hot Spot은 줄지만 Scratch/미세 형상 Contrast도 약해질 수 있다.

3. **“Diffuse Light가 모든 반사를 해결한다.”**

Scratch 방향, 깊이, 표면과 조명 방향에 따라 산란이 달라진다.

4. **“Dark Field면 모든 Scratch가 밝다.”**

Motion Blur, Saturation, Cycle Time과 Dark/Read Noise 조건을 함께 본다.

5. “Exposure를 늘리면 Noise 문제를 모두 해결한다.”

재질/각도/곡률에 따라 효과가 다르고 광량이 감소한다.

6. “편광을 쓰면 광택 반사가 완전히 사라진다.”

Lot, 오염, LED 열화, 위치·높이 변화와 Camera Drift를 검증해야 한다.

7. “좋은 조명을 고르면 Threshold 하나로 영구히 검사된다.”

10. 면접에서 나올 수 있는 질문

Q1. 머신비전에서 조명이 중요한 이유는 무엇인가요?

초보자가 이해할 수 있는 답변 조명은 단순히 밝게 하는 것이 아니라 정상과 결함이 서로 다른 밝기로 보이게 만들어 알고리즘이 구분하도록 합니다.

실무자 답변 조명 Geometry, 파장, 편광이 Object의 정반사·난반사·산란·투과를 선택해 Feature CNR을 결정한다. 좋은 조명은 간단한 Threshold/Blob으로도 Robustness를 높이고, 잘못된 조명은 복잡한 알고리즘으로도 Variation을 분리하기 어렵게 한다.

면접용 30초 답변 “조명은 밝기를 공급하는 부품이 아니라 검사 특징의 Contrast를 만드는 Optical Component입니다. 정상과 불량에 반사·산란·투과 차이를 Camera에 유리하게 전달해 CNR을 높이면 Threshold나 Edge 같은 단순한 알고리즘도 안정적으로 동작합니다.”

Q2. Bright Field와 Dark Field의 차이는 무엇인가요?

초보자가 이해할 수 있는 답변 Bright Field는 정상 표면의 주 반사광을 Camera가 받아 배경이 밝고, Dark Field는 정상 반사를 피하고 결함의 산란광을 받아 결함이 밝게 보이게 합니다.

실무자 답변 Bright Field는 Specular 또는 주 반사 경로를 Lens에 결합하고, Dark Field는 정상면 반사를 Lens 밖으로 보내 Scratch/Particle/Edge의 산란만 수집한다. 대상 표면과 결함 방향에 따라 입사각과 다방향 조명을 결정한다.

면접용 30초 답변 “Bright Field는 정상면 반사광이 Lens로 들어오도록 해 평탄면이 밝게 보이고, Dark Field는 정상 반사를 피하면서 결함의 산란광만 받아 어두운 배경에 Scratch나 Particle을 밝게 만듭니다. 방향성 결함은 여러 조명 방향으로 검증합니다.”

Q3. Back Light는 어떤 검사에 사용하나요?

초보자가 이해할 수 있는 답변 물체 뒤에서 비춰 밝은 배경과 어두운 외곽을 만들기 때문에 폭, 직경, Hole과 유무 검사에 사용합니다.

실무자 답변 Back Light는 Surface Texture를 억제하고 고Contrast Silhouette를 만들어 Threshold/Contour/Edge 기반 Profile Measurement에 유리하다. Edge Telecentricity, 광원 균일도, Object-Light 거리와 투명체 투과를 검증한다.

면접용 30초 답변 “Back Light는 밝은 배경과 Object Silhouette를 만들어 외곽, Hole, Gap, Diameter 같은 Profile 검사에 적합합니다. Threshold와 Edge 처리가 단순해지지만 정밀 치수에서는 광원의 Collimation과 Edge Blur, Lens Perspective를 함께 확인합니다.”

Q4. Coaxial Light와 Dome Light를 언제 구분해 사용하나요?

초보자가 이해할 수 있는 답변 Coaxial은 평평한 반사면을 정면에서 보고, Dome은 곡면이나 광택 물체를 여러 방향에서 부드럽게 비출 때 사용합니다.

실무자 답변 Coaxial은 Optical Axis 방향으로 평탄 Specular Surface의 Mark/얼룩 Contrast를 만든다. Dome은 광범위한 입사각으로 곡면/주름의 Hot Spot과 Shadow를 줄인다. Coaxial은 Tilt에, Dome은 미세 방향성 형상 Contrast 저하에 주의한다.

면접용 30초 답변 "평탄 반사면의 Pad나 Mark라면 Coaxial을 우선 검토하고, 곡면·광택 포장처럼 Hot Spot과 Shadow가 문제라면 Dome을 검토합니다. 최종 선택은 동일 Camera 조건에서 Feature CNR과 균일도, 알고리즘 검출률로 비교합니다."

Q5. 조명 후보를 어떻게 정량 비교하나요?

초보자가 이해할 수 있는 답변 같은 Camera 설정에서 정상과 결함의 평균 밝기 차이, Noise, 포화와 화면 전체 균일도를 비교합니다.

실무자 답변 Exposure/Gain/Aperture와 Sample Pose를 고정하고 Feature/Background ROI의 분포와 CNR, Clipping, Uniformity를 측정한다. 여러 정상/불량 Variation에서 Detection Rate, False Positive Rate와 Parameter Margin을 평가한다.

면접용 30초 답변 "Auto 설정을 끄고 동일 조건으로 촬영한 뒤 Feature와 Background의 평균·표준편차로 CNR을 계산하고 Saturation과 FOV 균일도를 봅니다. 그 후 Lot·위치·방향이 다른 Sample에서 검출률, 오검률과 Threshold Margin을 비교해 선택합니다."

Q6. Strobe 조명 사용 시 무엇을 확인하나요?

초보자가 이해할 수 있는 답변 Camera Exposure 시간과 조명 Pulse가 정확히 겹치는지, 너무 강하거나 자주 켜 LED가 손상되지 않는지 확인합니다.

실무자 답변 Exposure Active 대비 Trigger Delay/Jitter/Pulse Width, Peak Current, Duty Cycle과 Thermal Limit을 확인한다. Effective Exposure로 Motion Blur Budget을 계산하고 Frame별 밝기 Repeatability와 Controller Error를 Monitoring한다.

면접용 30초 답변 "Strobe는 Motion Blur를 줄이지만 Exposure와 Pulse Timing이 정확히 겹쳐야 합니다. Delay-Jitter-Pulse Width를 계측하고, Peak Current와 Duty Cycle이 Controller/LED 사양 안인지 확인하며 Frame 밝기 반복성과 열화를 관리합니다."

11. 실습 문제

실습 1: 조명 선택표 만들기

다음 검사별로 1차/2차 후보 조명, 예상 Image, 알고리즘과 실패 Risk를 작성한다.

- 금속 평판 Scratch
- Plastic Cap 외경
- PCB Pad의 Mark
- 광택 포장의 인쇄 누락
- 투명 Film 위 Particle

실습 2: 동일 Sample 조명 비교

최소 세 조명 조건을 동일 Exposure/Gain/Aperture로 촬영한다.

- Feature/Background 평균·표준편차
- CNR
- Low/High Clipping 비율
- 중앙/모서리 균일도
- Threshold 가능 범위

숫자가 좋은 후보와 실제 알고리즘 결과가 일치하는지 확인한다.

실습 3: 방향성 결합 검증

Scratch Target을 0°, 45°, 90°, 135°로 회전하거나 조명 Segment를 변경한다. 방향별 CNR과 Line Detection 결과를 기록하고 단일 조명 방향의 사각지대를 찾는다.

실습 4: Strobe Budget

조건:

- 속도 80 mm/s
- Sampling 3 $\mu\text{m}/\text{pixel}$
- 허용 Blur 0.4 Pixel
- 촬영 40 fps

최대 Effective Exposure와 Pulse Duty Cycle을 계산한다. 필요한 광량과 Datasheet에서 확인할 항목을 적는다.

Phase 6 미니 프로젝트: Lighting Comparison Analyzer

TEXT

동일 Sample / Pose / Camera Parameter

↓

조명 후보별 Image Set

↓

Feature & Background ROI Statistics

↓

CNR / Clipping / Uniformity / Histogram

↓

Algorithm Test

↓

Detection Rate / False Positive / Margin

↓

Lighting Selection Report

필수 기능

- 후보명과 Exposure/Gain/Light Current 기록
- Feature/Background ROI Mask 저장
- Image별/후보별 평균과 분산
- CNR 및 Saturation/Underexposure 비율
- Threshold Sweep 결과
- 숫자만으로 자동 최종 선택하지 않고 실패 영상 Review 포함

12. Chapter 핵심 요약

- 조명은 검사 특징의 Contrast를 만드는 Optical Component다.
- 장치 형태와 Bright/Dark Field Geometry를 구분한다.
- Back Light는 Silhouette, 외곽, Hole과 치수 검사에 유리하다.
- Dark Field는 Scratch, Particle, Burr의 산란광을 강조한다.
- Coaxial은 평탄 반사면, Dome은 곡면·광택면의 균일 조명에 유리하다.
- Directional Light는 형상을 강조하지만 방향 사각지대가 있다.
- 조명 후보는 동일 Camera 조건에서 CNR, Clipping, 균일도로 비교한다.
- Exposure/Gain/조명 세기는 Motion Blur, Noise, Saturation과 Trade-off다.
- 편광과 파장 선택은 반사 및 색 Contrast를 크게 바꿀 수 있다.
- 최종 선택은 실제 Variation의 검출률·오검률과 유지보수 재현성으로 검증한다.

7일차 권장 학습 순서

- ☐ 40분: 반사·산란·투과와 Bright/Dark Field 정리
- ☐ 60분: 조명 7종 비교표를 직접 다시 작성
- ☐ 40분: 조명과 Threshold/Edge/Blob/Matching 연결
- ☐ 40분: Exposure-Gain-Strobe 계산
- ☐ 60~90분: C++ Lighting Metrics 구현
- ☐ 60분: 실제 검사 5종의 조명 선택 실습
- ☐ 30분: 면접 Q1~Q6 30초 답변 연습

학습 완료 체크

- ☐ Bright Field와 Dark Field를 광선 Geometry로 설명한다.
- ☐ Back/Ring/Coaxial/Dome의 적합 대상과 한계를 설명한다.
- ☐ Diffuse와 Directional Light의 Trade-off를 설명한다.
- ☐ 조명 방식과 영상처리 알고리즘을 연결한다.
- ☐ CNR, Saturation, Uniformity로 후보를 비교한다.
- ☐ Motion Blur 기준으로 Strobe/Exposure를 계산한다.
- ☐ Lighting Comparison Analyzer를 설계하거나 구현했다.

다음 Chapter 예고

다음 Chapter에서는 조명으로 확보한 Image를 Threshold, Histogram, Filter, Edge, Morphology, Blob으로 처리하는 전통 영상처리 Pipeline을 C++/OpenCV로 구현한다.